

Coral

1.7.9

Volume I · Linguagem e Desenvolvimento

Sintaxe, bibliotecas, ferramentas, instalação, arquitetura e referência consolidada

EDIÇÃO 1.7.9

Revisão de leitura QoL 1 · outubro de 2026

Português corrente controlado. Contratos verificáveis. Ferramentas integradas.

Coleção oficial: consulte o relatório de qualidade e pendências editoriais.

Como usar este livro

Este volume ensina Coral em 42 capítulos e inclui glossário, respostas e índice remissivo. Cada capítulo apresenta objetivos, revisão e prática. As seções de aprofundamento podem ser lidas depois. Código Coral, terminal e saída usam rótulos distintos. Trechos dentro dos parágrafos têm fundo próprio. O capítulo 8 explica o realce de leitura. As respostas aparecem junto dos exercícios e no Apêndice B.

Caixas laterais destacam notas, compatibilidade e limites de evidência. Exemplos em Coral são validados pelo compilador da própria árvore antes da release.

Sumário navegável

Como usar este livro 2

Sumário navegável 3

Coral 1.7.9 · Volume I 5

Sobre esta edição 5

PARTE I: COMEÇAR COM CORAL 6

1. O que é Coral 6

2. Instalando Coral 7

3. Linux 8

4. Outras plataformas suportadas 10

5. VS Code 10

6. Projetos Coral 13

7. Primeiro programa (passo a passo) 14

8. Como ler exemplos e fichas de API 16

PARTE II: FUNDAMENTOS DA LINGUAGEM 18

9. Valores e variáveis 18

10. Tipos e tipagem opcional 20

11. Operadores e expressões 22

12. Condições 23

13. Repetições 24

14. Coleções, índices e fatias 26

15. Funções e escopo 28

16. Classes e objetos 32

17. Verificando tipos na prática 36

18. Erros, padrões e contextos 41

19. Geradores 44

20. Assíncrono 45

21. Programa principal 46

PARTE III: BIBLIOTECA FUNDAMENTAL COM ESPECIFICAÇÕES 49

22. Entrada e saída 49

23. Conversões 54

24. Texto 59

25. Coleções 71

26. Arquivos e caminhos 75

27. JSON e serialização 78

28. Matemática, datas e aleatoriedade 94

29. Persistência 99

PARTE IV: DESENVOLVENDO APLICAÇÕES 106

30. Projetos, módulos e coral.toml 106

31. Testes 109

32. Regras, eventos e tempo 110

33. Mundo 116

34. RPG 128

35. Jogos 130

36. Espaço e geometria 143

37. Procedural 145

38. Computação numérica e científica 148

39. Gráficos 151

40. Web 162

41. Sistema e hardware 171

42. Laboratório 174

APÊNDICE A: Glossário 177

APÊNDICE B: Respostas dos exercícios 181

APÊNDICE C: Índice remissivo 190

APÊNDICE D: Migração desde a Coral 1.7.0 193

Migração para a Coral 1.7.4 193

Migração para a Coral 1.7.6 193

APÊNDICE E: Mapa de recursos novos 195

Coral 1.7.9 · Volume I

Coral: Linguagem e Desenvolvimento

Sobre esta edição

Esta coleção pode ser consultada sem internet. O Volume I ensina e o Volume II reúne a referência e a arquitetura. O site pode usar as mesmas fontes e fichas; os livros não dependem dele. Coleção oficial em dois volumes para o runtime 1.7.9. O relatório de validação registra as lacunas editoriais conhecidas.

PARTE I: COMEÇAR COM CORAL

1. O que é Coral

Objetivo de leitura

Entender o que é um programa e reconhecer uma instrução Coral.

Antes de começar

Nenhum conhecimento de programação é necessário.

Coral é uma linguagem de programação em português corrente controlado. Um programa é uma sequência de instruções que o computador executa. Antes de estudar instalação e ferramentas, veja um programa inteiro:

Código Coral

```
mostre "Olá, Coral"
```

Saída esperada

```
Olá, Coral
```

O comando mostra `Olá, Coral` e termina a linha. O texto entre aspas é um valor; `mostre` é a instrução que o exibe. No Cap. 7 você executa e modifica esse programa passo a passo.

A Coral é usada em scripts, projetos com vários arquivos, simulações, dados e aplicações. A escrita natural tem formas definidas: você aprende essas formas em exemplos e consulta seus contratos nas fichas. Não é preciso saber Python para iniciar; a integração com Python é apresentada no Volume II, Parte V.

Este volume foi organizado para estudantes, professores e pessoas que desejam construir programas reais. Siga os capítulos na ordem ou use o índice remissivo para consultar um recurso. As seções Para ir além aprofundam o tema sem interromper a primeira leitura.

O que você aprendeu

Um programa reúne instruções. Em `mostre "Olá, Coral"`, a instrução é `mostre` e o valor é o texto entre aspas.

Prática

Escreva um programa que mostre seu nome e o nome Coral em duas linhas.

Conferir a resposta comentada

Use duas instruções `mostre`, uma com o seu nome entre aspas e outra com `"Coral"`. Cada instrução termina sua saída com uma quebra de linha.

Resposta do capítulo 1 no Apêndice B.

2. Instalando Coral

Objetivo de leitura

Escolher um modo de instalação e preparar os arquivos necessários.

Antes de começar

Leia o capítulo 1; tenha acesso ao pacote de distribuição.

Há quatro caminhos oficiais, todos apoiados pela mesma base de runtime.

Coral Setup, recomendado para uso comum

Abra `Instalacao/Coral_Setup.cmd` no Windows ou `Instalacao/Coral_Setup.sh` em sistemas POSIX. A interface usa o contrato `coral.manager/1`: não existe um segundo instalador escondido na GUI.

Coral Manager, recomendado para automação e diagnóstico

Terminal

```
python Instalacao/Runtime/coral-1.7.6.pyz manager diagnosticar --json
python Instalacao/Runtime/coral-1.7.6.pyz manager instalar --runtime coral-1.7.6.pyz
  --perfil estudante --json
python Instalacao/Runtime/coral-1.7.6.pyz manager instalar --runtime coral-1.7.6.pyz
  --perfil estudante --aplicar --json
```

Operações mutáveis produzem primeiro um plano. A alteração é aplicada apenas quando o usuário pede explicitamente.

Runtime portátil

Terminal

```
python3 Instalacao/Runtime/coral-1.7.6.pyz --versao
python3 Instalacao/Runtime/coral-1.7.6.pyz meu_programa.coral
```

Desenvolvimento do compilador

Terminal

```
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
python -m coral --self-check
```

A instalação persistente continua disponível pelos assistentes em `Instalacao/Assistentes`.

Uso sem internet

O pacote completo pode ser transferido por outro computador, dispositivo de armazenamento ou rede local. O runtime portátil `Instalacao/Runtime/coral-1.7.6.pyz` usa o Python disponível e inclui o núcleo Coral. Na máquina alvo, execute `python3 Instalacao/Runtime/coral-1.7.6.pyz --versao` e depois um arquivo Coral.

Isso não significa que toda opção do Setup seja offline. Perfis com dependências opcionais exigem que os pacotes estejam previamente disponíveis; a instalação por pip com downloads não funciona desconectada. Para o editor, transfira também o VSIX da distribuição e instale o arquivo localmente. Consulte o Cap. 5.

A instalação nativa e os pacotes de sistema possuem requisitos próprios. Consulte o Volume II, Parte III, seção Instalação, antes de escolher um fluxo para máquinas sem conexão.

O que você aprendeu

Setup orienta a instalação comum; Manager permite automação; o runtime portátil executa arquivos. Dependências opcionais precisam ser obtidas separadamente quando usadas.

Prática

Escolha a instalação para uma máquina sem internet e identifique o que precisa transferir antes.

Conferir a resposta comentada

Transfira o pacote completo da release e, quando houver dependências opcionais, os pacotes dessas dependências para a plataforma alvo. O runtime portátil do pacote não precisa baixar dependências do núcleo. A instalação com pip só é offline quando todos os wheels necessários estão disponíveis.

Resposta do capítulo 2 no Apêndice B.

3. Linux

Objetivo de leitura

Instalar em Linux, conferir a versão e identificar componentes opcionais.

Antes de começar

Tenha o pacote da Coral, apresentado no capítulo 2.

3.1 Requisitos

O executável Linux x86_64 inclui o pacote Coral com Python e pip privados. O modo portátil continua aceitando Python 3.10 ou superior quando o usuário escolhe executar o PYZ diretamente.

3.2 Setup ou assistente

Para instalar a versão atual pelo launcher da distribuição, execute:

Terminal

```
bash Instalacao/Coral_Setup.sh
```

O pacote `Projeto/dist/nativos/coral-setup_1.7.0_amd64.deb` é histórico da 1.7.0 e não deve ser confundido com um instalador nativo novo da 1.7.5. Os launchers `Instalacao/Coral_Setup.sh` e os assistentes de terminal continuam disponíveis para o fluxo portátil e de desenvolvimento.

3.3 Validar

Terminal

```
coral --versao
coral --self-check
coral --gate-1-3
coral --gate-1-4
coral diagnostico --completo
```

3.4 VS Code

O VSIX corrente fica em `Instalacao/Extensao_VSCode/coral-language-0.77.0.vsix`. Instale pela interface **Extensions: Install from VSIX...** ou pelo CLI do editor.

3.5 Componentes opcionais

Use `coral componentes listar`, `coral componentes verificar` e `coral componentes instalar`. Perfis como estudante, científico, hardware, jogos e completo agrupam capacidades sem transformar dependências opcionais em requisitos do núcleo.

O que você aprendeu

Confira o runtime antes de executar exemplos. O núcleo portátil e os componentes opcionais têm requisitos diferentes; o diagnóstico ajuda a identificar o ambiente.

Prática

Confirme no terminal a versão de Coral e execute um arquivo local.

Conferir a resposta comentada

Execute `coral --versao` e depois `coral programa.coral`. Se usar o runtime portátil, execute `python3 Instalacao/Runtime/coral-1.7.6.pyz --versao` e substitua `coral` por essa chamada.

Resposta do capítulo 3 no Apêndice B.

4. Outras plataformas suportadas

Objetivo de leitura

Distinguir portabilidade do núcleo e validação de uma plataforma.

Antes de começar

Conheça os caminhos de instalação do capítulo 2.

Linux é a plataforma autoritativa da Coral 1.7.6. Os fluxos e scripts preservados para Windows não representam uma qualificação nativa dessa plataforma nesta release. macOS também não recebe, nesta edição, a mesma declaração de evidência Linux.

O núcleo é escrito em Python e possui caminho portátil, mas portabilidade do código não comprova funcionamento de instaladores, janelas e drivers num sistema específico. Para usar outra plataforma, consulte a referência de instalação e registre sua validação local. O Volume II, Parte V, distingue compatibilidade de evidência de campo.

Este capítulo informa o estado efetivo da release; não depende de suporte futuro para permitir o estudo em Linux.

O que você aprendeu

Linux é a plataforma autoritativa desta edição. Um script disponível em outro sistema não comprova sua execução nem o funcionamento de drivers e janelas.

Prática

Explique por que possuir um script Windows não comprova a qualificação daquela plataforma.

Conferir a resposta comentada

O script registra um fluxo de instalação, mas a qualificação exige sua execução na plataforma. A 1.7.5 considera Linux autoritativo e não promete evidência nativa Windows.

Resposta do capítulo 4 no Apêndice B.

5. VS Code

Objetivo de leitura

Selecionar o runtime no VS Code e usar execução, depuração e auxílio à escrita.

Antes de começar

Instale Coral e VS Code; consulte os capítulos 2 e 3.

A extensão oficial é **Coral Language 0.77.0**.

Instalar

Use `coral-language-0.77.0.vsix` da pasta `Instalacao/Extensao_VSCode`. Depois execute **Developer: Reload Window**.

Confirmar o runtime

Use **Coral: Mostrar runtime selecionado**. A extensão respeita configuração explícita e, na ausência dela, consulta a instalação persistente.

Executar

Coral: Executar arquivo no terminal usa o runtime selecionado e mantém entrada e saída no terminal.

Depurar com F5

Abra um arquivo `.coral`, coloque um breakpoint e pressione F5. O DAP suporta breakpoints no arquivo principal e em módulos `.coral` locais importados, pilha, variáveis, evaluate e step in/over/out.

Estudar o código

O Modo Aprender e o Coral Ensinar explicam o programa sem exigir alteração do código fonte. O estudo automático no editor é opcional e pode ser ligado ou desligado pelo usuário.

Componentes e diagnóstico

A extensão também oferece comandos para verificar suporte opcional, instalar componentes pelo gerenciador oficial, abrir o Coral Manager e diagnosticar a integração VS Code.

Reconhecer símbolos pelas cores

A Coral 1.7.4 informa ao editor o papel dos símbolos que consegue resolver: classes, tipos, parâmetros, propriedades, variáveis, funções, métodos e módulos. A aparência depende do tema do VS Code. Para ligar ou desligar esse recurso, use `coral.semanticColoring.enable`.

Em `dieta do tipo Dieta`, o primeiro nome é um parâmetro e o segundo é uma classe. Em `defina seu dieta como dieta`, `seu dieta` é uma propriedade e a última ocorrência é o parâmetro. A diferença ajuda a ler o código sem mudar sua execução.

Se o código estiver incompleto ou a análise não puder comprovar um vínculo, o realce básico continua disponível. Uma cor não comprova que o programa está correto.

Escrever blocos e completar nomes

A opção `coral.editor.autoCloseBlocks` insere um corpo indentado e `fim` alinhado ao pressionar Enter após uma abertura completa. O completamento usa o contexto para sugerir nomes. Confira a sugestão selecionada antes de aceitar com Tab.

Em um `coral.toml` vazio, o comando **Coral: Inserir estrutura básica do coral.toml** cria o manifesto inicial. O arquivo possui um ícone próprio que acompanha temas claros e escuros.

Exemplo: parâmetro, propriedade e classe

Código Coral

```
crie a classe Dieta
fim
crie a classe Pessoa
    ao criar uma Pessoa com dieta do tipo Dieta
        defina seu dieta como dieta
    fim
fim
defina plano como Dieta()
defina pessoa como Pessoa(plano)
mostre "Pessoa criada"
```

Saída esperada

```
Pessoa criada
```

No construtor, `Dieta` designa a classe do parâmetro `dieta`. A atribuição guarda esse argumento na propriedade do objeto. O editor diferencia esses papéis quando consegue resolver os símbolos.

Consultas de tipo e chamadas em bloco na 1.7.5

Depois de `animal for do tipo`, o autocomplete oferece tipos nativos e classes válidas disponíveis no escopo. O nome da classe recebe realce de tipo; quando seu vínculo é comprovado, hover apresenta a declaração e a hierarquia, e ir para definição abre a declaração.

Ao pressionar Enter depois de `chame alvo com`, o editor acompanha a indentação e o argumento corrente. O fechamento automático respeita `coral.editor.autoCloseBlocks`. A ajuda de assinatura também atende construtores e métodos. A tolerância do editor a uma chamada incompleta serve para continuar digitando; o compilador ainda exige o fechamento correto.

Documentos abertos prevalecem sobre as cópias no disco. Ao fechar o arquivo, o editor volta a considerar a versão salva. Alterações de interface revalidam os consumidores afetados; não é preciso renomear arquivos ou reiniciar o editor para obter a atualização normal do índice.

O que você aprendeu

Executar mostra o comportamento do programa; depurar permite acompanhar seu estado. Completamento e cores auxiliam a escrita, mas não comprovam correção.

Prática

Localize a configuração de runtime da extensão e compare executar com depurar.

Conferir a resposta comentada

Executar inicia o programa no terminal. Depurar por F5 também conecta o adaptador e permite observar pontos de parada, pilha e variáveis. O runtime escolhido precisa corresponder à versão da aplicação.

Resposta do capítulo 5 no Apêndice B.

6. Projetos Coral

Objetivo de leitura

Criar um projeto a partir de um template e reconhecer seus arquivos principais.

Antes de começar

Consiga executar o comando `coral` no terminal.

Coral oferece templates oficiais:

Template	Objetivo
basico	Ponto de partida mínimo
terminal	Aplicativo interativo de terminal
jogo-2d	Projeto usando `coral.jogos`
aventura	Mundo, RPG e Regras Coral
simulacao	Estados e entidades
didatico	Exemplos graduais

Liste e crie:

Terminal

```
coral templates
coral novo MeuProjeto --template basico
cd MeuProjeto
coral executar
```

Projetos usam `coral.toml` para identificar entrada, versão e módulos. O VS Code oferece **Coral: Novo Projeto** e um Project Explorer próprio.

Fluxo recomendado de desenvolvimento

Referência para leitura

```
Editar .coral
   ↓
coral verificar / diagnósticos do LSP
   ↓
coral testar
   ↓
F5 / Debug Adapter
   ↓
coral naturalidade .
   ↓
coral documentar
```

O que você aprendeu

Um projeto organiza entrada, módulos e testes. O arquivo `coral.toml` registra sua configuração; editar, verificar e testar são etapas distintas.

Prática

Separe modelos e aplicação em arquivos diferentes e identifique a entrada no projeto.

Conferir a resposta comentada

Coloque classes e funções reutilizáveis em `modelos.coral` e importe os símbolos necessários em `principal.coral`. Configure o arquivo de entrada no `coral.toml`; esse caminho é independente do bloco opcional `programa principal`.

Resposta do capítulo 6 no Apêndice B.

7. Primeiro programa (passo a passo)

Objetivo de leitura

Salvar, executar e modificar seu primeiro programa.

Antes de começar

Tenha um runtime instalado e um editor de texto.

Salvar e executar

Crie um arquivo chamado `primeiro.coral` no editor e copie o programa abaixo. Salve o arquivo. Depois abra o terminal na pasta em que ele foi salvo e execute:

Terminal

```
coral primeiro.coral
```

Se estiver usando o runtime portátil, substitua `coral` pelo comando apresentado no capítulo 2. O bloco abaixo é o conteúdo do arquivo, não um comando para colar diretamente no terminal.

O programa completo

Código Coral

```
defina idade como 17

defina mensagem como ""

se idade for maior ou igual a 18 então
    defina mensagem como "maior de idade"
senão
    defina mensagem como "menor de idade"
fim

mostre mensagem
```

Saída esperada

```
menor de idade
```

- `defina` cria ou atualiza um nome.
- `como` separa o nome do valor.
- `se ... então` abre uma condição.
- `for maior ou igual a` é uma comparação escrita como português.
- `senão` fornece o caminho alternativo.
- `fim` encerra o bloco.
- `mostre` produz saída.

Experimente:

Terminal

```
coral explicar primeiro.coral
coral ensinar primeiro.coral
coral naturalidade primeiro.coral
```

`explicar`/Modo Aprender **executa** e rastreia valores. `ensinar` faz análise estática e não executa o programa. `naturalidade` verifica estilo e idioma da superfície Coral; não é uma prova de correção lógica.

O que você aprendeu

Salve o código em um arquivo `.coral` e execute esse arquivo. Aspas delimitam textos; `fim` encerra o bloco ao qual pertence.

Prática

Altere a mensagem do primeiro programa e preveja a saída.

Conferir a resposta comentada

Troque apenas o texto entre aspas. O comando `mostre` imprime o novo texto; as aspas não aparecem na saída.

Resposta do capítulo 7 no Apêndice B.

8. Como ler exemplos e fichas de API

Objetivo de leitura

Distinguir explicação, código, terminal, saída e contrato de uma função.

Antes de começar

Reconheça um exemplo simples do capítulo 7.

O que escrever e o que observar

A Coral usa palavras em português, mas segue formas definidas. A frase de uma explicação e a instrução de um programa têm papéis diferentes. Neste livro, trechos como `defina pontos como 10` recebem fonte própria e fundo discreto, mesmo dentro de um parágrafo.

Os exemplos em bloco possuem rótulos:

Rótulo	Como usar
Código Coral	Escreva ou copie em um arquivo `.coral`
Terminal	Execute no terminal, na pasta indicada
Saída esperada	Compare com o resultado; não copie para o programa
Configuração	Salve no arquivo de configuração indicado
Assinatura para consulta	Consulte os argumentos; não execute a assinatura como programa

Quando um exemplo não imprime texto, o livro informa isso em uma nota. O programa ainda pode criar valores, salvar arquivos ou alterar estado.

Ler o realce do código

Nos blocos Coral, palavras da linguagem, tipos e valores especiais recebem negrito. Instruções como `defina` se distinguem de nomes como `pontos`, escolhidos pelo programador. Aspas identificam textos; números mantêm sua escrita literal; comentários aparecem em tom discreto. A fonte monoespaçada, o fundo creme e o rótulo Código Coral distinguem o programa da explicação. O vermelho da marca aparece na faixa de anéis, sem reduzir o contraste do texto.

O realce do livro reconhece a escrita dos tokens. Ele não descobre o tipo de todo objeto nem valida o programa. A extensão do editor pode acrescentar informação semântica quando consegue resolver os símbolos.

Ler uma assinatura

Uma assinatura mostra como chamar uma função. Em `inteiro(valor, padrao=<ausente>)`, `valor` é obrigatório e `padrao` é opcional. Na chamada `inteiro("12")`, o texto `"12"` é o argumento. O retorno é o valor devolvido, neste caso `12`.

Termo	Significado
Parâmetro	Nome previsto na declaração da função
Argumento	Valor fornecido em uma chamada
Valor padrão	Alternativa usada quando o argumento opcional é omitido
Retorno	Valor que a operação devolve ao programa

As assinaturas extraídas do runtime usam notação Python para mostrar o contrato completo. `None` corresponde ao valor `nulo` da escrita Coral. `<ausente>` significa que o argumento não foi fornecido, o que é diferente de fornecer `nulo`. `*` separa parâmetros que precisam de nome; `/` identifica os apenas posicionais; `*partes` aceita vários argumentos posicionais.

Consultar uma ficha sem interromper o estudo

Primeiro entenda a intenção e o exemplo do capítulo. Depois consulte a ficha para conferir assinatura, tipos, obrigatoriedade, valores padrão, retorno, erros e observações. As fichas permanecem completas; no HTML, você pode abrir a seção de consulta quando precisar.

Uma forma natural é uma expressão oficial que corresponde a uma operação. `converta "12" para inteiro`, por exemplo, corresponde a `inteiro("12")`. A Coral não tenta adivinhar qualquer frase em português.

No HTML, ajuste fonte, largura e tema na barra de leitura. O botão Copiar copia apenas o conteúdo do bloco. As respostas ficam recolhidas para você tentar a atividade antes de conferir. No PDF, respostas e aprofundamentos aparecem por inteiro.

O Apêndice A reúne termos; o Apêndice C ajuda a localizar recursos. O Volume II contém a referência normativa e a arquitetura.

O que você aprendeu

Código é o que você escreve; saída é o que observa. Parâmetro é o nome declarado; argumento é o valor fornecido; retorno é o valor devolvido pela operação.

Prática

Diferencie parâmetro, argumento, padrão e retorno em uma ficha.

Conferir a resposta comentada

Parâmetro é o nome previsto pela função; argumento é o valor fornecido numa chamada; padrão é o valor usado quando um argumento opcional é omitido; retorno é o valor devolvido ao chamador.

Resposta do capítulo 8 no Apêndice B.

PARTE II: FUNDAMENTOS DA LINGUAGEM

9. Valores e variáveis

Objetivo de leitura

Criar uma variável, alterar seu valor e mostrar o resultado.

Antes de começar

Conheça `mostre`, apresentado no capítulo 7.

Guardar um valor

Uma variável associa um nome a um valor. Podemos usar o nome `pontos` para guardar a pontuação de um jogador. Para começar com dez pontos, escrevemos:

Código Coral

```
defina pontos como 10
```

Este exemplo não imprime texto no terminal.

Leia a instrução por partes:

Trecho	O que significa
`defina`	Inicia a instrução de atribuição
`pontos`	É o nome que receberá o valor
`como`	Separa o nome do valor
`10`	É o valor inicial

O nome `pontos` foi escolhido por nós. Você pode usar outro nome, desde que o escreva da mesma maneira nas instruções seguintes. Guardar um valor não significa mostrá lo no terminal.

Alterar e mostrar

Agora vamos acrescentar cinco pontos e retirar dois. A instrução `mostre pontos` exibe o valor que ficou guardado.

Arquivo completo de apoio:
`Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina pontos como 10
adicione 5 a pontos
subtraia 2 de pontos
mostre pontos
```

Saída esperada

```
13
```

O programa executa as instruções em ordem:

Depois da instrução	Valor de `pontos`
`defina pontos como 10`	`10`
`adicione 5 a pontos`	`15`
`subtraia 2 de pontos`	`13`

Atenção ao nome e ao texto `mostre pontos` exibe o valor da variável. Já `mostre "pontos"` exibe a palavra entre aspas.

Atribuir outro valor

Se `pontos` já existir, `defina pontos como 20` substitui seu valor atual por `20`. A forma alternativa `pontos recebe 20` também faz atribuição.

Um nome precisa existir antes de ser consultado. Neste exemplo, a primeira instrução define `pontos`; as outras usam esse nome. Mais adiante, você verá nomes introduzidos por parâmetros de funções e por repetições.

O que você aprendeu

O nome permite consultar um valor. As instruções são executadas em ordem: definir 10, adicionar 5 e subtrair 2 deixa 13.

Prática

Troque o valor inicial de `10` por `20` no exemplo de pontos. Antes de executar, preveja o resultado.

Conferir a resposta comentada

A saída será `23`: o programa começa com vinte pontos, adiciona cinco e retira dois. A ordem das instruções continua sendo a mesma.

Resposta do capítulo 9 no Apêndice B.

10. Tipos e tipagem opcional

Objetivo de leitura

Reconhecer tipos de valor e escrever anotações opcionais sem confundi las com conversão.

Antes de começar

Conheça variáveis e listas simples; consulte o capítulo 9.

Tipos de valor

Categoria	Exemplos	Observação
Inteiro	`0`, `19`, `-3`	Contagens e valores sem parte fracionária
Decimal	`2.5`, `0.01`	Usa ponto decimal no código
Texto	`"Coral"`, `'Ana'`	Preserva os espaços dentro das aspas
Booleano	`verdadeiro`, `falso`	Condições e resultados lógicos
Ausência	`nulo`	Representa ausência de valor
Coleções	`[1, 2]`, `{"nome": "Ana"}`	Agrupam vários valores

A execução usa objetos Python. Por isso, a impressão direta de certos valores segue sua representação, como `True`, `False` e `None`. Isso não muda a grafia recomendada no código Coral.

Tipagem opcional

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
defina idade do tipo inteiro como 19
crie a função dobro com numero do tipo inteiro retornando inteiro
    retorne numero vezes 2
fim
mostre dobro(idade)
```

Saída esperada

```
38
```

Uma anotação informa o tipo que você espera usar. Ela registra essa intenção para a análise e para o editor. A Coral continua dinamicamente tipada; uma anotação não converte automaticamente os dados nem garante a rejeição estática de toda incompatibilidade. Use conversão explícita quando o dado chega como texto.

Tipos reconhecidos nas anotações incluem `inteiro`, `decimal`, `texto`, `booleano`, `lista`, `dicionário`, `conjunto`, `tupla`, `qualquer` e nomes de classes. Os tipos concretos e as operações seguem os objetos usados em execução.

Coleções parametrizadas

As quatro formas estruturais novas são listas, conjuntos, dicionários e tuplas de posições fixas.

Código Coral

```
defina notas do tipo lista de decimal como [8.0, 9.5]
defina tags do tipo conjunto de texto como {"coral", "linguagem"}
defina idades do tipo dicionário de texto para inteiro como {"Ana": 19}
defina pessoa do tipo tupla de (texto, inteiro) como ("Ana", 19)
```

Este exemplo não imprime texto no terminal.

Tipos podem ser compostos. Por exemplo, `dicionário de texto para lista de inteiro` representa um dicionário cujos valores conhecidos são listas de inteiros.

Para ir além

Nulidade em tipos compostos

A forma `T ou nulo` indica que o valor pode ter o tipo informado ou ser nulo.

Código Coral

```
defina apelido do tipo texto ou nulo como nulo
```

Este exemplo não imprime texto no terminal.

A posição dos parênteses importa. `lista de inteiro ou nulo` é uma lista que também pode ser nula. `lista de (inteiro ou nulo)` é uma lista cujos elementos podem ser nulos.

A 1.7.3 não introduz uniões gerais. Formas como `inteiro ou texto` continuam fora do contrato.

Verificação conservadora

As anotações ajudam a análise estática e o editor. Elas não inserem coerções nem validações profundas em tempo de execução. Quando a Coral não consegue provar o tipo de um valor, a informação permanece desconhecida em vez de produzir uma certeza inventada.

Coleções mutáveis são tratadas de modo conservador. Uma `lista de inteiro` não é assumida compatível com `lista de decimal` apenas porque os tipos numéricos individuais possam ter relações úteis em outros contextos.

O que a análise consegue comprovar

A tipagem permanece opcional. Na 1.7.4, a análise também usa contratos públicos de módulos Coral importados no projeto. Uma assinatura conhecida permite conferir argumentos e retorno, inclusive em construtores e métodos.

Esses diagnósticos continuam conservadores: um tipo desconhecido não é tratado como incompatível. Código Python importado e atributos sem informação suficiente podem

permanecer sem conclusão. As advertências de tipos ajudam a revisar o programa; não substituem os testes.

O que você aprendeu

A anotação comunica a intenção e ajuda a análise. Ela não converte valores nem valida profundamente todos os dados durante a execução.

Prática

Diferencie uma lista que pode ser nula de uma lista cujos itens podem ser nulos.

Conferir a resposta comentada

`lista de inteiro ou nulo` permite nulo no valor externo; `lista de (inteiro ou nulo)` permite nulo dentro da lista. Essas anotações não convertem nem validam profundamente os valores em execução.

Resposta do capítulo 10 no Apêndice B.

11. Operadores e expressões

Objetivo de leitura

Combinar valores em expressões e controlar a ordem das operações.

Antes de começar

Conheça valores numéricos e variáveis, dos capítulos 9 e 10.

Operadores combinam valores. `mais`, `menos`, `vezes` e `dividido por` são formas de leitura corrente. Operadores simbólicos continuam disponíveis, especialmente `%`, `//` e `**`.

Intenção	Forma Coral	Forma simbólica
Soma	`a mais b`	`a + b`
Subtração	`a menos b`	`a - b`
Multiplicação	`a vezes b`	`a * b`
Divisão	`a dividido por b`	`a / b`
Igualdade	`a for igual a b`	`a == b`
Diferença	`a for diferente de b`	`a != b`
Ordem	`a for maior que b`	`a > b`
Limite incluso	`a for menor ou igual a b`	`a <= b`
Pertencimento	`a estiver em valores`	Forma natural recomendada

Não use `=` como teste de igualdade. Ele aparece em argumentos nomeados, enquanto comparações usam `for igual a` ou `==`.

O que você aprendeu

Uma expressão produz um valor. Use parênteses para explicitar a ordem; comparação usa `for igual a` ou `==`, e não `=`.

Prática

Calcule o dobro da soma de 3 e 4, preservando a ordem desejada.

Conferir a resposta comentada

Use `(3 mais 4) vezes 2`. O resultado é 14. Parênteses tornam a ordem explícita.

Resposta do capítulo 11 no Apêndice B.

12. Condições

Objetivo de leitura

Escolher um caminho com condições e combinar perguntas lógicas.

Antes de começar

Conheça expressões e comparações do capítulo 11.

Escolher um caminho

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina media como 7
se media for maior ou igual a 7 então
    mostre "Aprovado"
senão se media for maior ou igual a 5 então
    mostre "Recuperação"
senão
    mostre "Reprovado"
fim
```

Saída esperada

```
Aprovado
```

O programa testa os ramos em ordem e executa apenas o primeiro cuja condição for verdadeira. `senão` cobre os demais casos e é opcional. `fim` fecha a decisão inteira. A indentação comunica a estrutura, mas o encerramento explícito é indispensável.

Combinar condições

Use `e`, `ou` e `não`. Prefira parênteses ao combinar comparações com negação: `não (idade for menor que 18)`. O parser não interpreta toda frase possível em português; siga uma forma reconhecida.

A precedência implementada, da menor para a maior, é: `ou`, `e`, comparação, soma e subtração, produto e divisão, potência, operadores unários e operações de chamada, atributo e índice. Não presuma que todo caso de precedência seja idêntico ao Python; use parênteses para tornar explícita sua intenção, principalmente com potência e sinal negativo.

O que você aprendeu

A decisão testa os ramos em ordem e executa o primeiro verdadeiro. `senão` trata os demais casos; `fim` fecha a decisão.

Prática

Altere a média do exemplo para 6 e para 4; acrescente rejeição de valor negativo.

Conferir a resposta comentada

Para 6, o ramo de recuperação é escolhido; para 4, o de reprovação. Antes da comparação de aprovação, teste se a média é menor que zero e informe valor inválido.

Resposta do capítulo 12 no Apêndice B.

13. Repetições

Objetivo de leitura

Repetir instruções, percorrer intervalos e garantir o encerramento de um laço.

Antes de começar

Conheça variáveis e condições, dos capítulos 9 e 12.

Intervalos e condição de continuidade

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina total como 0
para numero de 1 até 4 faça
    adicione numero a total
fim
mostre total
defina restante como 3
enquanto restante for maior que 0 faça
    subtraia 1 de restante
fim
mostre restante
```

Saída esperada

```
10
0
```

`para numero de 1 até 4 faça` inclui o limite final: o total é 1 + 2 + 3 + 4. `até antes de 4` excluiria 4. O laço `enquanto` repete enquanto a condição permanece verdadeira; ele precisa alterar algum estado ou ter outra forma de encerramento quando a condição depende desse estado.

`para cada item em colecao faça` percorre os elementos de uma coleção ou de outro iterável. `repita 3 vezes` repete um bloco por quantidade, sem exigir que você mantenha o contador manualmente.

Interromper e pular uma passagem

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
para n de 0 até antes de 5 faça
    se n for igual a 1 então
        continue
    fim
    se n for igual a 3 então
        pare
    fim
    mostre n
fim
repita 2 vezes
    mostre "Coral"
fim
```

Saída esperada

```
0
2
Coral
Coral
```

`continue` pula o restante da passagem atual. `pare` encerra o laço mais próximo. Ambos exigem um contexto de repetição. O último bloco demonstra repetição por quantidade. Não escreva `pare` para encerrar uma função; nesse caso, use `retorne`.

Pensar na terminação

Antes de executar um `enquanto`, responda: qual estado muda? Qual condição permite sair? O corpo pode deixar de alcançar essa mudança? Isso evita laços infinitos. Para interromper um programa de terminal que não termina, use Ctrl+C.

O que você aprendeu

`até` inclui o limite; `até antes de` o exclui. Em `enquanto`, identifique o estado que muda e a condição que permite terminar.

Prática

Mostre os inteiros de 0 até antes de 3 e explique a terminação.

Conferir a resposta comentada

Use `para n de 0 até antes de 3 faça`, `mostre n` e `fim`. A saída é 0, 1 e 2 em linhas separadas; o limite superior não é visitado.

Resposta do capítulo 13 no Apêndice B.

14. Coleções, índices e fatias

Objetivo de leitura

Escolher uma coleção e acessar itens, chaves e fatias.

Antes de começar

Conheça variáveis e repetições, dos capítulos 9 e 13.

Lista: sequência mutável

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina notas como [7, 8, 9]
coloque 10 em notas
mostre quantidade de notas
mostre primeiro item de notas
mostre notas[1:3]
```

Saída esperada

```
4
7
[8, 9]
```

Uma lista preserva a ordem e pode crescer. `coloque` acrescenta um elemento; `estenda` incorpora os elementos de outra sequência. Os índices começam em zero. Na fatia `[1:3]`, o início é incluído e o fim é excluído; a lista original não é alterada por essa consulta.

Dicionário: valores por chave

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina aluno como {"nome": "Ana", "nota": 8}
defina a chave "cidade" de aluno como "Timon"
mostre valor de "nome" em aluno
mostre aluno["cidade"]
```

Saída esperada

```
Ana
Timon
```

Uma chave identifica um valor. A expressão `aluno["nome"]` é equivalente à consulta natural mostrada. Consultar uma chave inexistente gera erro; confira a existência ou use uma operação apropriada ao seu caso. Remova com `remova a chave "nota" de aluno`.

Outras coleções

Tuplas, como `(10, 20)`, são sequências imutáveis. Uma tupla com um único item precisa da vírgula: `(10,)`. Conjuntos guardam elementos únicos, sem contrato de ordem para apresentação. `{}` representa dicionário vazio; use `um conjunto vazio` para um conjunto sem elementos.

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina numeros como [1, 2, 3, 4]
defina pares como [n vezes 2 para cada n em numeros se n % 2 for igual a 0]
mostre pares
defina unicos como {1, 1, 2}
mostre quantidade de unicos
```

Saída esperada

```
[4, 8]
2
```

A compreensão produz uma nova coleção a partir de outra. Primeiro percorre os valores, aplica o filtro e só então transforma cada valor aceito. No exemplo, apenas 2 e 4 passam pelo filtro e são dobrados.

Referências e mutabilidade

Atribuir uma lista a outro nome não cria necessariamente uma cópia independente. Os dois nomes podem apontar para o mesmo objeto. Quando a separação é importante, use uma cópia explícita, como uma fatia completa `valores[:]`, e considere que objetos aninhados continuam compartilhados numa cópia superficial.

Desempacotamento é permitido, como `defina x e y como (10, 20)`, respeitando a quantidade de valores. Evite misturar elementos de tipos incompatíveis em operações de ordenação ou aritmética.

O que você aprendeu

Listas usam índices a partir de zero; dicionários usam chaves. Uma atribuição pode compartilhar o objeto, por isso alterar e copiar são operações diferentes.

Prática

Diferencie alterar uma lista e produzir outra lista.

Conferir a resposta comentada

Uma alteração muda o objeto compartilhado por referências. Uma operação que produz nova lista preserva a original; confira o campo Atenção da ficha antes de assumir qualquer uma das duas situações.

Resposta do capítulo 14 no Apêndice B.

15. Funções e escopo

Objetivo de leitura

Declarar funções, passar argumentos e usar o retorno em outra expressão.

Antes de começar

Conheça variáveis, expressões e condições.

Uma função dá nome a uma operação reutilizável. Parâmetros são nomes que recebem argumentos a cada chamada. `retorne` encerra a função e devolve um valor; mostrar um resultado na tela não é a mesma coisa que retorná lo.

Uma função simples

Vamos receber um número e devolver seu dobro. O nome `numero` é um parâmetro; na chamada `dobro(21)`, o argumento é `21`.

Código Coral

```
crie a função dobro com numero
    retorne numero vezes 2
fim
mostre dobro(21)
```

Saída esperada

```
42
```

O cabeçalho declara a função e seu parâmetro. O corpo calcula o dobro; `retorne` entrega o resultado. Fora da função, `mostre` exibe esse resultado.

Dar uma forma natural à chamada

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
crie a função dobro com numero chamada como "dobre {numero}"
    retorne numero vezes 2
fim
mostre dobre 21
mostre (dobre 4) mais (dobre 5)
```

Saída esperada

```
42
18
```

`chamada como` declara uma forma natural. Os campos entre chaves precisam corresponder aos parâmetros. A leitura natural pode participar de outras expressões; os parênteses no segundo resultado deixam a composição evidente. Não existe interpretação livre por IA.

Parâmetros opcionais e nomeados

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
crie a função saudar com nome e prefixo opcional "Olá"
    retorne prefixo mais ", " mais nome
fim
mostre saudar("Ana")
mostre saudar(nome="Bia", prefixo="Oi")
crie a função contar com vários valores
    retorne quantidade de valores
fim
mostre contar(10, 20, 30)
```

Saída esperada

```
Olá, Ana
Oi, Bia
3
```

Um valor padrão permite omitir o argumento. Argumentos nomeados associam explicitamente o nome ao valor. `vários valores` aceita uma quantidade variável de argumentos posicionais. A forma `várias opcoes nomeadas` aceita argumentos nomeados variáveis.

Use `execute funcao(...)` quando deseja chamar uma função apenas por seu efeito. Use a chamada dentro de `defina` ou `mostre` quando deseja consumir o retorno. Uma expressão de chamada solta não é uma instrução universalmente aceita na superfície Coral.

Escopo e funções internas

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
crie a função criar_contador
    defina valor como 0
    crie a função proximo
        use valor do escopo externo
        adicione 1 a valor
        retorne valor
    fim
    retorne proximo
fim
defina contar como criar_contador()
mostre contar()
mostre contar()
```

Saída esperada

```
1
2
```

O estado `valor` pertence à função externa e continua acessível pela função retornada. `use valor do escopo externo` permite alterar essa ligação. Sem a declaração adequada, uma atribuição interna pode criar outra variável ou produzir um erro de escopo.

As formas declarativas de chamada natural são registradas durante a análise da fonte. Em projetos com módulos separados, importações seletivas também podem disponibilizar as formas naturais exportadas. Confira o capítulo 30 para aliases e ambiguidades; não presuma que toda forma interna do módulo seja exportada.

Funções como valores

Uma função pode ser atribuída a uma variável, recebida como argumento ou retornada. Uma função anônima pode ser escrita como `função anônima com x que retorna x vezes 2`. Isso é uma expressão; atribua o resultado a um nome antes de usar, quando precisar de uma operação identificável.

Decoradores são declarados com `decore com decorador` antes da função. Eles exigem um objeto chamável previamente definido e são um recurso avançado de interoperabilidade. Prefira funções comuns até existir uma razão concreta para modificar o comportamento de outras funções.

Não existe uma promessa de cobertura estática equivalente a linguagens com verificação obrigatória de tipos. Testes e validação de entradas continuam necessários.

Chamadas explícitas em várias linhas

Quando uma chamada tem muitos argumentos, coloque cada argumento numa linha. `chame somar com` abre o corpo porque `com` termina a linha sem argumentos; `fim` encerra essa chamada.

Código Coral

```
crie a função somar com primeiro, segundo
    retorne primeiro mais segundo
fim
defina total como chame somar com
    10
    chame somar com
        2
        3
    fim
fim
mostre total
```

Saída esperada

```
15
```

O primeiro `fim` encerra a chamada interna e o segundo encerra a externa. Os argumentos são avaliados uma vez, na ordem escrita. A versão compacta `chame somar com 10, 5` continua válida.

Cada entrada aceita uma expressão completa. Comentários e linhas vazias não contam como argumentos. Uma vírgula final é opcional. Argumentos nomeados e expansões

seguem as regras da chamada compacta. Para uma chamada sem argumentos, use a forma compacta sem `com`; abrir um bloco vazio com `com` é erro.

A chamada em bloco também pode criar objetos e chamar métodos. Ela pode aparecer em atribuições, retornos e argumentos de outra chamada. Se fizer parte de uma expressão maior, use os agrupamentos da linguagem para deixar a continuação explícita. O fechamento deve ocupar sua própria linha.

Use `chame` quando a chamada participa da organização natural do programa; `execute funcao(...)` continua disponível para executar a função por seu efeito. As duas formas invocam a operação; consumir ou ignorar o retorno depende do contexto. O suporte de chamadas usa seu próprio `fim`, enquanto a lista de parâmetros da seção seguinte usa `fim com`.

Parâmetros em linhas separadas

Na Coral 1.7.6, uma declaração que termina em `com` abre uma lista de parâmetros. Escreva um parâmetro por linha e feche a lista com `fim com`. O corpo começa depois desse fechamento e conserva seu próprio `fim`.

Código Coral

```
crie a função somar com
    primeiro do tipo inteiro
    segundo do tipo inteiro opcional 2
fim com retornando inteiro
    retorne primeiro mais segundo
fim
mostre somar(3)
```

Saída esperada

```
5
```

A forma compacta continua válida: `crie a função somar com primeiro do tipo inteiro, segundo do tipo inteiro opcional 2 retornando inteiro`. As duas formas têm os mesmos parâmetros, padrões e retorno. Um padrão é avaliado uma vez na criação da função, conforme o contrato existente; mover sua escrita para outra linha não muda esse momento.

Métodos, construtores e declarações assíncronas também aceitam a lista explícita. Os parâmetros implícitos do método continuam implícitos. Tipos estruturais e parâmetros variádicos conservam suas regras. Uma chamada multilinha usada como padrão fecha com seu próprio `fim`, antes do `fim com` da declaração. Uma função sem parâmetros omite `com`; uma lista vazia é diagnosticada.

Essa organização é opcional. O editor oferece a conversão de cabeçalhos elegíveis como uma ação explícita; a formatação normal não transforma toda assinatura compacta em um bloco.

O que você aprendeu

Uma função reúne uma operação reutilizável. `retorne` devolve um valor; `mostre` exibe texto. Parâmetros recebem os argumentos da chamada.

Prática

Crie uma função que receba um inteiro e devolva seu dobro.

Conferir a resposta comentada

Declare `crie a função dobro com numero do tipo inteiro retornando inteiro`, use `retorne numero vezes 2` e feche com `fim`. Uma chamada com 21 devolve 42.

Resposta do capítulo 15 no Apêndice B.

16. Classes e objetos

Objetivo de leitura

Criar objetos com estado próprio e reconhecer classe, atributo, método e herança.

Antes de começar

Conheça funções e escopo do capítulo 15.

Objetos combinam estado e operações. Uma classe define como criar e manipular esses objetos. Cada instância pode manter valores próprios.

Arquivo: `Exemplos/Citados/Fundamentos/03_objetos_e_erros.coral`.

Código Coral

```
crie a classe Contador
    ao criar uma Contador com inicial
        defina seu valor como inicial
    fim
    método somar com quantidade
        adicione quantidade a seu valor
    fim
    propriedade atual
        retorne seu valor
    fim
fim
crie contador como uma Contador com 10
execute contador.somar(5)
mostre contador.atual
```

Saída esperada

```
15
```

O construtor `ao criar` recebe o estado inicial. `seu valor` acessa o atributo da instância atual. O método altera esse atributo; a propriedade oferece leitura sem exigir uma chamada com parênteses.

Herdar comportamento

Arquivo: `Exemplos/Citados/Fundamentos/03_objetos_e_erros.coral`.

Código Coral

```
crie a classe Pessoa
    ao criar uma Pessoa com nome
        defina seu nome como nome
    fim
fim
crie a classe Aluno herda de Pessoa
    ao criar uma Aluno com nome e nota
        inicialize a classe pai com nome
        defina sua nota como nota
    fim
fim
crie aluno como uma Aluno com "Ana" e 9
mostre aluno.nome
mostre aluno.nota
```

Saída esperada

```
Ana
9
```

`Aluno` reutiliza a estrutura de `Pessoa`. O comando `inicialize a classe pai` chama a inicialização herdada, e o construtor acrescenta a nota. Herança é útil quando a relação de especialização é real; para apenas guardar ou usar outro objeto, composição costuma ser mais direta.

Referência de membros

Construção	Papel
`método nome com parametros`	Operação de instância; `self` é implícito
`método estático nome`	Operação sem instância implícita
`método de classe nome`	Operação que recebe a classe como `cls`
`propriedade nome`	Getter acessado como atributo
`ao definir a propriedade nome com valor`	Setter
`ao converter para texto`	Representação textual do objeto
`ao obter tamanho`	Protocolo de tamanho
`ao iterar` e `ao obter próximo`	Protocolo de iteração

Esses cabeçalhos são trechos de referência: devem aparecer dentro de uma classe, com corpo e `fim` próprios. As formas `seu`, `sua`, `seus` e `suas` referem se a atributos do objeto atual. O nome do atributo precisa coincidir exatamente com o que foi definido.

Para ir além

Herança múltipla e ordem dos métodos

Uma classe pode declarar mais de uma base em ordem explícita:

Código Coral

```
crie a classe Identificavel
    passe
fim
crie a classe Publicavel
    passe
fim
crie a classe Livro herda de Identificavel e Publicavel
    passe
fim
```

Este exemplo não imprime texto no terminal.

A Coral usa a ordem de resolução C3, a mesma regra aplicada pelo backend Python. Em uma hierarquia em diamante, a base comum aparece uma única vez na ordem de resolução.

Bases repetidas, ciclos, bases conhecidas que não são classes e hierarquias comprovadamente inconsistentes recebem diagnóstico antes da execução. Quando uma base é opaca para a análise local, a Coral evita inventar uma ordem estática.

Chamada cooperativa ao pai

Dentro de métodos de instância ou de classe, `chame o método pai` continua a cadeia cooperativa segundo a MRO.

Código Coral

```
crie a classe Raiz
    método nomes retornando lista de texto
        retorne ["Raiz"]
    fim
fim
crie a classe A herda de Raiz
    método nomes retornando lista de texto
        retorne ["A"] mais (chame o método pai nomes)
    fim
fim
```

Este exemplo não imprime texto no terminal.

Construtores podem usar `inicialize a classe pai`. A Coral não chama automaticamente todos os construtores das bases. A cooperação precisa estar explícita nas classes que participam da cadeia.

Exemplo: uma célula que guarda conteúdo

O objeto abaixo pode guardar grama ou um animal. Seus métodos fazem perguntas sobre esse conteúdo sem precisar comparar textos ou repetir regras de herança.

Código Coral

```
crie a classe Celula
    ao criar uma Celula com conteudo
        defina seu conteudo como conteudo
    fim
    método tem_grama
        retorne seu conteudo for do tipo Grama
    fim
    método tem_herbivoro
        retorne seu conteudo for do tipo Herbivoro
    fim
    método tem_carnivoro
        retorne seu conteudo for do tipo Carnivoro
    fim
fim
crie a classe Grama
fim
crie a classe Animal
fim
crie a classe Herbivoro herda de Animal
fim
crie a classe Carnivoro herda de Animal
fim
crie a classe Onivoro herda de Herbivoro e Carnivoro
fim

defina celula como chame Celula com
    Onivoro()
fim
mostre celula.tem_grama()
mostre celula.tem_herbivoro()
mostre celula.tem_carnivoro()
```

Saída esperada

```
False
True
True
```

As classes consultadas nos métodos podem ser declaradas depois da classe Celula; elas precisam estar disponíveis quando esses métodos forem executados. `Onivoro` pertence às duas classes ancestrais do exemplo. Para a aplicação organizada em módulos e testes, consulte `Exemplos/Citados/Analise_1_7_5/Ecossistema`.

Bases de herança em linhas separadas

Quando uma classe tem várias bases, `herda de` no fim da declaração abre a lista. `fim herança` fecha apenas o cabeçalho; o corpo da classe conserva seu próprio `fim`. A forma sem acento `fim heranca` também é aceita.

Código Coral

```
crie a classe Animal
fim
crie a classe Identificavel
fim
crie a classe Herbivoro herda de
    Animal
    Identificavel
fim herança
fim
defina animal como Herbivoro()
mostre animal for do tipo Animal
```

Saída esperada

```
True
```

A ordem das bases permanece a ordem escrita. O contrato de herança múltipla e a MRO C3 são os mesmos da declaração compacta. Repetir uma base, usar um valor que não representa uma classe ou criar uma hierarquia inconsistente continua produzindo diagnóstico. A lista não aceita expressões arbitrárias no lugar dos nomes das bases.

O que você aprendeu

A classe descreve os objetos; cada instância pode guardar seu próprio estado. Métodos operam sobre esse estado, e a herança reutiliza comportamentos.

Prática

Explique a diferença entre instância, classe e atributo.

Conferir a resposta comentada

A classe descreve a estrutura e as operações; a instância é um objeto criado a partir dela; um atributo guarda parte do estado desse objeto. Duas instâncias podem manter estados diferentes.

Resposta do capítulo 16 no Apêndice B.

17. Verificando tipos na prática

Objetivo de leitura

Verificar tipos nativos e pertencimento a classes do programa.

Antes de começar

Conheça tipos e classes, dos capítulos 10 e 16.

Às vezes um programa recebe dados cujo tipo precisa verificar. A verificação responde uma pergunta; a conversão tenta obter outro valor. Esses papéis são diferentes. Aqui verificamos; o Cap. 23 ensina conversões.

Código Coral

```
de coral.tipos importe e_tipo, nome_tipo

defina valor como 12
mostre nome_tipo(valor)
se valor for do tipo inteiro então
    mostre "Pode ser usado como contagem"
fim
mostre e_tipo(verdadeiro, "inteiro")
```

Saída esperada

```
inteiro
Pode ser usado como contagem
False
```

A saída é `inteiro`, `Pode ser usado como contagem` e `False`, em linhas separadas. Booleano não é inteiro implícito no teste oficial de Coral.

`tipo_de` devolve um descritor `TipoCoral`; `nome_tipo` devolve texto e `e_tipo` devolve um booleano. Nomes compostos das anotações do Cap. 10 pertencem à análise opcional; não confunda aquela análise estática com uma validação profunda feita por `e_tipo`. Para uma classe criada pelo programa, prefira a expressão natural com o nome da classe. A API explícita continua disponível e recebe a própria classe, não um texto com seu nome.

Consultar classes do programa

Uma consulta de tipo pergunta se o objeto pertence à classe indicada. Uma instância de uma subclasse também pertence às classes das quais ela herda. A consulta não compara apenas nomes e não converte o objeto.

Código Coral

```
de coral.tipos importe e_tipo
crie a classe Animal
fim
crie a classe Herbivoro herda de Animal
fim
defina animal como Herbivoro()
mostre animal for do tipo Herbivoro
mostre animal for do tipo Animal
mostre animal não for do tipo Animal
mostre e_tipo(animal, Animal)
```

Saída esperada

```
True
True
False
True
```

`animal for do tipo Animal` tem o mesmo resultado de `e_tipo(animal, Animal)`. A negação equivale a `não e_tipo(animal, Animal)`. A expressão pode aparecer em uma condição, atribuição, repetição ou retorno. As formas nativas, como `valor for do tipo inteiro`, continuam iguais. Em classes importadas, use o nome ou o alias disponível no escopo.

Depois de `do tipo`, forneça uma classe real. Se `Grama` guardar o número 10, o diagnóstico informa que Grama existe, mas não representa um tipo ou classe. Se o nome não existir, a análise informa o tipo inexistente e pode sugerir um nome próximo. Essas mensagens são advertências conservadoras de análise; não substituem a execução e os testes. Evite ocultar um nome de classe com outra variável.

Para herança múltipla, a consulta conserva o contrato de `e_tipo` e reconhece os ancestrais válidos da classe. A ordem de resolução de métodos continua sendo a MRO C3 apresentada no Cap. 16, seção Para ir além.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`tipo_de(...)` · **módulo** `coral.tipos`

O que faz

Obtém o descritor da categoria do valor.

Assinatura

Assinatura para consulta

```
tipo_de(valor: 'Any') -> 'TipoCoral'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

TipoCoral; o atributo nome contém o nome da categoria.

Exemplo

Código Coral

```
de coral.tipos importe tipo_de
mostre tipo_de(12).nome
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

`(o tipo de 12).nome` consulta o mesmo contrato.

Atenção

Booleano não é inteiro implícito. Não é um validador profundo de coleções parametrizadas.

`nome_tipo(...)` · módulo `coral.tipos`

O que faz

Obtém o nome Coral da categoria do valor.

Assinatura

Assinatura para consulta

```
nome_tipo(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor, classe Python ou TipoCoral	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.tipos importe nome_tipo
mostre nome_tipo(12)
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

`o nome do tipo de 12` consulta o mesmo contrato.

Atenção

Booleano não é inteiro implícito. Não é um validador profundo de coleções parametrizadas.

`e_tipo(...)` · módulo `coral.tipos`

O que faz

Testa se o valor pertence à categoria informada.

Assinatura

Assinatura para consulta

```
e_tipo(valor: 'Any', esperado: 'str | type | TipoCoral') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor sobre o qual a operação atua.
esperado	texto de tipo, classe Python ou TipoCoral	sim	sem padrão	Nome, tipo ou descritor aceito pelo teste de tipo.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.tipos importe e_tipo
mostre e_tipo(verdadeiro, "inteiro")
```

Resultado

Saída esperada

```
False
```

Forma natural

`valor for do tipo inteiro` e `animal for do tipo Animal` usam o mesmo contrato. A negação é `animal não for do tipo Animal`.

Erros

Um nome de classe ausente ou ocultado por outro valor pode receber diagnóstico de análise. A API não transforma um nome textual arbitrário numa classe do programa.

Atenção

Booleano não é inteiro implícito. Classes são passadas como objetos reais, não como texto. Subclasses pertencem aos ancestrais válidos, inclusive na herança múltipla. Não valida profundamente coleções parametrizadas.

Veja também

Cap. 17, seção Consultar classes do programa; Cap. 16, seção Exemplo: uma célula que guarda conteúdo.

O que você aprendeu

Uma consulta de tipo responde uma pergunta e não converte o valor. Uma instância também pertence aos ancestrais válidos de sua classe.

Prática

Verifique se verdadeiro é do tipo inteiro.

Conferir a resposta comentada

O teste oficial `e_tipo(verdadeiro, "inteiro")` devolve falso. A superfície Coral não trata booleano como inteiro implícito.

Resposta do capítulo 17 no Apêndice B.

18. Erros, padrões e contextos

Objetivo de leitura

Tratar falhas previstas, selecionar padrões e encerrar recursos.

Antes de começar

Conheça condições e funções. Os exemplos de arquivo serão retomados no capítulo 26.

Tratar falhas esperadas

Arquivo: `Exemplos/Citados/Fundamentos/03_objetos_e_erros.coral`.

Código Coral

```
tente
    defina resultado como 10 dividido por 0
se der erro do tipo ZeroDivisionError
    mostre "Não é possível dividir por zero"
finalmente
    mostre "Operação encerrada"
fim
```

Saída esperada

```
Não é possível dividir por zero
Operação encerrada
```

`se der erro do tipo` seleciona a exceção a tratar. `finalmente` é executado ao sair do bloco, inclusive quando houve erro. Uma captura ampla pode esconder falhas que você deveria corrigir; prefira capturar o tipo que seu programa realmente espera.

Uma exceção própria pode ser declarada com `crie a exceção Nome`, corpo e `fim`. Use `lance um erro do tipo Nome com "mensagem"` para lançá la. `garanta que condicao com mensagem "..."` expressa uma verificação, frequentemente usada em testes.

Selecionar por padrão

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina comando como "iniciar"
combinar comando
    caso "iniciar"
        mostre "Começando"
    caso "sair" ou "parar"
        mostre "Encerrando"
    caso padrão
        mostre "Comando desconhecido"
fim
```

Saída esperada

```
Começando
```

`combinar` avalia casos em ordem. Há suporte a literais, alternativas, capturas, sequências, mapeamentos e outros padrões estruturais. O caso padrão cobre o que não foi reconhecido. Não confunda uma captura de nome com uma comparação contra uma variável: padrões possuem regras próprias.

Liberar recursos

Arquivo: `Exemplos/Citados/Dados/02_arquivos_json_e_caminhos.coral`.

Código Coral

```
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
com arquivo "dados.txt" aberto para leitura como arquivo faça
    defina conteudo como arquivo.read()
fim
mostre conteudo
```

Saída esperada

```
Coral
```

O gerenciador de contexto fecha o arquivo ao sair do bloco. A leitura por `arquivo.read()` é uma chamada avançada do objeto de arquivo. Para operações simples, a biblioteca padrão de arquivos geralmente é suficiente.

Distingua erros de parsing, que impedem compreender o código, de erros de execução, como divisão por zero ou arquivo ausente. `coral verificar` não executa o programa e, por isso, não comprova que um caminho existe ou que uma operação é válida para todos os dados.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`escrever_texto(...)` **· módulo** `coral.arquivos`

O que faz

Escreve texto no arquivo escolhido.

Assinatura

Assinatura para consulta

```
escrever_texto(caminho, texto, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.
texto	qualquer valor convertido com str	sim	sem padrão	Texto a consultar ou transformar.
codificacao	texto de codificação	não	'utf-8'	Codificação de texto usada na leitura ou escrita.

Retorno

Nulo.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto
mostre escrever_texto("dados.txt", "Coral")
```

Resultado

Saída esperada

```
None
```

Forma natural

`escreva "Coral" em "dados.txt"` devolve nulo; o caminho é avaliado antes do texto.

Erros

PermissionError para acesso recusado; erros de caminho ou codificação quando a operação não é possível.

Atenção

Cria ou substitui o arquivo, com efeito no sistema de arquivos. Confira o caminho antes de escrever.

O que você aprendeu

Capture o erro que você espera. `combinar` seleciona padrões; um bloco de contexto cuida do encerramento do recurso ao sair.

Prática

Capture uma divisão por zero e imprima uma mensagem ao encerrar.

Conferir a resposta comentada

Use `tente`, a divisão, `se der erro do tipo ZeroDivisionError` com a mensagem, e `finalmente` para a mensagem final. O bloco final roda tanto no caso normal quanto na falha tratada.

Resposta do capítulo 18 no Apêndice B.

19. Geradores

Objetivo de leitura

Produzir uma sequência sob demanda com uma função geradora.

Antes de começar

Conheça funções e repetições, dos capítulos 13 e 15.

Produzir valores sob demanda

Arquivo: `Exemplos/Citados/Fundamentos/04_geradores_e_assincrono.coral`.

Código Coral

```
crie a função numeros com limite
    para n de 0 até antes de limite faça
        produza n
    fim
fim
para cada n em numeros(3) faça
    mostre n
fim
```

Saída esperada

```
0
1
2
```

`produza` suspende a execução de uma função geradora e entrega um valor por vez. Na próxima solicitação, a função continua de onde parou. Isso permite percorrer uma sequência sem construir todos os valores antecipadamente. `produza de outra_colecao` delega a produção a outro iterável.

O que você aprendeu

`produza` entrega um valor e suspende a função. Na próxima solicitação, a execução continua de onde parou.

Prática

Explique por que produzir números pode consumir menos memória que montar uma lista inteira.

Conferir a resposta comentada

Uma função geradora entrega um valor a cada solicitação e mantém apenas seu estado de execução. A economia existe quando o consumidor também processa os valores progressivamente.

Resposta do capítulo 19 no Apêndice B.

20. Assíncrono

Objetivo de leitura

Coordenar operações assíncronas e usar `aguarde` no contexto adequado.

Antes de começar

Conheça funções; complete primeiro os exemplos síncronos.

Programação assíncrona permite suspender uma operação enquanto outra pode avançar no mesmo ciclo de eventos. Ela não torna automaticamente todo cálculo paralelo nem elimina o custo de operações bloqueantes.

Arquivo: `Exemplos/Citados/Fundamentos/04_geradores_e_assincrono.coral`.

Código Coral

```
crie a função assíncrona dobro com numero
    espere 0 segundos
    retorne numero vezes 2
fim
crie a função assíncrona principal
    defina resultado como aguarde dobro(21)
    mostre resultado
fim
execute assincronamente principal()
```

Saída esperada

```
42
```

A função `dobro` é assíncrona. `aguarde` obtém seu resultado dentro de outra função assíncrona. `execute assincronamente` inicia a execução no nível principal. `espere 0 segundos` cede ao ciclo de eventos sem introduzir uma demora perceptível no exemplo.

Regras de contexto

`aguarde`, laços assíncronos e contextos assíncronos exigem função ou método assíncrono. Não copie apenas uma linha `aguarde` para o topo de um arquivo e espere que funcione. O exemplo inclui a estrutura necessária.

`crie tarefa como uma tarefa para operacao()` registra uma tarefa dentro de um contexto com ciclo de eventos ativo. Depois, `aguarde tarefa` obtém o resultado. Laços podem usar `para cada item assincronamente em fonte faça`; a fonte precisa implementar um iterador assíncrono.

Classes avançadas podem implementar `ao iterar assincronamente`, `ao obter próximo assincronamente`, `ao entrar assincronamente` e `ao sair assincronamente`. Esses nomes não transformam um objeto comum em assíncrono sem um comportamento correspondente.

Para programas iniciais de terminal, prefira fluxo síncrono. Use o modelo assíncrono quando houver uma necessidade concreta de coordenar espera e atividades, e teste tratamento de erro e encerramento das tarefas.

O que você aprendeu

Uma função assíncrona pode ceder a execução enquanto espera. Isso não transforma automaticamente um cálculo em trabalho paralelo.

Prática

Explique o papel de aguarde dentro de uma função assíncrona.

Conferir a resposta comentada

`aguarde` suspende a função até que a operação entregue um resultado, permitindo ao ciclo de eventos avançar outras tarefas. Não transforma um cálculo bloqueante em cálculo paralelo.

Resposta do capítulo 20 no Apêndice B.

21. Programa principal

Objetivo de leitura

Organizar a entrada da aplicação e distinguir execução direta de importação.

Antes de começar

Conheça funções e módulos básicos.

Definir a entrada da aplicação

A Coral aceita uma entrada principal opcional para organizar aplicações sem obrigar todo arquivo a usar esse modelo.

Código Coral

```
programa principal
    mostre "Olá"
fim
```

Saída esperada

```
Olá
```

Quando o arquivo é executado diretamente, a Coral inicializa primeiro o módulo inteiro e depois inicia o bloco `programa principal` uma única vez. Isso permite declarar funções e classes abaixo do bloco e utilizá las dentro dele. Quando o mesmo arquivo é importado como módulo, a entrada não é iniciada.

O bloco não recebe parâmetros, não possui anotação de retorno e não cria uma função pública chamada `principal`. Uma função comum com esse nome continua válida e precisa ser chamada normalmente. Também não é permitido devolver um valor da entrada. Um `retorne` vazio pode encerrar o bloco antecipadamente.

Organização sugerida

Código Coral

```
programa principal
    chame apresentar com "Ana"
fim

crie a função apresentar com nome
    mostre "Olá, " mais nome
fim
```

Saída esperada

```
Olá, Ana
```

A entrada é opcional. Scripts lineares continuam aceitos exatamente como antes:

Código Coral

```
mostre "Olá"
```

Saída esperada

```
Olá
```

`chame` e `execute`

`chame nome com argumentos` é a forma corrente da Coral para invocar uma função pelo nome e por seus argumentos, especialmente quando a intenção é realizar uma ação e ignorar o valor devolvido.

`execute EXPRESSAO` executa uma expressão de chamada por efeito. É útil para chamadas explícitas como `execute objeto.metodo(1)` ou quando a forma de expressão é mais adequada. Para consumir um retorno, use a chamada dentro de uma expressão, por exemplo em `defina` ou `mostre`.

Nenhuma dessas formas é usada para iniciar `programa principal`. A própria execução direta do arquivo inicia o bloco automaticamente.

Importação

Um arquivo que contém `programa principal` ainda pode ser usado como módulo. A importação disponibiliza seus símbolos públicos, mas não inicia a aplicação. Isso evita efeitos inesperados em testes, bibliotecas e projetos com vários módulos.

Editor

A extensão Coral reconhece a entrada no outline, hover, dobramento, indentação e realce. Ao pressionar Enter no cabeçalho, o fechamento com `fim` segue a configuração de fechamento automático de blocos. O snippet de entrada principal permanece uma escolha contextual e não substitui automaticamente identificadores digitados pelo usuário.

Onde a entrada pode aparecer

A entrada não pode ficar dentro de condição, repetição, função, classe, `tente` ou `combinar`. Só pode existir uma entrada principal válida por módulo. REPL e execução interativa de trechos rejeitam esse bloco porque não representam a execução completa do módulo.

O que você aprendeu

O bloco `programa principal` é opcional. A execução direta o inicia após inicializar o módulo; uma importação não inicia esse bloco.

Prática

Coloque a função chamada abaixo do bloco programa principal. O que acontece ao importar o arquivo?

Conferir a resposta comentada

A execução direta inicializa o módulo inteiro e só depois inicia a entrada, portanto a função já está disponível. A importação disponibiliza os símbolos, mas não inicia o bloco principal.

Resposta do capítulo 21 no Apêndice B.

PARTE III: BIBLIOTECA FUNDAMENTAL COM ESPECIFICAÇÕES

22. Entrada e saída

Objetivo de leitura

Ler uma resposta no teclado e mostrar uma mensagem com o texto recebido.

Antes de começar

Conheça variáveis e concatenação, dos capítulos 9 e 11.

Pedir uma resposta

Um programa de terminal pode mostrar uma pergunta e esperar uma resposta. A função `ler_linha`, do módulo `coral.entrada`, devolve o que foi digitado como texto.

Salve este programa em `saudacao.coral` e execute com `coral saudacao.coral`:

Código Coral

```
# Exemplo interativo: forneça Ana quando o programa pedir um nome.
de coral.entrada importe ler_linha

defina nome como ler_linha("Nome: ")
mostre "Olá, " mais nome
```

Saída esperada

```
Nome: Olá, Ana
```

O resultado acima registra o texto escrito pelo programa quando a entrada fornecida é `Ana`. Em um terminal comum, a digitação também aparece: você verá `Nome: Ana`, pressionará Enter e depois verá `Olá, Ana`. O eco da digitação pertence ao terminal, não à saída produzida pelo programa.

Leia o exemplo em três passos:

1. A importação disponibiliza `ler_linha`.
2. A função mostra `Nome: `, espera a resposta e devolve texto. A atribuição guarda esse texto em `nome`.
3. `mais` une a saudação e o nome; `mostre` exibe a mensagem.

Erro comum Uma entrada com aparência numérica ainda é texto. Para somar uma idade digitada, converta o valor antes do cálculo, como ensina o capítulo 23.

Repetir a entrada em um teste

Ao testar automaticamente, podemos fornecer uma resposta conhecida sem digitar. `FonteEntradaSequencial` guarda as respostas e `usar_fonte_entrada` seleciona essa fonte durante o bloco.

Arquivo completo de apoio: `Exemplos/Citados/Dados/01_entrada_conversoes_e_texto.coral`.

Código Coral

```
de coral.entrada importe ler_linha, FonteEntradaSequencial, usar_fonte_entrada

defina entrada como FonteEntradaSequencial(["Ana"])
com usar_fonte_entrada(entrada) faça
    defina nome como ler_linha("Nome: ")
fim
mostre "Olá, " mais nome
```

Saída esperada

```
Olá, Ana
```

A fonte sequencial não reproduz o eco de um terminal. Ela permite verificar a mesma lógica com dados previsíveis. `ler_linha` traduz o fim da fonte para o erro Coral `FimDeEntrada`; um teste precisa fornecer respostas suficientes.

A superfície de entrada também pode ser usada por REPL, depurador e outras ferramentas. Para começar um programa comum, a leitura manual do primeiro exemplo é suficiente.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`ler_linha(...)` · **módulo** `coral.entrada`

O que faz

Lê uma linha da fonte explícita, do contexto de entrada ou do terminal.

Assinatura

Assinatura para consulta

```
ler_linha(mensagem: 'str | None' = None, *, fonte: 'FonteEntrada | FonteEntradaFuncao |
  None' = None) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	texto ou nulo	não	None	Texto mostrado antes de ler a entrada; nulo equivale a texto vazio.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	FonteEntrada ou FonteEntradaFuncao ou nulo	não	None	Fonte explícita de entrada; omitida, usa o contexto ou o terminal.

Retorno

Texto sem a quebra de linha final.

Exemplo

Código Coral

```
de coral.entrada importe ler_linha
mostre ler_linha("Nome: ")
```

Resultado

Saída esperada

```
Nome: Ana
```

Forma natural

`leia uma linha com "Nome: "` tem o mesmo contrato. Veja o Cap. 22.

Erros

FimDeEntrada quando a fonte termina; não é um texto vazio.

`formatar_valor(...)` · módulo `coral.formatacao`

O que faz

Formata um valor básico usando sua grafia Coral.

Assinatura

Assinatura para consulta

```
formatar_valor(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor representável como texto Coral	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.formatacao importe formatar_valor
mostre formatar_valor(verdadeiro)
```

Resultado

Saída esperada

```
verdadeiro
```

`montar_texto(...)` · módulo `coral.formatacao`

O que faz

Monta texto a partir de vários valores, separador e final.

Assinatura

Assinatura para consulta

```
montar_texto(*partes: 'Any', separador: 'str' = '', final: 'str' = '') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
partes	argumentos variáveis, representáveis como texto Coral	não	sem padrão	Valores convertidos e reunidos no texto final.
separador	valor representável como texto Coral	não	''	Texto que separa as partes.
final	valor representável como texto Coral	não	''	Texto acrescentado após a última parte.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.formatacao importe montar_texto
mostre montar_texto("Coral", 3, separador=" ")
```

Resultado

Saída esperada

```
Coral 3
```

`FonteEntradaSequencial(...)` · módulo `coral.entrada`

O que faz

Fornece linhas pré definidas para repetir uma leitura de entrada em testes.

Construtor

Assinatura para consulta

```
FonteEntradaSequencial(linhas: 'Iterable[str]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
linhas	iterável de textos	sim	sem padrão	Linhas fornecidas à fonte em ordem de leitura.

Atributos importantes

Atributo	Significado
`linhas`	Coleção de textos fornecida ao criar a fonte.

Métodos principais

Assinatura para consulta

```
ler_linha(self, mensagem: 'str' = '') -> 'str'
```

Retorno

Instância de FonteEntradaSequencial.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntradaSequencial
defina fonte como FonteEntradaSequencial(["Ana"])
mostre fonte.ler_linha()
```

Resultado

Saída esperada

```
Ana
```

Erros

FimDeEntrada quando não há mais linhas.

`usar_fonte_entrada(...)` · módulo `coral.entrada`

O que faz

Escolhe uma fonte de entrada apenas durante um bloco de contexto.

Assinatura

Assinatura para consulta

```
usar_fonte_entrada(fonte: 'FonteEntrada | FonteEntradaFuncao')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	FonteEntrada	sim	sem padrão	Fonte usada temporariamente no contexto de entrada.

Retorno

Gerenciador de contexto; entrar nele devolve a fonte.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntradaSequencial, usar_fonte_entrada, ler_linha
defina fonte como FonteEntradaSequencial(["Ana"])
com usar_fonte_entrada(fonte) faça
    mostre ler_linha()
fim
```

Resultado

Saída esperada

```
Ana
```

Atenção

Restaura a fonte anterior ao sair do bloco, inclusive em caso de erro.

O que você aprendeu

`ler_linha` devolve texto. Para calcular com uma entrada, converta o valor explicitamente; uma saudação pode usar o texto como foi lido.

Prática

Leia um nome e mostre uma saudação.

Conferir a resposta comentada

Leia uma linha com `ler_linha("Nome: ")` ou a forma natural equivalente. Guarde o texto e use `mostre "Olá, " mais nome`. A conversão é desnecessária para um nome.

Resposta do capítulo 22 no Apêndice B.

23. Conversões

Objetivo de leitura

Converter valores e escolher como tratar uma entrada inválida.

Antes de começar

Conheça tipos e entrada de teclado, dos capítulos 10 e 22.

Uma entrada de teclado é texto. Converter significa produzir um valor com outro tipo antes de usá lo no cálculo. Coral oferece `inteiro`, `decimal`, `texto` e `booleano`.

Código Coral

```
de coral.conversoes importe inteiro, decimal
mostre inteiro("12")
mostre decimal("2,5")
mostre tente converter "inválido" para inteiro, senão nulo
```

Saída esperada

```
12
2.5
None
```

A saída é 12, 2.5 e None, em linhas separadas. A conversão estrita não transforma um número fracionário em inteiro por truncamento e não aceita booleano como inteiro implícito. Em texto decimal, ponto ou vírgula são aceitos, mas sua mistura é ambígua.

Um valor padrão é devolvido quando a conversão falha. Sem padrão, o erro público é `ErroConversao`. A forma natural e a API explícita têm o mesmo contrato. Argumentos são avaliados uma vez; o padrão também é avaliado numa conversão bem sucedida. Consulte o Cap. 18 para tratamento de erros.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`inteiro(...)` · **módulo** `coral.conversoes`

O que faz

Converte um inteiro, um decimal exato ou uma representação textual inteira.

Assinatura

Assinatura para consulta

```
inteiro(valor: 'Any', padrao: 'Any' = <ausente>) -> 'int | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro, decimal finito sem fração ou texto de inteiro, exceto booleano	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Inteiro, ou o padrão explicitamente fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe inteiro
mostre inteiro("12")
```

Resultado

Saída esperada

```
12
```

Forma natural

`converta "12" para inteiro` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Não trunca decimais fracionários e rejeita booleanos. Textos admitem sinal e espaços nas extremidades, sem separadores de milhar. O padrão é retornado como foi fornecido, inclusive nulo.

`decimal(...)` · módulo `coral.conversoes`

O que faz

Converte um número ou texto para decimal finito, aceitando ponto ou vírgula decimal.

Assinatura

Assinatura para consulta

```
decimal(valor: 'Any', padrao: 'Any' = <ausente>) -> 'float | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro, decimal finito ou texto numérico, exceto booleano	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Decimal finito, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe decimal
mostre decimal("2,5")
```

Resultado

Saída esperada

```
2.5
```

Forma natural

`converta "2,5" para decimal` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Aceita ponto ou vírgula decimal, mas rejeita a mistura dos dois no mesmo texto. Rejeita NaN, infinitos e booleanos. O padrão é retornado como foi fornecido, inclusive nulo.

`texto(...)` · módulo `coral.conversoes`

O que faz

Produz texto com a grafia Coral para os valores básicos.

Assinatura

Assinatura para consulta

```
texto(valor: 'Any', padrao: 'Any' = <ausente>) -> 'str | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor representável por str, incluindo nulo e booleanos	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Texto, ou o padrão se sua produção falhar.

Exemplo

Código Coral

```
de coral.conversoes importe texto
mostre texto(nulo)
```

Resultado

Saída esperada

```
nulo
```

Forma natural

`converta nulo para texto` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

nulo, verdadeiro e falso usam grafia Coral; os demais valores usam str. Se a representação falha, devolve o padrão fornecido, inclusive nulo, ou levanta ErroConversao quando o padrão foi omitido.

`booleano(...)` · módulo `coral.conversoes`

O que faz

Converte representações explícitas de verdadeiro e falso.

Assinatura

Assinatura para consulta

```
booleano(valor: 'Any', padrao: 'Any' = <ausente>) -> 'bool | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	booleano, número 0/1 ou texto booleano explícito	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Booleano, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe booleano
mostre booleano("sim")
```

Resultado

Saída esperada

```
True
```

Forma natural

`converta "sim" para booleano` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Aceita verdadeiro/falso, números exatamente 0/1 e textos normalizados: verdadeiro,true,sim,1; falso,false,nao,não,0. Não usa a regra Python de qualquer objeto não vazio ser verdadeiro. O padrão é retornado como foi fornecido, inclusive nulo.

O que você aprendeu

Conversão produz outro valor. Sem padrão, uma falha produz `ErroConversao`; com padrão, a operação devolve o valor alternativo indicado.

Prática

Converta 12 e depois uma entrada inválida com padrão nulo.

Conferir a resposta comentada

`inteiro("12")` devolve 12. `inteiro("inválido", nulo)` devolve nulo. Sem o padrão, a entrada inválida produz `ErroConversao`.

Resposta do capítulo 23 no Apêndice B.

24. Texto

Objetivo de leitura

Limpar espaços e transformar textos sem alterar o valor original.

Antes de começar

Conheça textos e chamadas de função.

Texto guarda caracteres e preserva espaços dentro de aspas. Para limpar uma entrada, use `aparar`; para comparações com uma regra de caixa explícita, use `maiusculas` ou `minusculas`.

Código Coral

```
de coral.texto importe aparar, maiusculas
mostre maiusculas(aparar("  Coral  "))
mostre "coral" começa com "co"
```

Saída esperada

```
CORAL
True
```

A saída é CORAL e True, em linhas separadas. Essas operações produzem texto ou respostas novas; não alteram a string original. Prefixos e sufixos são comparados exatamente como fornecidos. Normalização Unicode, caixa e equivalência cultural são decisões distintas; não remova acentos ou normalize nomes automaticamente sem um motivo.

Índices e fatias são ensinados no Cap. 14. Para a família completa, consulte o Volume II, Parte II, seção Biblioteca `coral.texto`.

Textos com várias linhas

Na Coral 1.7.9, três aspas duplas ou três aspas simples delimitam um texto multilinha. O recurso é geral: serve para SQL, HTML, Markdown, mensagens extensas e outros conteúdos textuais. Ele não executa nem interpreta o conteúdo.

Quando a abertura termina a linha e o conteúdo começa na linha seguinte, essa primeira quebra é estrutural. Se o fechamento também fica sozinho em sua linha, a última quebra estrutural é removida. O recuo comum é retirado e o recuo relativo interno é preservado.

Código Coral

```
defina mensagem como """
    primeira linha
      segunda linha
    terceira linha
"""
mostre mensagem
```

Saída esperada

```
primeira linha
  segunda linha
terceira linha
```

O corpo é opaco para a estrutura do programa. Palavras como `fim`, parênteses, comentários aparentes e nomes que parecem imports continuam sendo caracteres do texto. Strings de uma linha continuam com o mesmo comportamento das versões anteriores. Se o fechamento faltar, o editor aponta o início do literal com o diagnóstico P180.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`maiusculas(...)` · **módulo** `coral.texto`

O que faz

Produz a versão em maiúsculas do texto.

Assinatura

Assinatura para consulta

```
maiusculas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe maiusculas
mostre maiusculas("Coral")
```

Resultado

Saída esperada

```
CORAL
```

Forma natural

`maiúsculas de "Coral"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`minusculas(...)` · módulo `coral.texto`

O que faz

Produz a versão em minúsculas do texto.

Assinatura

Assinatura para consulta

```
minusculas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe minusculas
mostre minusculas("Coral")
```

Resultado

Saída esperada

```
coral
```

Forma natural

`minúsculas de "Coral"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`aparar(...)` · módulo `coral.texto`

O que faz

Retira espaços em branco nas extremidades do texto.

Assinatura

Assinatura para consulta

```
aparar(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe aparar
mostre aparar("  Coral  ")
```

Resultado

Saída esperada

```
Coral
```

Forma natural

`apare o texto de " Coral "` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`comeca_com(...)` · módulo `coral.texto`

O que faz

Verifica se o texto começa com o prefixo informado.

Assinatura

Assinatura para consulta

```
comeca_com(valor, prefixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
prefixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no início.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.texto importe comeca_com
mostre comeca_com("coral", "co")
```

Resultado

Saída esperada

```
True
```

Forma natural

`"coral" começa com "co"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`termina_com(...)` · módulo `coral.texto`

O que faz

Verifica se o texto termina com o sufixo informado.

Assinatura

Assinatura para consulta

```
termina_com(valor, sufixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
sufixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no final.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.texto importe termina_com
mostre termina_com("coral", "al")
```

Resultado

Saída esperada

```
True
```

Forma natural

`"coral" termina com "al"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`substituir(...)` · módulo `coral.texto`

O que faz

Produz um texto substituindo ocorrências do trecho informado.

Assinatura

Assinatura para consulta

```
substituir(valor, antigo, novo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
antigo	qualquer valor convertido com str	sim	sem padrão	Trecho a substituir.
novo	qualquer valor convertido com str	sim	sem padrão	Texto que ocupa o lugar do trecho encontrado.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe substituir
mostre substituir("casa", "a", "o")
```

Resultado

Saída esperada

```
coso
```

Forma natural

`substitua "a" por "o" em "casa"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`dividir(...)` · módulo `coral.texto`

O que faz

Divide o texto em partes.

Assinatura

Assinatura para consulta

```
dividir(valor, separador=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
separador	texto não vazio ou nulo	não	None	Texto que separa as partes.

Retorno

Lista de textos.

Exemplo

Código Coral

```
de coral.texto importe dividir
mostre dividir("a,b", ",")
```

Resultado

Saída esperada

```
['a', 'b']
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`juntar(...)` · módulo `coral.texto`

O que faz

Reúne textos com um separador.

Assinatura

Assinatura para consulta

```
juntar(separador, valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
separador	qualquer valor convertido com str	sim	sem padrão	Texto que separa as partes.
valores	iterável de valores convertidos com str	sim	sem padrão	Coleção de entrada.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.texto importe juntar
mostre juntar(",", ["a", "b"])
```

Resultado

Saída esperada

```
a,b
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`linhas(...)` · módulo `coral.texto`

O que faz

Obtém as linhas de um texto.

Assinatura

Assinatura para consulta

```
linhas(valor, *, manter_quebras=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
manter_quebras	booleano	não	False	Se verdadeiro, mantém os caracteres de quebra de linha nas partes devolvidas.

Retorno

Lista de textos.

Exemplo

Código Coral

```
de coral.texto importe linhas
mostre linhas("a\nb")
```

Resultado

Saída esperada

```
['a', 'b']
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`localizar(...)` · módulo `coral.texto`

O que faz

Procura um trecho no texto.

Assinatura

Assinatura para consulta

```
localizar(valor, trecho, inicio=0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
trecho	qualquer valor convertido com str	sim	sem padrão	Texto procurado a partir do índice inicial.
inicio	valor convertível com int	não	0	Índice inicial da busca.

Retorno

Índice inteiro; menos um quando não encontrado.

Exemplo

Código Coral

```
de coral.texto importe localizar
mostre localizar("coral", "ra")
```

Resultado

Saída esperada

```
2
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`contar(...)` · módulo `coral.texto`

O que faz

Conta ocorrências de um trecho.

Assinatura

Assinatura para consulta

```
contar(valor, trecho, inicio=0, fim=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
trecho	qualquer valor convertido com str	sim	sem padrão	Texto cujas ocorrências serão contadas.
inicio	valor convertível com int	não	0	Índice inicial da busca.
fim	valor convertível com int ou nulo	não	None	Limite final da busca.

Retorno

Quantidade inteira.

Exemplo

Código Coral

```
de coral.texto importe contar
mostre contar("banana", "a")
```

Resultado

Saída esperada

```
3
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`remover_prefixo(...)` · módulo `coral.texto`

O que faz

Retira o prefixo se estiver presente.

Assinatura

Assinatura para consulta

```
remover_prefixo(valor, prefixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
prefixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no início.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe remover_prefixo
mostre remover_prefixo("coral", "co")
```

Resultado

Saída esperada

```
ral
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`remover_sufixo(...)` · módulo `coral.texto`

O que faz

Retira o sufixo se estiver presente.

Assinatura

Assinatura para consulta

```
remover_sufixo(valor, sufixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
sufixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no final.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe remover_sufixo
mostre remover_sufixo("coral", "al")
```

Resultado

Saída esperada

```
cor
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`normalizar_unicode(...)` · módulo `coral.texto`

O que faz

Normaliza a representação Unicode com a forma escolhida.

Assinatura

Assinatura para consulta

```
normalizar_unicode(valor, forma='NFC')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
forma	texto NFC, NFD, NFKC ou NFKD	não	'NFC'	Forma de normalização Unicode.

Retorno

Texto normalizado.

Exemplo

Código Coral

```
de coral.texto importe normalizar_unicode
mostre normalizar_unicode("Coral")
```

Resultado

Saída esperada

```
Coral
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

O que você aprendeu

As operações retornam novos valores. Retirar espaços, mudar caixa e normalizar caracteres são decisões diferentes.

Prática

Remova espaços externos de um texto e converta para maiúsculas.

Conferir a resposta comentada

`aparar(" Coral ")` devolve `"Coral"`; passar esse resultado para `maiusculas` devolve `"CORAL"`. O texto original não é modificado.

Resposta do capítulo 24 no Apêndice B.

25. Coleções

Objetivo de leitura

Ordenar, contar e remover repetições de coleções.

Antes de começar

Conheça as estruturas do capítulo 14.

Uma coleção agrupa valores. Além de acessar seus itens, podemos ordenar, remover repetições, contar ocorrências e consultar o primeiro elemento.

Código Coral

```
de coral.colecoes importe unicos, ordenar, contar

defina valores como [3, 1, 3, 2]
mostre unicos(valores)
mostre ordenar(valores)
mostre contar(valores)
mostre valores
```

Saída esperada

```
[3, 1, 2]
[1, 2, 3, 3]
{3: 2, 1: 1, 2: 1}
[3, 1, 3, 2]
```

A saída é [3, 1, 2], [1, 2, 3, 3], {3: 2, 1: 1, 2: 1} e [3, 1, 3, 2], em linhas separadas. A origem permanece igual. `unicos` mantém a primeira ocorrência e precisa de itens utilizáveis como chaves. `contar` produz um dicionário de frequências; `primeiro` devolve nulo quando a coleção é vazia.

No Cap. 14 você encontra a estrutura das coleções; aqui encontra as operações. Tipos parametrizados ficam no Cap. 10.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`ordenar(...)` · **módulo** `coral.colecoes`

O que faz

Ordena os valores e produz uma nova lista.

Assinatura

Assinatura para consulta

```
ordenar(valores, reverso=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens comparáveis	sim	sem padrão	Coleção de entrada.
reverso	booleano	não	False	Se verdadeiro, inverte a ordem da classificação.

Retorno

Lista ordenada.

Exemplo

Código Coral

```
de coral.colecoes importe ordenar
mostre ordenar([3, 1, 2])
```

Resultado

Saída esperada

```
[1, 2, 3]
```

Forma natural

`ordene os valores de [3, 1, 2]` é equivalente para estes argumentos.

Atenção

Retorna uma lista nova; não modifica a coleção original. Requer que os itens possam ser comparados entre si.

`unicos(...)` · módulo `coral.colecoes`

O que faz

Conserva a primeira ocorrência de cada valor.

Assinatura

Assinatura para consulta

```
unicos(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens hashable	sim	sem padrão	Coleção de entrada.

Retorno

Lista sem duplicatas.

Exemplo

Código Coral

```
de coral.colecoes importe unicos
mostre unicos([3, 1, 3, 2])
```

Resultado

Saída esperada

```
[3, 1, 2]
```

Forma natural

`os valores únicos de [3, 1, 3, 2]` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`contar(...)` · módulo `coral.colecoes`

O que faz

Conta as ocorrências de cada valor.

Assinatura

Assinatura para consulta

```
contar(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens hashable	sim	sem padrão	Coleção de entrada.

Retorno

Dicionário de frequências.

Exemplo

Código Coral

```
de coral.colecoes importe contar
mostre contar([3, 1, 3])
```

Resultado

Saída esperada

```
{3: 2, 1: 1}
```

Forma natural

`conte as ocorrências de [3, 1, 3]` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`primeiro(...)` · módulo `coral.colecoes`

O que faz

Obtém o primeiro elemento ou o padrão se não houver elemento.

Assinatura

Assinatura para consulta

```
primeiro(valores, padrao=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável	sim	sem padrão	Coleção de entrada.
padrao	qualquer valor	não	None	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Elemento ou padrão; por padrão, nulo.

Exemplo

Código Coral

```
de coral.colecoes importe primeiro
mostre primeiro([])
```

Resultado

Saída esperada

```
None
```

Forma natural

`o primeiro de []` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`agrupar_pares(...)` · módulo `coral.colecoes`

O que faz

Combina chaves e valores por posição para construir um dicionário.

Assinatura

Assinatura para consulta

```
agrupar_pares(chaves, valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
chaves	iterável de chaves hashable	sim	sem padrão	Coleção de chaves.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores	sim	sem padrão	Coleção de valores, combinada por posição com as chaves.

Retorno

Dicionário com os pares; chaves repetidas mantêm o último valor.

Exemplo

Código Coral

```
de coral.colecoes importe agrupar_pares
mostre agrupar_pares(["a", "b"], [1, 2])
```

Resultado

Saída esperada

```
{'a': 1, 'b': 2}
```

Atenção

Consome chaves e valores até terminar o iterável mais curto. Chaves repetidas ficam com o último valor correspondente.

O que você aprendeu

`unicos` conserva a primeira ocorrência; `ordenar` organiza valores; `contar` produz frequências. Confira na ficha se a operação altera a origem.

Prática

Remova valores repetidos de [3, 1, 3, 2] e ordene o resultado.

Conferir a resposta comentada

`unicos` produz [3, 1, 2], conservando a primeira ocorrência. `ordenar` produz [1, 2, 3]. A lista original permanece igual.

Resposta do capítulo 25 no Apêndice B.

26. Arquivos e caminhos

Objetivo de leitura

Gravar e ler texto, reconhecendo a pasta usada por um caminho relativo.

Antes de começar

Conheça textos, funções e tratamento de erros.

Gravar e ler dados

Arquivo: `Exemplos/Citados/Dados/02_arquivos_json_e_caminhos.coral`.

Código Coral

```
de coral.arquivos importe escrever_texto e ler_texto
execute escrever_texto("mensagem.txt", "Olá, arquivo!")
defina conteudo como ler_texto("mensagem.txt")
mostre conteudo
```

Saída esperada

```
Olá, arquivo!
```

O caminho relativo é resolvido a partir da pasta de trabalho, não necessariamente da pasta em que o arquivo `.coral` está salvo. Se você executa de outra pasta, um caminho como `mensagem.txt` aponta a essa outra pasta. Para um projeto, comece pelo terminal na raiz e organize dados em diretórios conhecidos.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`ler_texto(...)` · **módulo** `coral.arquivos`

O que faz

Lê todo o texto de um arquivo.

Assinatura

Assinatura para consulta

```
ler_texto(caminho, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.
codificacao	texto de codificação	não	'utf-8'	Codificação de texto usada na leitura ou escrita.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.arquivos importe ler_texto
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
mostre ler_texto("dados.txt")
```

Resultado

Saída esperada

```
Coral
```

Forma natural

`leia o texto de "dados.txt"` é equivalente.

Erros

A leitura pode produzir FileNotFoundError ou PermissionError. Consulte o Cap. 18.

Atenção

O exemplo cria seu próprio arquivo na pasta de trabalho.

`existe(...)` · módulo `coral.arquivos`

O que faz

Verifica se o caminho existe, usando a normalização oficial.

Assinatura

Assinatura para consulta

```
existe(caminho)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.arquivos importe existe
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
mostre existe("dados.txt")
```

Resultado

Saída esperada

```
True
```

Forma natural

`existe o caminho "dados.txt"` é equivalente.

Erros

A leitura pode produzir FileNotFoundError ou PermissionError. Consulte o Cap. 18.

Atenção

O exemplo cria seu próprio arquivo na pasta de trabalho.

O que você aprendeu

Um caminho relativo parte da pasta de trabalho. Permissões, codificação e existência do arquivo fazem parte do uso real da operação.

Prática

Escreva Coral num arquivo e leia o conteúdo.

Conferir a resposta comentada

Use `escrever_texto("dados.txt", "Coral")` e `ler_texto("dados.txt")`. A escrita tem efeito no sistema de arquivos e a leitura devolve texto. Considere permissão, codificação e pasta de trabalho.

Resposta do capítulo 26 no Apêndice B.

27. JSON e serialização

Objetivo de leitura

Ler e escrever arquivos e validar registros externos com conversão explícita e diagnósticos por campo.

Antes de começar

Conheça dicionários, listas e textos.

Serializar estruturas

Arquivo: `Exemplos/Citados/Dados/02_arquivos_json_e_caminhos.coral`.

Código Coral

```
de coral.json importe para_json e de_json
defina original como {"nome": "Ana", "nota": 9}
defina documento como para_json(original)
defina recuperado como de_json(documento)
mostre recuperado["nome"]
mostre recuperado["nota"]
```

Saída esperada

```
Ana
9
```

JSON preserva estruturas básicas como objetos, listas, textos, números, booleanos e ausência. Ele não serializa automaticamente qualquer classe personalizada. `de_json` interpreta dados JSON; não executa código Coral.

Processar arquivos e CSV aos poucos

Quando os dados crescem, um iterador permite processar um registro de cada vez. Um lote reúne uma quantidade limitada de registros para uma etapa de trabalho. Isso reduz a necessidade de guardar todos os dados na memória, embora o tamanho de cada registro ainda importe.

Código Coral

```
de coral.arquivos importe escrever_texto e ler_lotes_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\nBia,20\nCaio,30\n")
com ler_lotes_csv("dados.csv", tamanho_lote=2) como lotes faça
    para cada lote em lotes faça
        mostre tamanho de lote
    fim
fim
```

Saída esperada

```
2
1
```

A abertura acontece no primeiro avanço, não na construção do leitor. O leitor é de passagem única e fecha no fim ou em erro. Para parar antes, use `com` ou `fechar()`; não dependa da coleta de lixo. Linhas de texto conservam suas quebras; registros CSV podem conter vírgulas e quebras dentro de aspas. Cabeçalhos devem ser únicos, não vazios, e as linhas precisam manter a largura.

Os escritores exigem textos, consomem gradualmente e substituem o destino. Não inserem quebras em `escrever_linhas` nem convertem valores implicitamente em `escrever_csv`; um erro pode deixar destino parcial. Um leitor usado como entrada de um escritor deve continuar sob o `com` externo do chamador. `coral.formatos.de_csv` continua materializando dados, conforme seu contrato anterior.

Veja o projeto executável `Exemplos/Projetos/Importador_Gradual`: lotes de dois registros, filtro `ativo` igual a `sim`, nomes aparados e saída `nome,valor`. As fichas deste capítulo detalham cada contrato. A referência completa está no Volume II e em `Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md`.

Para sequências de campos, nomes não são inferíveis: sem colunas explícitas, não é emitido cabeçalho mesmo com cabecalho=True. Objetos permitem inferir nomes na ordem do primeiro registro; colunas explícitas definem nomes e ordem.

Validar registros externos com esquemas

Na Coral 1.7.8, `coral.esquemas` descreve contratos de cadastro, pesquisa e inventário com quatro chamadas explícitas: `campo`, `esquema`, `validar_registro` e `validar_registros`. Nenhuma forma natural nova foi criada. A validação aceita ou relata dados; não converte textos, não preenche valores padrão e não altera a entrada.

Ausência significa que a chave não existe. Nulo significa que a chave existe com `nulo`. Um campo opcional pode estar ausente e ainda recusar nulo; um campo obrigatório e anulável exige a chave, mas aceita nulo. Inteiro rejeita booleanos; decimal aceita números finitos e rejeita booleanos. Limites são inclusivos.

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
de coral.conversoes importe inteiro

defina cadastro como esquema({"idade": campo("inteiro", minimo=0), "apelido":
  campo("texto", obrigatorio=falso)}, permitir_extras=falso)
defina bruto como {"idade": "18"}
mostre validar_registro(bruto, cadastro)["erros"][0]["regra"]
defina convertido como {"idade": inteiro(bruto["idade"])}
mostre validar_registro(convertido, cadastro)["valido"]
mostre validar_registro({"idade": 18, "apelido": nulo}, cadastro)["erros"][0]["regra"]
```

Saída esperada

```
tipo
True
nulo
```

A conversão `inteiro` é explícita e pode falhar antes da validação; preserve esse erro no relatório de importação. JSON também precisa validar dados após `de_json`. Não confunda texto CSV vazio, chave ausente e nulo JSON. O projeto `Exemplos/Projetos/Importador_Validado` combina conversão, validação e relatório JSON.

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registros

defina pesquisa como esquema({"medida": campo("decimal", minimo=0), "grupo":
  campo("texto", permitidos=["controle", "ensaio"])})
defina resultado como validar_registros([{"medida": -1, "grupo": "outro"}, {"medida": 2,
   "grupo": "ensaio"}], pesquisa, limite_erros=1, linha_inicial=2, origem="pesquisa.csv")
mostre resultado["erros"][0]["campo"]
mostre resultado["erros"][0]["regra"]
mostre resultado["erros"][0]["registro"]
mostre resultado["erros"][0]["linha"]
mostre resultado["erros"][0]["origem"]
mostre resultado["verificados"]
mostre resultado["interrompido"]
```

Saída esperada

```
medida
minimo
1
2
pesquisa.csv
1
True
```

Erros contêm `campo`, `regra`, `mensagem`, `registro`, `linha` e `origem`, sem copiar valores sensíveis. A ordem declarada dos campos determina a ordem do diagnóstico; extras recusados vêm depois, na ordem do registro. Tipo incompatível não avalia limites desse campo. `modo="primeiro"` para no primeiro erro; `modo="coletar"` para ao atingir `limite_erros`, inteiro positivo que não aceita booleano. `interrompido` sinaliza a política, sem alegar que o restante foi examinado. `verificados` conta somente registros consumidos.

`linha_inicial` informa a linha lógica do primeiro registro e os seguintes somam 1; o índice de registro começa em 1. CSV com campos multilinha não permite deduzir linha física pelo

índice. `origem` é fornecida pelo chamador. A validação consome uma passagem e guarda apenas erros, sem fechar iteradores alheios. Ao usar `ler_csv`, mantenha o leitor dentro de `com`, inclusive quando o limite interromper a validação. Esquema e opções inválidos falham antes do consumo; falhas de leitura/conversão propagam ao chamador.

Consulte as quatro fichas completas neste capítulo, a referência no Volume II e `Documentacao/Usuario/Coral_Esquemas_1_7_8.md`.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`para_json(...)` · **módulo** `coral.json`

O que faz

Transforma valores compatíveis com JSON em texto JSON.

Assinatura

Assinatura para consulta

```
para_json(valor, identar=2)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	valor serializável pelo json do Python	sim	sem padrão	Valor sobre o qual a operação atua.
identar	inteiro, texto ou nulo	não	2	Quantidade de espaços de recuo; nulo produz a representação sem recuo.

Retorno

Texto JSON.

Exemplo

Código Coral

```
de coral.json importe para_json
mostre para_json([1, 2])
```

Resultado

Saída esperada

```
[
  1,
  2
]
```

Forma natural

`serialize json de [1, 2]` é equivalente.

Erros

TypeError para objetos não serializáveis.

Atenção

Preserva Unicode. Chaves compatíveis não textuais são convertidas para texto. NaN e Infinity são permitidos pelo serializador padrão, embora consumidores de JSON estrito possam recusá-los.

`de_json(...)` · módulo `coral.json`

O que faz

Interpreta um texto JSON e reconstrói seus valores.

Assinatura

Assinatura para consulta

```
de_json(texto)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	str, bytes ou bytearray com JSON	sim	sem padrão	Texto a consultar ou transformar.

Retorno

Valor compatível com JSON: lista, dicionário, texto, número, booleano ou nulo.

Exemplo

Código Coral

```
de coral.json importe de_json
mostre de_json("[1, 2]")
```

Resultado

Saída esperada

```
[1, 2]
```

Forma natural

`interprete json de "[1, 2]"` é equivalente.

Erros

JSONDecodeError para texto inválido.

`ler_linhas(...)` · módulo `coral.arquivos`

O que faz

Iterador de textos, preservando terminações de linha. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_linhas(caminho, codificacao='utf-8') -> 'Iterator[str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.

Retorno

Iterador de textos, preservando terminações de linha.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_linhas
execute escrever_texto("linhas.txt", "Ana\nBia")
com ler_linhas("linhas.txt") como linhas faça
    para cada linha em linhas faça
        mostre tamanho de linha
    fim
fim
```

Resultado

Saída esperada

```
4
3
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_csv(...)` · módulo `coral.arquivos`

O que faz

Iterador de objetos com chaves e valores textuais se cabecalho=True; listas de textos caso contrário. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_csv(caminho, codificacao='utf-8', *, delimitador=',', cabecalho=True) ->
  'Iterator[dict[str, str] | list[str]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Iterador de objetos com chaves e valores textuais se cabecalho=True; listas de textos caso contrário.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\n")
com ler_csv("dados.csv") como registros faça
    para cada registro em registros faça
        mostre registro["nome"]
    fim
fim
```

Resultado

Saída esperada

```
Ana
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_lotes_csv(...)` · módulo `coral.arquivos`

O que faz

Iterador de listas de registros; lote final possivelmente menor. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_lotes_csv(caminho, tamanho_lote=100, codificacao='utf-8', *, delimitador=',',
  cabecalho=True) -> 'Iterator[list[dict[str, str] | list[str]]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
tamanho_lote	inteiro positivo, não booleano	não	100	Máximo de registros por lote: inteiro positivo, exceto booleano.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Iterador de listas de registros; lote final possivelmente menor.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_lotes_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\nBia,20\nCaio,30\n")
com ler_lotes_csv("dados.csv", tamanho_lote=2) como lotes faça
    para cada lote em lotes faça
        mostre tamanho de lote
    fim
fim
```

Resultado

Saída esperada

```
2
1
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`escrever_linhas(...)` · módulo `coral.arquivos`

O que faz

Nulo; o destino recebe os textos sem quebras inseridas. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
escrever_linhas(caminho, linhas: 'Iterable[str]', codificacao='utf-8') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
linhas	iterável de textos	sim	sem padrão	Iterável de textos com as terminações desejadas; sem conversão implícita.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.

Retorno

Nulo; o destino recebe os textos sem quebras inseridas.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_linhas e ler_texto
execute escrever_linhas("saida.txt", ["Ana\n", "Bia"])
mostre ler_texto("saida.txt")
```

Resultado

Saída esperada

```
Ana
Bia
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Escrita substitui o destino e pode ficar parcial em erro; campos precisam ser textos. O chamador fecha leitores de entrada com com externo. Objetos CSV exigem chaves exatas e sequências largura exata.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`escrever_csv(...)` · módulo `coral.arquivos`

O que faz

Nulo; o destino recebe CSV com ordem de colunas explícita ou inferida. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
escrever_csv(caminho, registros: 'Iterable[Mapping[str, str] | Sequence[str]]',
  codificacao='utf-8', *, delimitador=',', colunas=None, cabecalho=True) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
registros	iterável de objetos ou sequências textuais	sim	sem padrão	Iterável de objetos texto/texto ou sequências de textos de largura uniforme.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
colunas	sequência de textos ou nulo	não	None	Sequência ordenada de nomes; nulo infere nomes do primeiro objeto.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Nulo; o destino recebe CSV com ordem de colunas explícita ou inferida.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_csv e ler_csv
execute escrever_csv("saida.csv", [{"nome": "Ana", "valor": "10"}], colunas=["nome",
  "valor"])
com ler_csv("saida.csv") como registros faça
    para cada registro em registros faça
        mostre registro["nome"]
        mostre registro["valor"]
    fim
fim
```

Resultado

Saída esperada

```
Ana
10
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Escrita substitui o destino e pode ficar parcial em erro; campos precisam ser textos. O chamador fecha leitores de entrada com com externo. Objetos CSV exigem chaves exatas e sequências largura exata. Para sequências de campos, nomes não são inferíveis: sem colunas explícitas, não é emitido cabeçalho mesmo com cabecalho=True. Objetos permitem inferir nomes na ordem do primeiro registro; colunas explícitas definem nomes e ordem.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`campo(...)` · **módulo** `coral.esquemas`

O que faz

Cria a especificação declarativa de um campo; não converte nem preenche dados.

Assinatura

Assinatura para consulta

```
campo(tipo: 'str', *, obrigatorio: 'bool' = True, anulavel: 'bool' = False, minimo: 'int
   | float | None' = None, maximo: 'int | float | None' = None, tamanho_minimo: 'int |
  None' = None, tamanho_maximo: 'int | None' = None, permitidos: 'Iterable | None' = None)
   -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	texto	sim	sem padrão	Tipo canônico: inteiro, decimal, texto, booleano, lista ou dicionario.
obrigatorio	booleano	não	True	Exige presença da chave; verdadeiro por padrão.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
anulavel	booleano	não	False	Aceita nulo quando a chave está presente; falso por padrão.
minimo	inteiro ou decimal finito ou nulo	não	None	Limite numérico inferior inclusivo; nulo desativa.
maximo	inteiro ou decimal finito ou nulo	não	None	Limite numérico superior inclusivo; nulo desativa.
tamanho_minimo	inteiro não negativo, exceto booleano, ou nulo	não	None	Comprimento mínimo inclusivo; nulo desativa.
tamanho_maximo	inteiro não negativo, exceto booleano, ou nulo	não	None	Comprimento máximo inclusivo; nulo desativa.
permitidos	iterável de valores ou nulo	não	None	Iterável de valores aceitos; nulo desativa, coleção vazia não aceita nenhum valor não nulo.

Retorno

Dicionário de especificação, copiado e novamente conferido pelo consumidor.

Exemplo

Código Coral

```
de coral.esquemas importe campo
defina idade como campo("inteiro", minimo=0, maximo=120)
mostre idade["tipo"]
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError para tipos de opções inválidos, incluindo limites numéricos não finitos ou booleanos; ValueError para tipo canônico desconhecido, tamanho negativo, regras incompatíveis ou limites contraditórios.

Atenção

Inteiro rejeita booleanos; decimal aceita inteiro/decimal finitos e rejeita booleanos. Regras numéricas só servem para inteiro/decimal; comprimento só para texto/lista/dicionario. Limites são inclusivos. Ausência e nulo são independentes: opcional permite ausência, anulavel permite nulo presente. Nulo aceito não percorre regras de valor. Especificações não retêm referências mutáveis do chamador.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`esquema(...)` · módulo `coral.esquemas`

O que faz

Compõe campos em um esquema de registro com ordem declarada de diagnóstico.

Assinatura

Assinatura para consulta

```
esquema(campos: 'Mapping[str, dict]', *, permitir_extras: 'bool' = True) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campos	mapeamento texto/especificação de campo	sim	sem padrão	Mapeamento de nomes textuais não vazios para especificações de campo.
permitir_extras	booleano	não	True	Permite chaves fora da declaração; verdadeiro por padrão.

Retorno

Dicionário de esquema com especificações copiadas; sem dados do registro.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
defina cadastro como esquema({"nome": campo("texto", tamanho_minimo=1)},
  permitir_extras=falso)
mostre validar_registro({"nome": "Ana"}, cadastro)["valido"]
```

Resultado

Saída esperada

```
True
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError para campos ou opções de tipo inválido; ValueError para nome vazio ou especificação inválida, incluindo regras e limites contraditórios.

Atenção

A ordem dos campos declarados define a ordem dos erros; extras recusados aparecem depois, na ordem do registro. Não há campos aninhados, preenchimento padrão, conversão

ou mutação dos dados. Use especificações produzidas por campo. A conferência ocorre também ao consumir o esquema.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`validar_registro(...)` · módulo `coral.esquemas`

O que faz

Confere um registro e relata falhas sem incluir o valor sensível nem modificar a entrada.

Assinatura

Assinatura para consulta

```
validar_registro(registro: 'Mapping', esquema: 'Mapping', *, indice: 'int' = 1, linha:
  'int | None' = None, origem: 'str | None' = None, modo: 'str' = 'coletar', limite_erros:
   'int' = 100) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registro	mapeamento	sim	sem padrão	Mapeamento com os dados; outra entrada resulta em erro de regra registro.
esquema	mapeamento de esquema	sim	sem padrão	Esquema criado por esquema e reconferido no consumo.
indice	inteiro positivo, exceto booleano	não	1	Identificador lógico do registro, inteiro positivo base 1; padrão 1.
linha	inteiro positivo, exceto booleano, ou nulo	não	None	Linha informada pelo chamador, inteiro positivo ou nulo; não inferida.
origem	texto ou nulo	não	None	Rótulo textual opcional fornecido pelo chamador; nunca inferido.
modo	texto: primeiro ou coletar	não	'coletar'	primeiro interrompe no primeiro erro; coletar usa o limite global.
limite_erros	inteiro positivo, exceto booleano	não	100	Máximo positivo de erros; inteiro excluindo booleano, padrão 100.

Retorno

Dicionário com valido (booleano), erros (lista) e interrompido (booleano). Cada erro contém campo, regra, mensagem, registro (índice base 1), linha e origem; não contém valor.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
defina contrato como esquema({"apelido": campo("texto", obrigatorio=falso)})
mostre validar_registro({}, contrato)["valido"]
defina resultado como validar_registro({"apelido": nulo}, contrato, indice=3, linha=8,
  origem="cadastro.json")
mostre resultado["erros"][0]["regra"]
mostre resultado["erros"][0]["registro"]
mostre resultado["erros"][0]["linha"]
```

Resultado

Saída esperada

```
True
nulo
3
8
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError/ValueError para esquema ou opções inválidos. Dados inválidos são diagnósticos, não exceções: registro, ausente, nulo, tipo, minimo, maximo, tamanho_minimo, tamanho_maximo, permitido e extra.

Atenção

Tipo errado encerra a avaliação daquele campo. Ausente opcional é válido; nulo presente exige anulavel. primeiro e coletar ao alcançar limite marcam interrompido, sem afirmar exame do restante. Erros seguem ordem declarada e depois extras. Não há conversão; nenhum dado de entrada é alterado.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`validar_registros(...)` · módulo `coral.esquemas`

O que faz

Valida um iterável em passagem única e acumula apenas erros, com limite global.

Assinatura

Assinatura para consulta

```
validar_registros(registros: 'Iterable[Mapping]', esquema: 'Mapping', *, modo: 'str' =
  'coletar', limite_erros: 'int' = 100, linha_inicial: 'int | None' = None, origem: 'str |
   None' = None) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registros	iterável de mapeamentos	sim	sem padrão	Iterável de registros; cada item não mapeamento produz erro registro.
esquema	mapeamento de esquema	sim	sem padrão	Esquema validado antes de avançar a entrada.
modo	texto: primeiro ou coletar	não	'coletar'	primeiro interrompe no primeiro erro; coletar usa o limite global.
limite_erros	inteiro positivo, exceto booleano	não	100	Máximo positivo de erros; inteiro excluindo booleano, padrão 100.
linha_inicial	inteiro positivo, exceto booleano, ou nulo	não	None	Linha lógica do primeiro registro; nulo omite linha. O índice seguinte soma 1.
origem	texto ou nulo	não	None	Rótulo textual opcional fornecido pelo chamador; nunca inferido.

Retorno

Dicionário com valido (booleano), erros (lista), verificados (número de registros consumidos) e interrompido (booleano). Os erros têm campo, regra, mensagem, registro, linha e origem.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registros
defina inventario como esquema({"quantidade": campo("inteiro", minimo=0)})
defina resultado como validar_registros([{"quantidade": -1}, {"quantidade": -2},
  {"quantidade": 4}], inventario, limite_erros=1, linha_inicial=2,
  origem="inventario.csv")
mostre resultado["verificados"]
mostre resultado["interrompido"]
mostre resultado["erros"][0]["linha"]
```

Resultado

Saída esperada

```
1
True
2
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError/ValueError para esquema, iterável ou opções inválidos; esquema e opções falham antes do primeiro consumo. Falhas do iterador de entrada propagam ao chamador. Dados incompatíveis produzem as regras de validar_registro.

Atenção

Índice de registro é base 1 e linha = linha_inicial + índice - 1. Isso é contagem lógica, não linha física de CSV multilinha. origem não é inferida. Limite positivo é global; nenhum registro adicional é lido após parar. Não guarda registros nem fecha iteradores de terceiros; o chamador mantém o contexto com e o fechamento. Consumo não altera dados; conversão é explícita antes da validação.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

O que você aprendeu

Arquivos podem ser processados em passagem única. JSON conserva estruturas básicas; CSV oferece textos que exigem conversão explícita. Esquemas distinguem chave ausente de nulo e acumulam erros com limite global; o chamador mantém o fechamento do leitor.

Prática

Converta um dicionário para JSON e reconstrua o valor.

Conferir a resposta comentada

Use `para_json` para obter texto JSON e `de_json` para recuperar o dicionário. A ida e volta preserva os valores compatíveis com JSON, não objetos arbitrários ou identidade de instância.

Resposta do capítulo 27 no Apêndice B.

28. Matemática, datas e aleatoriedade

Objetivo de leitura

Usar cálculos fundamentais e distinguir datas, durações e aleatoriedade.

Antes de começar

Conheça números e coleções.

A biblioteca fundamental reúne cálculo, datas e aleatoriedade. Use `coral.matematica` para operações escalares e reduções simples. Use `coral.numerico`, apresentado no Cap. 38, para matrizes e estatística mais ampla.

Código Coral

```
de coral.matematica importe soma, media, raiz
mostre soma([2, 4, 6])
mostre media([2, 4, 6])
mostre raiz(9)
```

Saída esperada

```
12
4.0
3.0
```

A saída é 12, 4.0 e 3.0, em linhas separadas. A média exige observações; a raiz real exige domínio válido. Evite concluir que toda redução aceita vazio ou todo cálculo aceita infinito.

Datas e durações têm papéis diferentes: uma data identifica um dia, uma duração representa um intervalo. A referência de `coral.datas` no Volume II, Parte II, mostra os formatos e fusos.

Uma fonte pseudoaleatória com semente ajuda a repetir simulações. Ela não serve para senhas ou tokens. A semente e a versão do algoritmo fazem parte do contexto de reprodução. O Cap. 37 usa esse princípio em receitas procedurais; `coral.criptografia` fornece operações explícitas para valores secretos.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`soma(...)` · **módulo** `coral.matematica`

O que faz

Soma os valores da coleção.

Assinatura

Assinatura para consulta

```
soma(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores somáveis	sim	sem padrão	Coleção de entrada.

Retorno

Número.

Exemplo

Código Coral

```
de coral.matematica importe soma
mostre soma([2, 4, 6])
```

Resultado

Saída esperada

```
12
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`media(...)` · módulo `coral.matematica`

O que faz

Calcula a média aritmética de observações.

Assinatura

Assinatura para consulta

```
media(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável não vazio de valores numéricos	sim	sem padrão	Coleção de entrada.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe media
mostre media([2, 4, 6])
```

Resultado

Saída esperada

```
4.0
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`raiz(...)` · módulo `coral.matematica`

O que faz

Calcula a raiz quadrada real.

Assinatura

Assinatura para consulta

```
raiz(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real não negativo	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe raiz
mostre raiz(9)
```

Resultado

Saída esperada

```
3.0
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`absoluto(...)` · módulo `coral.matematica`

O que faz

Obtém a magnitude sem o sinal.

Assinatura

Assinatura para consulta

```
absoluto(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número ou objeto com __abs__	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Número.

Exemplo

Código Coral

```
de coral.matematica importe absoluto
mostre absoluto(-3)
```

Resultado

Saída esperada

```
3
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`potencia(...)` · módulo `coral.matematica`

O que faz

Eleva a base ao expoente.

Assinatura

Assinatura para consulta

```
potencia(base, expoente)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
base	valor compatível com potência	sim	sem padrão	Base da potência.
expoente	expoente compatível com a base	sim	sem padrão	Expoente aplicado à base.

Retorno

Número produzido pela potência; o tipo depende dos operandos.

Exemplo

Código Coral

```
de coral.matematica importe potencia
mostre potencia(2, 3)
```

Resultado

Saída esperada

```
8
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

O que você aprendeu

Cada cálculo tem um domínio válido. Datas identificam momentos; durações medem intervalos; sementes ajudam a reproduzir simulações.

Prática

Calcule a média de 2, 4 e 6.

Conferir a resposta comentada

A soma é 12 e a quantidade é 3, portanto a média é 4. `coral.matematica.media([2, 4, 6])` devolve 4.0. Não use uma coleção vazia.

Resposta do capítulo 28 no Apêndice B.

29. Persistência

Objetivo de leitura

Salvar estado recuperável e distinguir dados persistentes de recursos ativos.

Antes de começar

Conheça JSON e objetos, dos capítulos 16 e 27.

A superfície pública `coral.persistencia/1` usa um envelope versionado com contrato, versão de esquema, tipo Coral e carga útil compatível com JSON. Mundo, RPG, Jogos e Laboratório podem persistir estado sem serializar callbacks, janelas, listeners vivos ou identidade acidental de objetos Python.

Em Jogos, por exemplo:

Código Coral

```
crie um jogo chamado aventura sem janela
salve o jogo aventura em "partida.coral-save"
carregue o jogo de "partida.coral-save" como aventura_reaberta
```

Este exemplo não imprime texto no terminal.

Um jogo reaberto usa `BackendNulo`, começa parado e restaura somente estado persistente. Combates contextuais preservam participantes, ordem, índice e rodada utilizando as entidades reconstruídas do Mundo.

SQLite local na Coral 1.7.9

Para aplicações que precisam consultar dados depois de encerrar o programa, `coral.sqlite` oferece uma camada pequena sobre SQLite. SQL permanece explícito. Valores entram por parâmetros separados do texto SQL; parâmetros não substituem nomes de tabela, coluna ou fragmentos de consulta.

Código Coral

```
de coral.sqlite importe abrir

defina banco como abrir(":memoria:")
defina criar_tabela como """
    CREATE TABLE itens (
        id INTEGER PRIMARY KEY,
        nome TEXT NOT NULL,
        quantidade INTEGER NOT NULL
    )
"""
execute banco.executar(criar_tabela)
execute banco.executar("INSERT INTO itens(nome, quantidade) VALUES (?, ?)", ["Ana", 3])
execute banco.confirmar()
defina item como banco.consultar_um("SELECT nome, quantidade FROM itens WHERE nome = ?",
   ["Ana"])
mostre item["nome"]
mostre item["quantidade"]
execute banco.fechar()
```

Saída esperada

```
Ana
3
```

`abrir(caminho, timeout=5.0)` devolve uma `Conexao`. O caminho especial `:memoria:` cria um banco temporário em memória. `timeout` deve ser numérico, não negativo, e booleanos não são aceitos como números nesse contrato.

Operação da conexão	Contrato de uso
`executar(sql, parametros=())`	Executa um comando e retorna `linhas_afetadas` e `ultimo_id`.
`executar_muitos(sql, parametros)`	Executa o mesmo comando para um iterável de sequências ou mapeamentos.
`consultar(sql, parametros=())`	Materializa todas as linhas como lista de dicionários.
`consultar_um(sql, parametros=())`	Retorna um dicionário ou `nulo` quando não há linha.
`confirmar()`	Confirma alterações pendentes.
`reverter()`	Desfaz alterações pendentes desde a última confirmação.
`transacao()`	Contexto explícito: confirma na saída normal e reverte se o bloco falhar.
`fechar()`	Encerra a conexão; é idempotente. Uso posterior da conexão falha.

Parâmetros aceitam sequência ou mapeamento. Texto e bytes isolados são recusados para evitar vinculação acidental caractere a caractere. `executar_muitos` exige um iterável de conjuntos de parâmetros. Erros operacionais do SQLite são expostos como `ErroOperacionalSQLite`; demais falhas da camada usam `ErroSQLite`, preservando a causa original.

Uma transação contextual não pode ser aninhada e não começa se já houver alteração pendente. A API não cria savepoints implícitos. Para um banco em arquivo, confirme antes de fechar quando quiser conservar as alterações. O projeto `Exemplos/Projetos/Catalogo_SQLite` demonstra cadastro, consulta, atualização, remoção e reabertura persistente.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`abrir(...)` · **módulo** `coral.sqlite`

O que faz

Abre uma conexão SQLite local com timeout explícito e resultados adaptados à API Coral.

Assinatura

Assinatura para consulta

```
abrir(caminho: 'str | Path', *, timeout: 'float' = 5.0) -> 'Conexao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou Path	sim	sem padrão	Caminho do banco. O texto :memoria: cria um banco somente em memória.
timeout	inteiro ou decimal não negativo, exceto booleano	não	5.0	Tempo máximo, em segundos, aguardado pelo SQLite em bloqueios; número não negativo, exceto booleano.

Retorno

Conexao aberta. O chamador deve confirmar ou reverter alterações e fechar a conexão.

Exemplo

Código Coral

```
de coral.sqlite importe abrir
defina banco como abrir(":memoria:")
mostre banco.fechada
execute banco.fechar()
mostre banco.fechada
```

Resultado

Saída esperada

```
False
True
```

Forma natural

Não há forma natural específica. Use abrir(...) de coral.sqlite e mantenha SQL explícito.

Erros

TypeError para caminho/timeout incompatíveis; ValueError para caminho vazio ou timeout negativo; ErroOperacionalSQLite quando o driver não consegue abrir o banco.

Atenção

Parâmetros SQL pertencem às operações da conexão, não a abrir. :memoria: não persiste após o fechamento.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md; Exemplos/Projetos/Catalogo_SQLite.

`Conexao(...)` · módulo `coral.sqlite`

O que faz

Representa uma conexão SQLite com consultas em dicionários, confirmação, reversão e transação contextual explícitas.

Construtor

Assinatura para consulta

```
Conexao(caminho: 'str | Path', *, timeout: 'float' = 5.0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou Path	sim	sem padrão	Caminho do banco. O texto :memoria: cria um banco somente em memória.
timeout	inteiro ou decimal não negativo, exceto booleano	não	5.0	Tempo máximo, em segundos, aguardado pelo SQLite em bloqueios; número não negativo, exceto booleano.

Atributos importantes

Atributo	Significado
`fechada`	Booleano somente leitura que indica se fechar() já encerrou a conexão.
`em_transacao`	Booleano somente leitura que reflete a transação atual; exige conexão aberta.

Métodos principais

Assinatura para consulta

```
executar(self, sql: 'str', parametros=()) -> 'dict[str, int | None]'
```

Referência para leitura

```
executar_muitos(self, sql: 'str', parametros) -> 'dict[str, int | None]'
```

Referência para leitura

```
consultar(self, sql: 'str', parametros=()) -> 'list[dict[str, Any]]'
```

Referência para leitura

```
consultar_um(self, sql: 'str', parametros=()) -> 'dict[str, Any] | None'
```

Referência para leitura

```
confirmar(self) -> 'None'
```

Referência para leitura

```
reverter(self) -> 'None'
```

Referência para leitura

```
fechar(self) -> 'None'
```

Referência para leitura

```
transacao(self) -> "Iterator['Conexao']"
```

Retorno

Instância de Conexao pronta para uso até fechar().

Exemplo

Código Coral

```
de coral.sqlite importe Conexao
defina banco como Conexao(":memoria:")
mostre banco.fechada
execute banco.fechar()
mostre banco.fechada
```

Resultado

Saída esperada

```
False
True
```

Forma natural

Normalmente obtenha a conexão por abrir(...); construir Conexao(...) diretamente é equivalente no contrato atual.

Erros

ErroSQLite para uso após fechamento ou estado transacional incompatível; ErroOperacionalSQLite para falhas operacionais do driver; TypeError/ValueError para argumentos fora do contrato.

Atenção

executar/consultar usam parâmetros separados do SQL. transacao() não aceita aninhamento nem começa sobre alteração pendente. fechar() é idempotente.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md; Exemplos/Projetos/Catalogo_SQLite.

`ErroSQLite(...)` · módulo `coral.sqlite`

O que faz

Exceção base da camada SQLite Coral, usada para estado inválido e erros SQLite não classificados como operacionais.

Construtor

Assinatura para consulta

```
ErroSQLite(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Instância de exceção usada para identificar falhas SQLite na camada Coral.

Exemplo

Código Coral

```
de coral.sqlite importe ErroSQLite
de coral.tipos importe e_tipo
defina erro como ErroSQLite("falha")
mostre e_tipo(erro, ErroSQLite)
```

Resultado

Saída esperada

```
True
```

Forma natural

Não há forma natural específica para construir este erro; normalmente ele é produzido pela camada coral.sqlite.

Erros

A própria classe representa erro; não introduz validação adicional ao construtor herdado de RuntimeError.

Atenção

Prefira tratar esta classe ou ErroOperacionalSQLite em torno de operações de banco, em vez de depender diretamente das exceções sqlite3.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md.

`ErroOperacionalSQLite(...)` · módulo `coral.sqlite`

O que faz

Especialização de ErroSQLite para falhas operacionais do driver, como bloqueio, abertura ou SQL rejeitado pelo SQLite.

Construtor

Assinatura para consulta

```
ErroOperacionalSQLite(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Instância de exceção usada para identificar falhas SQLite na camada Coral.

Exemplo

Código Coral

```
de coral.sqlite importe ErroSQLite e ErroOperacionalSQLite
de coral.tipos importe e_tipo
defina erro como ErroOperacionalSQLite("bloqueado")
mostre e_tipo(erro, ErroOperacionalSQLite)
mostre e_tipo(erro, ErroSQLite)
```

Resultado

Saída esperada

```
True
True
```

Forma natural

Não há forma natural específica para construir este erro; normalmente ele é produzido pela camada coral.sqlite.

Erros

A própria classe representa erro operacional; não introduz validação adicional ao construtor herdado.

Atenção

É subclasse de ErroSQLite. Operações coral.sqlite traduzem sqlite3.OperationalError para esta classe preservando a causa original.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md.

O que você aprendeu

Persistência guarda estado declarativo com contrato e esquema. Janelas, funções de reação e recursos ativos não são serializados como se fossem dados comuns.

Prática

Explique por que persistência não deve guardar callbacks e janelas ativas.

Conferir a resposta comentada

Esses objetos dependem de execução e recursos externos. O formato portátil guarda estado declarativo, com contrato e esquema, e reconstrói apenas os tipos que possuem adaptadores explícitos.

Resposta do capítulo 29 no Apêndice B.

PARTE IV: DESENVOLVENDO APLICAÇÕES

30. Projetos, módulos e coral.toml

Objetivo de leitura

Separar módulos, configurar a entrada e executar um projeto completo.

Antes de começar

Conheça funções, classes e o projeto básico do capítulo 6.

Criar a estrutura

Terminal

```
coral novo MinhaFerramenta --template basico
cd MinhaFerramenta
coral executar
coral testar
```

`coral.toml` identifica a raiz do projeto. Ele pode definir nome, versão, arquivo de entrada, requisito de versão Coral, caminhos de módulos e caminhos de testes.

Configuração TOML

```
[projeto]
nome = "minha-ferramenta"
versao = "0.1.0"
[coral]
versao = ">=1.7.4"
[entrada]
arquivo = "principal.coral"
[modulos]
caminhos = ["."]
[testes]
caminhos = ["testes"]
```

Caminhos são relativos à raiz configurada. `coral executar` compila os módulos para `.coral_build` e executa a entrada. Essa pasta contém saída regenerável. Não a use como local dos seus arquivos fonte.

Importar funções locais

Um arquivo `calculos.coral` pode declarar uma função `media`. Em `principal.coral`, use `de calculos importe media` e depois `media(notas)`. Nomeie seus módulos de forma clara e evite colidir com `coral`, `json`, `sys` e outros módulos do ambiente.

Na 1.5.8, a interface pública do módulo também carrega formas naturais exportadas. Uma função declarada com uma forma como `"dobre {numero}"` pode ser usada pelo consumidor

após uma importação seletiva. Alias de importação preserva a forma adaptada, enquanto duas formas externas ambíguas produzem diagnóstico em vez de uma escolha silenciosa.

O build guarda hashes de fonte e de interface. Sem mudança, módulos já válidos são reutilizados. Uma mudança apenas de implementação recompila o módulo afetado; uma mudança de interface também invalida os consumidores. O resultado de um build incremental deve ser equivalente ao de um build limpo.

Recursos de projeto precisam ser declarados no `coral.toml`. O build copia somente os recursos declarados e o runtime resolve seus caminhos a partir do pacote construído, de modo que a pasta de build possa ser movida sem perder o vínculo com esses arquivos.

A execução de um arquivo isolado não equivale à construção de um projeto: para módulos locais Coral, prefira `coral executar` na raiz do projeto. O comando `coral arquivo.coral` é apropriado a programas independentes.

Projeto curto: Cadastro em três arquivos

Separe as classes, a aplicação e os testes de um cadastro. Declare o arquivo de entrada no coral.toml.

A orientação de solução está no Apêndice B, seção Projeto 30.

Contratos entre módulos na 1.7.4

Quando uma classe ou função possui tipos declarados, a análise do consumidor aproveita essa informação. Imports seletivos, aliases e reexportações preservam a origem do símbolo quando ela pode ser resolvida. Métodos herdados e o construtor herdado também podem fornecer contratos.

O editor usa os arquivos abertos para analisar alterações ainda não salvas. Ao mudar o contrato de um módulo, os consumidores abertos recebem diagnósticos atualizados. A análise não executa os módulos importados para descobrir seus tipos.

O build incremental considera o contrato público das dependências transitivas. Por isso, mudar uma assinatura atrás de um módulo que a reexporta também invalida o consumidor. Alterações somente no corpo preservam o contrato público.

O projeto completo de demonstração está em `Exemplos/Citados/Analise_1_7_4/Cadastro`: leia `modelos.coral`, `fachada.coral` e `principal.coral` e execute seus testes na raiz do projeto.

Imports seletivos em linhas separadas

Uma linha que termina em `de modulo importe` abre uma lista de nomes exportados. Cada linha recebe um nome, com alias opcional; `fim` fecha a importação. Esse bloco não possui corpo de execução.

Código Coral

```
de math importe
    sqrt como raiz
    floor
fim
mostre raiz(9)
mostre floor(2.8)
```

Saída esperada

```
3.0
2
```

Isso equivale a `de math importe sqrt como raiz, floor`. A resolução do módulo, os aliases e a ordem de importação permanecem os mesmos. A lista explícita não aceita curingas nem itens vazios. O editor mantém a navegação entre cada item e sua declaração de origem.

Um ambiente próprio para o projeto

Na Coral 1.7.6, `[dependencias].python` declara requisitos Python e `[ambiente].python` limita o interpretador. O fluxo resolve wheels para um alvo específico, registra versões e hashes em `coral.lock`, prepara uma geração isolada e verifica essa geração antes de executar. A execução de arquivo, projeto, build e testes pela CLI usa o mesmo executor quando o projeto adota esse lock.

Configuração TOML

```
[projeto]
nome = "meu-projeto"

[coral]
versao = ">=1.7.6"

[dependencias]
python = ["biblioteca-didatica==1.0"]

[ambiente]
python = ">=3.10,<3.15"
```

O nome acima é ilustrativo. Para um fluxo executável sem registro remoto, use `Exemplos/Projetos/Ambiente_Reproduzivel`, que gera suas próprias wheels didáticas. Com uma pasta de wheels compatíveis, o fluxo é:

Terminal

```
coral manager ambiente resolver --projeto . --cache wheels --offline --json
coral manager ambiente preparar --projeto . --offline --aplicar --json
coral manager ambiente verificar --projeto . --json
coral executar
```

Sem `--permitir-rede`, a resolução não consulta um registro remoto. Preparar sem `--aplicar` apenas apresenta o plano. Uma falha na instalação conserva a geração ativa anterior; repetir a preparação reutiliza uma geração válida. Um ambiente ausente, alterado ou incompatível impede a execução de um projeto que adotou lock. Projetos antigos sem lock conservam seu fluxo anterior.

Esta edição aceita wheels e requisitos de nome e versão. URLs, caminhos, VCS, extras e marcadores escritos no manifesto ficam fora desse contrato inicial. O lock pertence ao alvo registrado: não copie uma venv pronta para outra máquina ou caminho. Veja `Documentacao/Usuario/Coral_Ambientes_Reproduziveis_1_7_6.md` para restrições, cancelamento e os limites de qualificação.

O que você aprendeu

O manifesto organiza o projeto. Imports disponibilizam símbolos; alterações de contrato podem exigir reconstruir consumidores.

Prática

Diferencie arquivo de entrada, importação e bloco programa principal.

Conferir a resposta comentada

O projeto escolhe qual arquivo executar; a importação carrega um módulo; o bloco principal organiza a ação da execução direta. Bibliotecas reutilizáveis não devem iniciar a aplicação durante a importação.

Resposta do capítulo 30 no Apêndice B.

31. Testes

Objetivo de leitura

Escrever verificações e executar testes sem depender de sua ordem.

Antes de começar

Conheça funções e projetos, dos capítulos 15 e 30.

Testes nativos

Desde a linha 1.2, um bloco `teste "nome"` contém verificações e termina com `fim`. Execute com `coral testar`; simplesmente executar um arquivo de testes como programa não é o mesmo fluxo. Na 1.5.8, o runner prepara o projeto uma vez por sessão e reutiliza esse build entre arquivos de teste.

O bloco de teste pode declarar casos parametrizados, erro esperado, motivo de pulo, semente, limite de duração, grupo e recursos temporários. Hooks explícitos de preparação e encerramento continuam separados do teste básico, de modo que um caso simples não precisa carregar toda a infraestrutura avançada.

O conjunto `Exemplos/Citados/Projetos_Completos/Notas/` traz um projeto completo com `calculos.coral`, `principal.coral` e `testes/basico.coral`. Copie essa pasta para seu diretório de trabalho e execute:

Terminal

```
coral executar
coral testar
coral dependencias
coral documentar
```

A execução imprime `8.0` e o teste da média deve passar. `coral documentar` gera uma referência de símbolos; isso não substitui o manual de uso nem inventa explicações das regras de negócio.

O runner isola os módulos locais durante o carregamento de cada arquivo de testes. Testes no mesmo arquivo ainda podem compartilhar objetos importados. Evite depender da ordem de execução.

Projeto curto: Contrato de uma função

Teste o dobro para zero, um valor positivo e um negativo. Acrescente uma entrada textual inválida se a aplicação recebe texto.

A orientação de solução está no Apêndice B, seção Projeto 31.

O que você aprendeu

Testes conferem comportamentos esperados. `coral testar` usa o fluxo próprio do runner; executar um arquivo de testes como programa é outro fluxo.

Prática

Escreva uma verificação para uma função dobro e provoque uma falha intencional.

Conferir a resposta comentada

Use `garanta que dobro(3) for igual a 6`. Trocar 6 por 7 provoca a falha da verificação. Reponha o valor correto antes de considerar o teste aprovado.

Resposta do capítulo 31 no Apêndice B.

32. Regras, eventos e tempo

Objetivo de leitura

Reagir a eventos e mudanças e controlar o tempo lógico.

Antes de começar

Conheça condições e estado, dos capítulos 9 e 12.

A ideia principal da programação reativa em Coral é simples: **algo acontece e uma regra reage**. O programa não precisa transformar cada comportamento em um loop manual nem em callbacks de baixo nível. A superfície natural descreve o acontecimento, a condição opcional e a reação; o runtime mantém ordem determinística, tempo lógico e rastreamento.

O exemplo canônico mais curto está em `Exemplos/Regras/01_eventos_observacao_e_tempo.coral`:

Código Coral

```
defina ultimo_dano como 0
quando ocorrer "dano" como evento se evento.quantidade for maior que 3
    defina ultimo_dano como evento.quantidade
    mostre evento.origem
fim

emita o evento "dano" com quantidade 5 e origem "espada"
```

Saída esperada

```
espada
```

O objeto `evento` expõe os dados nomeados do acontecimento. Nesse exemplo, `quantidade` é carga útil e `origem` identifica quem publicou o evento. A emissão é local e determinística: ela não cria uma thread, não inicia uma tarefa em segundo plano e não transforma o programa em um sistema distribuído.

Observar mudanças sem inventar eventos manuais

Quando a intenção é reagir a um valor que mudou, diga isso diretamente:

Código Coral

```
defina vida como 10
quando vida mudar
    mostre vida
fim

avalie as regras
defina vida como 9
avalie as regras
```

Saída esperada

```
9
```

Também existem cruzamentos de limite:

Código Coral

```
quando temperatura passar de 40
    mostre "Temperatura alta."
fim

quando temperatura cair abaixo de 30
    mostre "Temperatura baixa."
fim
```

Este exemplo não imprime texto no terminal.

O exemplo executável é `Exemplos/Regras/01_eventos_observacao_e_tempo.coral`. Uma atribuição que mantém o mesmo valor não conta como mudança. `passar de` e `cair abaixo de` reagem ao cruzamento da fronteira, não a toda avaliação posterior que permaneça do mesmo lado.

Prioridade, cancelamento e consumo

Prioridades nomeadas evitam números mágicos: `baixa`, `normal`, `alta` e `crítica`. Reações de maior prioridade são consideradas primeiro; empates preservam a ordem de registro.

Código Coral

```
quando ocorrer "rpg.dano.antes" como evento com prioridade crítica
    cancele evento
fim
```

Este exemplo não imprime texto no terminal.

Cancelar só tem efeito sobre acontecimentos declarados como canceláveis e deve ocorrer **antes** do efeito. A Coral não tenta desfazer uma alteração já aplicada. O RPG usa esse contrato em `rpg.dano.antes`: se o evento for cancelado, a vida não é modificada. Veja `Exemplos/Regras/02_prioridade_e_cancelamento.coral`.

`consuma evento` tem outro significado: impede que o acontecimento continue sua propagação para níveis superiores, mas não representa por si só o cancelamento da operação de domínio.

Limites, cooldown e debounce

Uma reação pode executar somente uma vez ou um número máximo de vezes:

Código Coral

```
uma vez quando ocorrer "visita"
    mostre "Primeira visita."
fim
```

Este exemplo não imprime texto no terminal.

Cooldown limita a frequência de um evento:

Código Coral

```
quando ocorrer "ataque" no máximo uma vez a cada 2 segundos
    mostre "Ataque aceito."
fim
```

Este exemplo não imprime texto no terminal.

Debounce é adequado quando a ação deve ocorrer somente depois de um período sem novas mudanças:

Código Coral

```
quando texto mudar espere 300 milissegundos sem novas mudanças faça
    mostre texto
fim
```

Este exemplo não imprime texto no terminal.

Essas formas usam o tempo lógico do motor. Não usam `sleep` escondido.

Tempo determinístico e relógio de teste

Temporizadores continuam disponíveis com `depois de` e `a cada`. Em testes, o relógio pode ser avançado instantaneamente:

Código Coral

```
defina pulsos como 0
a cada 2 segundos faça
    adicione 1 a pulsos
fim

avance o relógio de teste em 5 segundos
mostre pulsos
```

Saída esperada

```
2
```

O exemplo `Exemplos/Regras/01_eventos_observacao_e_tempo.coral` valida tempo sem esperar cinco segundos reais. Isso mantém testes rápidos e reproduzíveis.

Grupos e regras temporárias

Regras nomeadas podem ser reunidas em grupos:

Código Coral

```
regra "chuva_venenosa"
    quando falso
    então
        passe
fim
regra "raios"
    quando falso
    então
        passe
fim
grupo tempestade faça
    use chuva_venenosa
    use raios
fim

desative grupo tempestade
ative grupo tempestade
```

Este exemplo não imprime texto no terminal.

Uma regra desativada também pode ser ligada por duração ou até uma condição:

Código Coral

```
defina temperatura como 35
regra "alerta"
    quando temperatura for maior que 30
    então
        passe
fim
ative alerta durante 30 segundos
ative alerta até temperatura for menor que 30
```

Este exemplo não imprime texto no terminal.

A duração é cooperativa: o motor aplica a expiração quando seu tempo lógico avança.

Sequências de dois acontecimentos

A 1.5.6 reconhece sequências deliberadamente simples A → B:

Código Coral

```
quando ocorrer "botao_a" e depois ocorrer "botao_b" dentro de 2 segundos
    mostre "Combinação reconhecida."
fim
```

Este exemplo não imprime texto no terminal.

A janela temporal é opcional. Esta edição não transforma a linguagem em um processador geral de eventos complexos; sequências arbitrariamente longas permanecem fora do escopo.

Eventos espaciais em 2D e 3D

Os observadores espaciais reutilizam a baseline geral da linguagem:

Código Coral

```
defina jogador como posição(-1, 5)
defina floresta como região(retângulo(0, 0, 10, 10))

quando jogador entrar em floresta
    mostre "Entrou na floresta."
fim
```

Este exemplo não imprime texto no terminal.

Também há `sair de`, proximidade, toque e interseção. As mesmas ideias funcionam em 3D; misturar 2D e 3D continua exigindo conversão explícita. O exemplo canônico é `Exemplos/Regras/03_eventos_espaciais.coral`.

Estados observáveis

Para uma pequena máquina de estados, declare a intenção sem construir uma classe auxiliar:

Código Coral

```
estado porta começa como fechada

quando ocorrer "abrir_porta"
    mude porta para aberta
fim
```

Este exemplo não imprime texto no terminal.

Esse recurso é um estado observável leve. Ele não é um statechart hierárquico e não cria estados paralelos implicitamente.

Histórico e explicação

O comando:

Código Coral

```
mostre histórico de eventos
```

Saída esperada

```
Nenhum evento ou regra foi registrado no histórico reativo.
```

ativa rastreamento detalhado desde o início do programa e apresenta a cadeia de decisões reativas. O runtime registra o resultado de uma condição quando ela é avaliada; o modo Ensinar usa esse registro para explicar por que uma regra executou ou foi suprimida **sem reavaliar a condição** e sem repetir efeitos colaterais.

Motores e avaliação

`avalie as regras` processa condições e observadores no motor padrão. `atualize as regras por ... segundos` também avança tempo lógico, timers, cooldowns, debounce e ativações temporárias. Jogos podem atualizar seu próprio motor durante o ciclo de quadros, enquanto Mundo e RPG publicam acontecimentos de domínio no mesmo modelo de eventos.

Quando nenhum motor específico é informado, a superfície natural usa o motor padrão. Formas históricas como `quando ocorrer "moeda" recebendo valor` continuam válidas por compatibilidade.

API avançada

A API orientada a objetos permanece em `coral.regras`, `coral.tempo_eventos` e módulos auxiliares para ferramentas, integrações e casos que a superfície natural não cobre. Código introdutório deve preferir as frases deste capítulo. `regra`, eventos, observadores, sequências, grupos e controles reativos são nós do parser e da AST Coral; não dependem de substituição textual frágil nem de interpretação por IA.

Projeto curto: Alerta de estoque

Modele uma quantidade de itens e uma regra que informe quando o estoque ficar abaixo de três. Controle o tempo com relógio de teste.

A orientação de solução está no Apêndice B, seção Projeto 32.

O que você aprendeu

Uma regra reage a um acontecimento. Ordem, prioridade e tempo são explícitos; cancelar um evento e consumir sua propagação têm papéis diferentes.

Prática

Diferencie evento, regra e relógio de teste.

Conferir a resposta comentada

O evento registra um acontecimento; a regra escolhe como reagir; o relógio de teste permite controlar o tempo explicitamente. Uma passagem de tempo simulada não deve depender de espera real.

Resposta do capítulo 32 no Apêndice B.

33. Mundo

Objetivo de leitura

Organizar valores em células, camadas e regiões de um mapa.

Antes de começar

Conheça coleções e posição como conjunto de coordenadas.

A Coral 1.5.7 introduz uma infraestrutura discreta de mapas em `coral.mundo`. Ela não é uma engine de jogos: um mapa organiza **valores no espaço**, permite consultar relações locais e comunica mudanças usando a infraestrutura reativa já existente. O mesmo contrato serve para simulações, ciência, robótica, logística, autômatos celulares e, opcionalmente, jogos.

A ideia central é separar cinco coisas que costumam ser confundidas:

- uma **posição** é uma localização contínua no espaço, como `(2.4, 7.1)`;
- uma **célula** é uma localização discreta inteira, como `(2, 7)`;
- uma **camada** diz que tipo de informação está sendo armazenada;
- o **valor** é o dado guardado naquela célula e camada;
- uma **região** agrupa células ou associa uma forma espacial a uma parte do mapa.

Célula e posição

Células usam coordenadas inteiras. Uma topologia quadrada usa coluna e linha; uma topologia hexagonal usa `q` e `r`; e um mapa voxel usa `x`, `y` e `z`. Posições contínuas continuam pertencendo à baseline espacial apresentada no capítulo anterior.

Código Coral

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
defina 31.5 na célula (4, 2) da camada temperatura do mapa clima
defina leitura como o valor da célula (4, 2) da camada temperatura do mapa clima
garanta que leitura for igual a 31.5
```

Este exemplo não imprime texto no terminal.

Ausência não significa `nulo`, zero, falso ou texto vazio. Uma célula pode simplesmente não possuir conteúdo em determinada camada.

Topologia

A topologia define a dimensionalidade, a forma de vizinhança e a conversão entre célula e coordenada de mundo.

Código Coral

```
crie um mapa chamado colmeia com topologia hexagonal
defina vizinhas como as células vizinhas de (2, menos 1) no mapa colmeia
garanta que quantidade de vizinhas for igual a 6
```

Este exemplo não imprime texto no terminal.

Código Coral

```
crie um mapa chamado volume com topologia voxel
adicione uma camada chamada densidade ao mapa volume
defina 0.82 na célula (3, 4, 5) da camada densidade do mapa volume
defina amostra como o valor da célula (3, 4, 5) da camada densidade do mapa volume
garanta que amostra for igual a 0.82
```

Este exemplo não imprime texto no terminal.

A topologia quadrada possui quatro vizinhos ortogonais por padrão. A hexagonal possui seis vizinhos axiais. A voxel possui seis vizinhos ortogonais em três dimensões. Mapas podem ser finitos ou não limitados; uma célula ausente em um mapa não limitado continua sendo uma coordenada válida, apenas sem conteúdo.

Camada e valor

Camadas são nomeadas, ordenadas e esparsas. Elas podem armazenar números, textos, booleanos, listas, dicionários e outros valores portáteis reconhecidos pela persistência Coral. `BlocoMapa` existe como conveniência para terreno ou material, mas não é a unidade obrigatória do sistema.

Código Coral

```
crie um mapa chamado laboratorio com topologia quadrada
adicione uma camada chamada temperatura ao mapa laboratorio
adicione uma camada chamada umidade ao mapa laboratorio
defina 22.0 na célula (1, 0) da camada temperatura do mapa laboratorio
defina 0.61 na célula (1, 0) da camada umidade do mapa laboratorio
defina t como o valor da célula (1, 0) da camada temperatura do mapa laboratorio
defina u como o valor da célula (1, 0) da camada umidade do mapa laboratorio
garanta que t for igual a 22.0
garanta que u for igual a 0.61
```

Este exemplo não imprime texto no terminal.

Vizinhança

Vizinhança pertence à topologia, não ao domínio do programa. Isso permite que o mesmo comando seja usado por um autômato celular, um robô ou um jogo.

Código Coral

```
crie um mapa chamado grade com topologia quadrada
defina proximas como as células vizinhas de (4, 2) no mapa grade
garanta que quantidade de proximas for igual a 4
```

Este exemplo não imprime texto no terminal.

Consultas de mapa são expressões e podem participar de cálculos e comparações. Por exemplo, `soma mais o valor da célula celula da camada temperatura do mapa laboratorio` permanece uma única expressão estrutural da linguagem.

Região

Uma região pode ser formada por células explícitas ou, pela API avançada, por uma forma espacial serializável. Regiões discretas servem para setores de armazém, biomas, zonas de segurança, territórios ou qualquer agrupamento irregular.

Código Coral

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
crie uma região chamada zona_quente no mapa clima com as células [(4, 2), (5, 2), (5,
  3)]
defina presentes como as regiões da célula (5, 2) no mapa clima
garanta que quantidade de presentes for igual a 1
```

Este exemplo não imprime texto no terminal.

Em mapas não limitados, uma região definida por forma não precisa materializar uma quantidade infinita de células. Quando uma enumeração puder ser infinita, a API exige limites explícitos.

Travessia

Travessia é opcional e genérica. Um conteúdo pode permitir ou bloquear passagem, declarar custo não negativo e registrar um motivo. A 1.5.7 fornece adjacência e custos, mas não introduz um algoritmo oficial de caminhos.

Código Coral

```
crie um mapa chamado deposito com topologia quadrada
adicione uma camada chamada ocupacao ao mapa deposito
coloque o bloco "estante" na célula (8, 3) da camada ocupacao do mapa deposito
bloqueie a travessia pela célula (8, 3) da camada ocupacao do mapa deposito porque
  "corredor ocupado"
```

Este exemplo não imprime texto no terminal.

Uma consulta pura de travessia não publica eventos. Uma tentativa explícita pode publicar `mapa.travessia_bloqueada` no barramento já utilizado pelo mundo.

Eventos de mapa

Mapas associados a `Mundo` usam **o mesmo barramento** `mundo.regras.eventos`; não existe um motor de regras oculto por mapa. Um mapa independente também funciona sem barramento quando o programa não precisa de reatividade.

Código Coral

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
defina ultima como 0

quando o valor da célula (4, 2) da camada temperatura do mapa clima mudar como evento
    defina ultima como evento.valor_novo
fim

defina 31.5 na célula (4, 2) da camada temperatura do mapa clima
garanta que ultima for igual a 31.5
```

Este exemplo não imprime texto no terminal.

Também é possível observar entrada e saída de regiões. O resolvedor de localização é relido a cada avaliação, portanto mover o alvo não congela a posição inicial.

Persistência

`Mapa` participa do envelope `coral.persistencia/1` com o identificador `coral.mundo/Mapa`. O formato recomendado para um mapa isolado é `.coralmapa`. Topologia, camadas, conteúdos portáteis e regiões são preservados; Pygame, superfícies, callbacks e recursos gráficos abertos não fazem parte do arquivo.

Na API avançada, a persistência genérica já é suficiente:

Código Python

```
from coral.persistencia import carregar, salvar

salvar(mapa, "clima.coralmapa")
restaurado = carregar("clima.coralmapa")
```

`Mundo` também persiste seus mapas nomeados. Documentos de versões anteriores que não possuem o campo `mapas` continuam sendo carregados com zero mapas.

Visualização opcional com Jogos

`coral.jogos` não define o significado de um mapa. Ele oferece apenas conversões de tela e um adaptador 2D para mapas quadrados e hexagonais. Mapas voxel continuam válidos no núcleo, mas não recebem uma projeção 3D implícita.

Código Coral

```
de coral.jogos importe Camera e RenderizadorMapa2D e BackendNulo

crie um mapa chamado sala com topologia quadrada
adicione uma camada chamada terreno ao mapa sala
coloque o bloco "parede" na célula (1, 1) da camada terreno do mapa sala

defina camera como Camera(0, 0, 64, 64)
defina backend como BackendNulo()
defina renderizador como RenderizadorMapa2D(sala, camera=camera)
defina desenhadas como renderizador.desenhar(backend)
garanta que desenhadas for igual a 1
```

Este exemplo não imprime texto no terminal.

O conjunto de exemplos oficiais inclui uma grade de temperatura, um autômato celular e um mapa de ocupação robótica justamente para provar que visualização é um consumidor opcional e não a fundação da API.

`coral.mundo` é a API oficial da Coral 1.2 para mundos, entidades, atributos, inventários, relações e pequenos modelos de simulação. Internamente, relações formam um grafo dirigido, mas a superfície recomendada é escrita como Coral corrente.

Forma recomendada

Código Coral

```
crie um mundo chamado campanha com nome "Minha Campanha"

crie uma entidade chamada ana com nome "Ana" no mundo campanha
crie uma entidade chamada castelo com nome "Castelo" no mundo campanha

relacione ana com castelo como "viaja para" no mundo campanha

encontre um caminho de ana até castelo no mundo campanha como rota
```

Este exemplo não imprime texto no terminal.

A intenção é que a leitura revele o modelo sem exigir que o iniciante conheça construtores, métodos ou a implementação em grafos.

Entidades

Entidades podem guardar atributos, tags e inventário. A API de baixo nível permanece disponível para código avançado, interoperabilidade e extensão da biblioteca.

Relações e caminhos

`relacione` cria uma relação dirigida. `encontre um caminho` usa a busca de caminho fornecida por `Mundo`, atualmente baseada em busca em largura para caminhos não ponderados.

Eventos e regras

Cada mundo possui um `MotorRegras`, portanto pode hospedar regras e eventos próprios:

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
defina vida como 2
regra "perigo" em campanha.regras
    quando vida for menor ou igual a 3
    então
        mostre "Perigo!"
fim
```

Este exemplo não imprime texto no terminal.

API orientada a objetos

Formas como `Mundo(...)`, `mundo.criar_entidade(...)` e `mundo.relacionar(...)` continuam válidas. Elas são consideradas superfície avançada e não a forma principal dos templates e tutoriais oficiais quando existe frase Coral equivalente.

Precisões preservadas na linha 1.2

As relações são dirigidas, mas `Mundo.caminho` e a frase natural de busca consideram ambas as direções por padrão (`bidirecional=True`). A API avançada aceita `bidirecional=falso` numa expressão Coral para seguir somente saídas. A implementação atual rejeita entidades externas que tenham apenas o mesmo nome de uma entidade local. `para_json` exporta o estado genérico do mundo; não é um formato completo de persistência de personagens e combates.

Um mundo reúne entidades e relações. Entidades têm identidade própria: dois objetos com o mesmo nome não são intercambiáveis. A biblioteca recusa relações com objetos externos que apenas repetem o nome de uma entidade registrada.

Arquivo: `Exemplos/Citados/Mundo/01_mundo_e_rpg.coral`.

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie uma entidade chamada vila no mundo campanha
crie uma entidade chamada bosque no mundo campanha
relacione vila com bosque como "estrada" no mundo campanha
encontre um caminho de bosque até vila no mundo campanha como rota
mostre quantidade de rota
```

Saída esperada

```
2
```

A relação criada é dirigida, mas a busca padrão considera ambas as direções. Por isso, o exemplo encontra um caminho de bosque para vila. A API avançada `campanha.caminho(bosque, vila, bidirecional=falso)` permite seguir apenas a direção de saída. A busca usa BFS para caminhos não ponderados.

Personagens e turnos

Arquivo: `Exemplos/Citados/Mundo/01_mundo_e_rpg.coral`.

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina no mundo campanha com 12 de vida
crie um personagem chamado monstro no mundo campanha com 6 de vida
role um dado de 6 lados usando a semente 7 como ataque
cause ataque.total de dano a monstro
cure 1 de vida de monstro
mostre monstro.vida
crie um combate chamado batalha com heroina e monstro
inicie o combate batalha usando a semente 4
mostre batalha.rodada
```

Saída esperada

```
4
1
```

O dano reduz vida sem deixá la negativa; a cura respeita o limite máximo. A semente torna a rolagem reproduzível para o exemplo. O combate organiza iniciativa e turnos, sem fornecer um sistema completo de regras de RPG. Use `avance o combate batalha` para avançar ao participante seguinte.

Referência dos componentes

Componente	Elementos principais
`coral.mundo`	`Mundo`, `Entidade`, `Relacao`, `criar_mundo`
Entidade	atributos, tags, inventário, `obter`, `definir`, `guardar`
Mundo	`criar_entidade`, `relacionar`, `relacionados`, `caminho`, `atualizar`
`coral.rpg`	`Personagem`, `Rolagem`, `CombateTurnos`, `rolar`
Rolagem	expressão, resultados individuais, modificador e `total`
Combate	participantes, ordem, índice, rodada e participante atual

`Mundo.para_json` exporta o estado genérico do mundo. Não é um salvamento integral de campanha: vida específica de personagem, estado de combate e lógica de retorno precisam de modelagem própria. Não prometa persistência completa usando somente essa função.

Projeto curto: Mapa de ocupação

Crie uma camada de ocupação num mapa quadrado e consulte uma célula ocupada e outra livre, sem janela.

A orientação de solução está no Apêndice B, seção Projeto 33.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`BackendNulo(...)` · **módulo** `coral.jogos`

O que faz

Simula entrada e desenho sem criar uma janela.

Construtor

Assinatura para consulta

```
BackendNulo(*, dt_fixo: 'float | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dt_fixo	float | None	não	None	Intervalo de tempo fixo do backend, quando fornecido.

Atributos importantes

Atributo	Significado
`quadros`	Quantidade de quadros apresentados na simulação.

Métodos principais

Assinatura para consulta

```
definir_teclas(self, *teclas: 'str') -> 'None'
```

Referência para leitura

```
definir_mouse(self, *, posicao: 'tuple[int, int]' = (0, 0), pressionados=(),
  clicados=(), soltos=(), roda: 'tuple[int, int]' = (0, 0)) -> 'None'
```

Referência para leitura

```
processar_eventos(self) -> 'bool'
```

Retorno

Instância de BackendNulo.

Exemplo

Código Coral

```
de coral.jogos importe BackendNulo
defina backend como BackendNulo()
mostre backend.quadros
```

Resultado

Saída esperada

```
0
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`Camera(...)` · módulo `coral.jogos`

O que faz

Controla a posição e a transformação da região visualizada.

Construtor

Assinatura para consulta

```
Camera(x: 'float' = 0.0, y: 'float' = 0.0, largura: 'float' = 800.0, altura: 'float' =
  450.0, alvo: 'Any | None' = None, suavidade: 'float' = 1.0, zoom: 'float' = 1.0,
  rotacao: 'float' = 0.0, zoom_minimo: 'float | None' = None, zoom_maximo: 'float | None'
  = None, deslocamento_visual_x: 'float' = 0.0, deslocamento_visual_y: 'float' = 0.0,
  constante_tempo: 'float | None' = None, zona_morta: 'tuple[float, float] | None' = None,
   enquadramento_x: 'float' = 0.0, enquadramento_y: 'float' = 0.0, antecipacao: 'float' =
  0.0, limites: 'Retangulo | None' = None, travar_x: 'bool' = False, travar_y: 'bool' =
  False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	não	0.0	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	float	não	0.0	Coordenadas verticais da série, ou posição vertical do objeto.
largura	float	não	800.0	Largura em unidades da apresentação.
altura	float	não	450.0	Altura em unidades da apresentação.
alvo	Any | None	não	None	Objeto que a câmera acompanha.
suavidade	float	não	1.0	Fator de suavização usado no acompanhamento.
zoom	float	não	1.0	Fator de ampliação da câmera.
rotacao	float	não	0.0	Ângulo de rotação da apresentação.
zoom_minimo	float | None	não	None	Limite inferior permitido para zoom.
zoom_maximo	float | None	não	None	Limite superior permitido para zoom.
deslocamento_visual_x	float	não	0.0	Deslocamento horizontal adicional da apresentação.
deslocamento_visual_y	float	não	0.0	Deslocamento vertical adicional da apresentação.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
constante_tempo	float | None	não	None	Constante temporal da suavização quando configurada.
zona_morta	tuple[float, float] | None	não	None	Região em que o alvo pode se mover sem deslocar a câmera.
enquadramento_x	float	não	0.0	Deslocamento horizontal desejado no enquadramento.
enquadramento_y	float	não	0.0	Deslocamento vertical desejado no enquadramento.
antecipacao	float	não	0.0	Fator de antecipação do movimento do alvo.
limites	Retangulo | None	não	None	Retângulo que limita o deslocamento da câmera.
travar_x	bool	não	False	Impede acompanhamento horizontal.
travar_y	bool	não	False	Impede acompanhamento vertical.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal da câmera.
`y`	Posição vertical da câmera.
`largura`	Largura da área visualizada.
`altura`	Altura da área visualizada.
`zoom`	Fator de ampliação.

Métodos principais

Assinatura para consulta

```
seguir(self, sprite: 'Sprite', suavidade: 'float' = 1.0, *, constante_tempo: 'float |
  None' = None, zona_morta: 'tuple[float, float] | None' = None, enquadramento:
  'tuple[float, float] | None' = None, antecipacao: 'float | None' = None, travar_x: 'bool
   | None' = None, travar_y: 'bool | None' = None)
```

Referência para leitura

```
atualizar(self, dt: 'float | None' = None)
```

Referência para leitura

```
mundo_para_tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Referência para leitura

```
tela_para_mundo(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Retorno

Instância de Camera.

Exemplo

Código Coral

```
de coral.jogos importe Camera
defina camera como Camera(0, 0, 64, 64)
mostre camera.largura
```

Resultado

Saída esperada

```
64.0
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`RenderizadorMapa2D(...)` · módulo `coral.jogos`

O que faz

Converte as células de um mapa em comandos de desenho 2D.

Construtor

Assinatura para consulta

```
RenderizadorMapa2D(mapa: 'Mapa', *, camadas: 'Iterable[str] | None' = None, camera:
  'Camera | None' = None, resolvedor_estilo: 'ResolvedorEstiloMapa | None' = None,
  area_visivel: 'tuple[float, float, float, float] | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	Mapa	sim	sem padrão	Mapa que deve ser visualizado.
camadas	Iterable[str] | None	não	None	Nomes das camadas a considerar.
camera	Camera | None	não	None	Câmera que controla a região e transformação da visualização.
resolvedor_estilo	ResolvedorEstiloMapa | None	não	None	Função que define a aparência de uma célula.
area_visivel	tuple[float, float, float, float] | None	não	None	Retângulo de mundo que delimita a consulta visual.

Atributos importantes

Atributo	Significado
`mapa`	Mapa consultado pelo renderizador.
`camera`	Câmera fornecida, quando houver.
`area_visivel`	Limite explícito de consulta quando usado.

Métodos principais

Assinatura para consulta

```
celulas_visiveis(self) -> 'tuple[Any, ...]'
```

Referência para leitura

```
desenhar(self, backend: 'BackendJogos') -> 'int'
```

Retorno

Instância de RenderizadorMapa2D.

Exemplo

Código Coral

```
de coral.jogos importe RenderizadorMapa2D, BackendNulo
crie um mapa chamado sala com topologia quadrada
adicione uma camada chamada terreno ao mapa sala
coloque o bloco "parede" na célula (1, 1) da camada terreno do mapa sala
defina renderizador como RenderizadorMapa2D(sala, area_visivel=(0, 0, 64, 64))
mostre renderizador.desenhar(BackendNulo())
```

Resultado

Saída esperada

```
1
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

O que você aprendeu

Uma célula é discreta; uma posição pode ser contínua. Camadas separam informações, e uma célula ausente não equivale a guardar zero ou `nulo`.

Prática

Modele dois objetos e uma relação fora do domínio de jogos.

Conferir a resposta comentada

Um laboratório pode modelar equipamentos e medições, com uma relação indicando o equipamento que gerou cada medição. Mundo oferece entidades e relações sem exigir janela ou combate.

Resposta do capítulo 33 no Apêndice B.

34. RPG

Objetivo de leitura

Criar personagens e simular ações com estado e sementes explícitos.

Antes de começar

Conheça Mundo e regras, dos capítulos 32 e 33.

`coral.rpg` é uma camada pequena construída sobre `coral.mundo`. O objetivo é oferecer peças úteis para aventuras, protótipos e demonstrações, sem tentar substituir sistemas completos de RPG.

Personagens

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina com nome "Coralina" no mundo campanha com 12 de vida
crie um personagem chamado slime com nome "Slime de Musgo" no mundo campanha com 6 de
  vida

defina o atributo "forca" de heroina como 2
```

Este exemplo não imprime texto no terminal.

Dados

Código Coral

```
role um dado de 20 lados como ataque
role 3 dados de 6 lados como dano
role um dado de 6 lados usando a semente 7 como dado
```

Este exemplo não imprime texto no terminal.

Sementes permitem exemplos e testes reproduzíveis.

Dano e cura

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina no mundo campanha com 12 de vida
crie um personagem chamado slime no mundo campanha com 20 de vida
role um dado de 6 lados usando a semente 42 como dado
cause 3 de dano a slime
cause dado.total de dano a slime como dano_causado
cure 2 de vida de heroina
```

Este exemplo não imprime texto no terminal.

Combate por turnos

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina no mundo campanha com 12 de vida
crie um personagem chamado slime no mundo campanha com 20 de vida
crie um combate chamado batalha com heroina e slime
inicie o combate batalha usando a semente 4
avance o combate batalha
```

Este exemplo não imprime texto no terminal.

A camada fornece rolagens, personagens, vida, atributos, testes e ordem de turnos. O modelo de mundo, relações e eventos continua vindo de `coral.mundo`.

API avançada

Classes e funções como `Personagem`, `CombateTurnos` e `rolar(...)` permanecem disponíveis para extensão e interoperabilidade. A documentação introdutória prioriza as frases naturais acima.

Projeto curto: Duelo reproduzível

Prepare dois personagens com estado inicial fixo e uma semente conhecida para as jogadas. Registre turnos e estado final.

A orientação de solução está no Apêndice B, seção Projeto 34.

O que você aprendeu

RPG acrescenta vida, dados e turnos sobre o modelo de Mundo. Uma semente conhecida permite reproduzir as rolagens sob o mesmo contrato.

Prática

Identifique os dados que devem ser reproduzíveis numa simulação de combate.

Conferir a resposta comentada

Estado inicial, participantes, regras, ordem de turno e semente ou resultados das jogadas precisam ser controlados. Uma vitória observada numa única execução não comprova equilíbrio.

Resposta do capítulo 34 no Apêndice B.

35. Jogos

Objetivo de leitura

Reconhecer jogo, cena e sprite e testar a lógica antes da apresentação gráfica.

Antes de começar

Conheça eventos e objetos, dos capítulos 16 e 32.

A Coral usa Pygame CE como backend gráfico oficial, mas a superfície principal é `coral.jogos`. O programa deve poder ser lido como uma descrição do jogo, sem obrigar o usuário a escrever o código repetitivo necessário para configurar o backend gráfico.

Instalação do backend gráfico

Terminal

```
python -m pip install pygame-ce
coral jogos-check
```

Exemplo recomendado

Código Coral

```
de coral.jogos importe AZUL e AMARELO

crie um jogo chamado jogo com largura 800 e altura 450 sem janela
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40 e cor AMARELO

adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim

    se a tecla "esquerda" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a esquerda
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenhe o texto "Use as setas" em 20 e 20 no jogo jogo
fim

execute jogo.executar(max_quadros=2)
```

Este exemplo não imprime texto no terminal.

Recursos atuais

A API inclui:

- janela e loop de jogo;
- teclado;
- desenho de formas e texto;
- sprites;
- cenas;
- colisão retangular;
- assets;
- animações;
- câmera simples;
- áudio;
- regras e eventos integrados ao loop.

Geometria compartilhada

Neste modelo, `coral.jogos` continua 2D, mas colisões de sprites reutilizam as relações geométricas gerais da base espacial. A direção da dependência permanece única: Jogos conhece a infraestrutura espacial; a base espacial não conhece sprites, cenas nem Pygame. O contrato histórico de Jogos também é preservado: sobreposição interior conta como colisão, enquanto apenas encostar pelas bordas não conta.

O exemplo `Exemplos/Jogos/01_geometria_compartilhada.coral` executa sem janela e sem Pygame. Ele compara dois sprites sobrepostos, um terceiro apenas tangente e usa, no mesmo programa, uma consulta espacial geral sobre posições comuns. A versão equivalente na biblioteca geral fica em `Exemplos/Citados/Jogos/04_geometria_compartilhada.coral`.

Mouse e tela cheia

A Coral oferece mouse e tela cheia parte do contrato público de `coral.jogos`. `criar_jogo(..., tela_cheia=verdadeiro)` permite iniciar no modo de tela cheia pela API avançada, e `Jogo` pode consultar ou alternar esse estado durante a execução. O comportamento padrão permanece em janela quando a opção é omitida.

A entrada de mouse diferencia estado contínuo de eventos do quadro. `posicao_mouse()` e `botao_mouse_pressionado(...)` consultam o estado atual; `mouse_clicado(...)`, `mouse_solto(...)` e `roda_mouse()` representam eventos observados no quadro corrente. Essa distinção evita confundir manter um botão pressionado com um novo clique. Os nomes de botão aceitos incluem `esquerdo`, `meio` e `direito`.

No `BackendNulo`, `definir_mouse(...)` fixa um estado e `programar_mouse(...)` descreve quadros determinísticos, permitindo testar interação sem abrir janela. O mesmo backend registra mudanças de tela cheia. A abertura real de janela, a entrada do sistema e o modo de tela cheia continuam dependendo do Pygame CE e do ambiente gráfico.

Folhas de sprites e transformações visuais

A biblioteca oferece folhas de sprites para uso sem janela sem tornar Pygame uma dependência do núcleo. `Spritesheet` descreve uma folha ou atlas, `QuadroSprite` representa um recorte imutável, e animações podem consumir esses quadros sem abandonar o formato antigo baseado em caminhos de imagens.

`Sprite` oferece transformação visual independente da colisão e do estado de `coral.mundo`: escala uniforme, rotação em graus, espelhamento horizontal e vertical, origem e pivô local. As origens estruturadas reconhecidas são `topo_esquerdo`, `centro` e `centro_inferior`. Um pivô personalizado é informado como `(x, y)` nas unidades locais anteriores à escala. O ponto escolhido permanece ancorado na posição do sprite enquanto a representação visual é transformada.

A chamada estruturada `Jogo.desenhar_imagem(...)` aceita os mesmos parâmetros opcionais. Omiti los preserva a semântica anterior. O `BackendNulo` registra comandos transformados sem abrir arquivos ou janela; o backend Pygame aplica as operações à imagem carregada ou ao quadro recortado. A abertura de janela e o resultado visual físico continuam sujeitos a teste de campo com Pygame CE.

Estas interfaces pertencem à camada avançada e interoperável. Elas foram mantidas como fundação técnica até os gates de contrato, erros e regressão ficarem verdes; só então partes com ganho pedagógico claro começaram a receber frases Coral naturais.

Na implementação atual, a integração com `coral.mundo` ganha uma ponte visual opcional do lado de `coral.jogos`. `Cena.representar(...)` associa uma entidade a um `Sprite` por atributos de posição explicitamente nomeados, mantendo a entidade como fonte de verdade. A dependência não é invertida: `coral.mundo` continua sem importar jogos, sprites ou Pygame, de modo que a mesma entidade pode existir em simulações sem janela e em aplicações sem interface.

A biblioteca atual não mantém mais o antigo laboratório versionado de desenvolvimento. O exemplo `Exemplos/Citados/Jogos/02_sprites_transformacoes_e_animacao.coral` concentra a superfície estável de sprites, transformações e automações em modo sem janela; validações históricas de spritesheets, recortes e estados permanecem na suíte de regressão.

Na implementação atual, uma primeira fatia da superfície natural é promovida para parser/AST: `carregue a spritesheet ...`, `pegue o quadro ...`, `crie uma animação ...`, `use o quadro ...`, `use a animação ...` e `represente ... com o sprite ... na cena ...`. Essas frases compilam para a API estrutural já estabilizada e são conhecidas pela semântica, tabela de símbolos, Naturalidade, Ensinar e LSP. Métodos diretos continuam válidos como API avançada, mas a documentação para iniciantes prioriza as frases Coral.

O exemplo `Exemplos/Citados/Jogos/02_sprites_transformacoes_e_animacao.coral` demonstra a superfície estável de sprites sem janela e sem Pygame CE.

Na implementação atual, as transformações visuais estabilizadas recebem a superfície natural `ajuste a escala do sprite ...`, `gire o sprite ... em ... graus`, `espelhe o sprite ...` e `ancore o sprite ...`. A ancoragem por origem aceita topo esquerdo, centro

e centro inferior; selecionar uma origem limpa um pivô personalizado anterior. `ancore ... no pivô X e Y` volta a definir um ponto local personalizado que prevalece na transformação. O editor e as ferramentas de análise reconhecem essas formas.

O mesmo exemplo demonstra escala, rotação, espelhamento, ancoragem, colisão associada e automação matemática sem janela.

Na implementação atual, o modelo direcional deixa de tratar `cima`, `baixo`, `esquerda` e `direita` como uma lista estrutural fixa. A base compartilhada passa a ser um conjunto arbitrário de direções nomeadas, cada uma associada a um vetor bidimensional. Duas, quatro e oito direções são apenas predefinições convenientes do mesmo mecanismo. Vetores diagonais são normalizados para preservar a distância efetiva do movimento.

Código Coral

```
de coral.jogos importe Sprite
defina jogador como Sprite(0, 0, 10, 10)
crie um conjunto de direções chamado diagonais
    direção nordeste com vetor 1 e -1
    direção sudoeste com vetor -1 e 1
fim
use movimento com diagonais em jogador
mova jogador 10 para nordeste
```

Este exemplo não imprime texto no terminal.

A animação direcional usa os mesmos nomes livres e usa um bloco declarativo:

Código Coral

```
de coral.jogos importe carregar_spritesheet
defina folha como carregar_spritesheet("heroi.png", 192, 64, 32, 32)
crie uma animação direcional chamada andar usando folha
    direção esquerda na linha 0
    direção direita na linha 1
    quadros 0 até 5
    velocidade 10 quadros por segundo
    repetir
fim
```

Este exemplo não imprime texto no terminal.

Movimento e apresentação não precisam possuir a mesma quantidade de direções. `use movimento em 8 direções no jogador` pode ser ligado a uma animação com apenas esquerda e direita por `ligue a animação andar ao movimento de jogador no sprite visual`. A ponte visual escolhe a direção disponível mais próxima do vetor lógico. O estado também separa direção de movimento e direção de olhar: `pare o movimento de jogador` limpa somente o movimento atual e conserva para onde o alvo estava olhando.

O núcleo compartilhado do movimento direcional vive internamente em `coral.direcoes` e ainda não constitui uma nova biblioteca pública estável. `coral.mundo` continua sem importar sprites, animações ou Pygame, enquanto `coral.jogos` consome o estado direcional apenas para apresentação. O exemplo `Exemplos/Citados/Jogos/02_sprites_transformacoes_e_animacao.coral` reúne a demonstração atual em modo sem janela.

BackendNulo

`BackendNulo` executa a lógica sem abrir uma janela. Ele existe para testes automatizados, CI, simulações e desenvolvimento em ambientes sem vídeo. Isso permite testar a maior parte da API mesmo sem Pygame CE instalado.

Pygame CE

Pygame CE é o backend gráfico oficial de `coral.jogos`. A documentação do Pygame CE cobre janela/display, desenho, eventos, teclado, imagens, áudio, tempo e sprites, que são as capacidades sobre as quais a camada Coral é construída.

API avançada

O runtime também pode ser manipulado por classes e métodos (`Jogo`, `Cena`, `Sprite`, `jogo.executar()`, etc.). Essa camada é válida para extensão da engine e interoperabilidade, mas os templates oficiais priorizam a sintaxe natural Coral quando ela existe.

Instalar a dependência correta

Pygame CE é o backend gráfico oficial. O runtime básico funciona sem ele. Instale Pygame usando o Python da instalação persistente. No Linux, com a pasta padrão:

Terminal

```
"$HOME/.local/share/coral/ambiente/bin/python" -m pip install pygame-ce
coral jogos-check
```

Se você mudou `XDG_DATA_HOME` ou o destino, use o caminho mostrado pelo diagnóstico. A detecção de `pygame` comprova importação, mas não comprova abertura de janela, áudio ou comportamento de drivers.

Criar um jogo

Terminal

```
coral novo MeuJogo --template jogo-2d
cd MeuJogo
coral executar
```

O template contém as importações de cores, uma cena e um sprite controlado pelo teclado. No evento de atualização, `dt` representa o tempo do quadro em segundos. Multiplicar a velocidade por `dt` permite expressar deslocamento por segundo em vez de deslocamento fixo por quadro.

Ler a arquitetura do jogo

Peça	Responsabilidade
Jogo	janela, eventos, ritmo de atualização e encerramento
Cena	agrupar sprites e organizar entrada, saída, atualização e desenho
Sprite	posição, dimensões, aparência, velocidade e colisão retangular
Assets	localizar arquivos de imagem e som dentro de uma raiz
Animacao	alternar quadros ao longo do tempo
Camera	transformar coordenadas do mundo para a tela
MotorRegras	avaliar regras e temporizadores durante o ciclo
BackendNulo	exercitar lógica e registrar comandos sem janela

Cores como `AZUL` e `AMARELO` precisam ser importadas. Um sprite com imagem depende de um arquivo existente. Mantenha caminhos de assets relativos à pasta de trabalho prevista. O build de projetos não copia automaticamente os arquivos de mídia.

Teste sem janela

Arquivo: `Exemplos/Citados/Jogos/01_jogo_e_entrada_sem_janela.coral`.

Código Coral

```
de coral.jogos importe criar_jogo e BackendNulo
defina backend como BackendNulo()
defina jogo como criar_jogo("Teste", backend=backend)
execute jogo.executar(max_quadros=2)
mostre backend.quadros
```

Saída esperada

```
2
```

Essa é uma demonstração deliberada da API avançada para testes. Ela limita a execução a dois quadros e verifica o contador do backend. Não prova que o mesmo ambiente possui suporte gráfico. Para desenvolvimento de um jogo real, teste também janela, teclado, encerramento e áudio no seu computador.

A sintaxe natural completa de criação e callbacks está exemplificada no template `jogo-2d` e em `Documentacao/Desenvolvimento/Projeto/JOGOS.md`. Os componentes `Sprite`, `Cena`, `Assets`, `Animacao` e `Camera` possuem interfaces avançadas em `coral/jogos/objetos.py`.

Construir o primeiro jogo, passo a passo

O arquivo `principal.coral` do template `jogo-2d` tem o programa abaixo. Esta versão de estudo executa dois quadros sem janela, para permitir a verificação automática. Para abrir uma janela, retire `sem janela` da criação do jogo e substitua a última instrução por `inicie jogo`. Esse modo exige Pygame CE e ambiente gráfico e permanece ativo até o encerramento do jogo.

Código Coral

```
# Versão de estudo: executa dois quadros sem janela.
de coral.jogos importe AZUL e AMARELO

crie um jogo chamado jogo com largura 800 e altura 450 sem janela
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40 e cor AMARELO
adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim
    se a tecla "esquerda" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a esquerda
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenhe o texto "Use ← e -> para mover" em 20 e 20 no jogo jogo com tamanho 24
fim

execute jogo.executar(max_quadros=2)
```

Este exemplo não imprime texto no terminal.

1. A importação disponibiliza as duas cores. A criação do jogo define dimensões de 800 por 450 pixels, mesmo neste modo sem janela.
2. A cena reúne o conteúdo visível. O sprite começa em x=100 e y=180, com largura e altura de 40 pixels. Adicioná lo à cena e selecionar a cena são operações separadas.
3. O evento de atualização consulta o estado das setas a cada quadro. `180 vezes dt` desloca o jogador de acordo com o tempo decorrido. As coordenadas de tela crescem para a direita e para baixo.
4. O evento de desenho limpa o fundo e escreve a instrução. Em seguida, o ciclo desenha a cena, incluindo seus sprites. Desenho deve ocorrer novamente a cada quadro.
5. `execute jogo.executar(max_quadros=2)` limita o estudo a dois quadros. No modo com janela, `inicie jogo` entra no ciclo de execução até o encerramento. Para começar, altere a cor, a velocidade e a posição inicial, uma mudança por vez.

O programa não impede o jogador de sair da janela: limites e regras de colisão pertencem à lógica do jogo. Também não carrega imagens ou sons. Acrescente esses recursos depois de conferir o movimento básico.

Referência prática dos objetos gráficos

A superfície natural é a primeira opção para iniciar jogos. As assinaturas abaixo documentam a API de componentes para integração e comportamentos mais específicos. `objeto.metodo(...)` usa a instância que você criou. `self` não é passado nas chamadas.

Operação	Contrato
`criar_jogo(titulo, largura=800, altura=450, fps=60, backend=None)`	Cria o jogo; backend omitido seleciona Pygame
`jogo.executar(max_quadros=None)`	Executa até o encerramento; o limite é útil em testes
`jogo.parar()`	Solicita encerramento do ciclo
`jogo.mudar_cena(cena)`	Sai da cena anterior e entra na nova
`jogo.tecla_pressionada(tecla)`	Consulta uma tecla no backend
`jogo.tocar_som(caminho, volume=1.0)`	Reproduz o arquivo pelo backend; depende de áudio disponível
`jogo.desenhar_imagem(caminho, x, y, largura=None, altura=None, ...)`	Desenha imagem ou `QuadroSprite`; na API atual aceita escala, rotação, espelhamento, origem e pivô opcionais
`Cena(nome)`	Cria uma coleção de sprites e callbacks
`cena.adicionar(sprite)`	Adiciona e devolve o sprite
`cena.remover(sprite)`	Remove se estiver presente
`Sprite(x, y, largura, altura, imagem=None, cor=BRANCO)`	Parâmetros iniciais do sprite; também aceita velocidades, visibilidade, nome, dados, animação e, na API atual, escala, rotação, espelhamento, origem e pivô
`sprite.mover(dx, dy)`	Soma deslocamentos à posição
`sprite.atualizar(dt)`	Avança velocidades e animação; a cena chama essa operação
`sprite.colide_com(outro)`	Testa sobreposição retangular; não resolve fisicamente a colisão
`Assets(raiz="assets")`	Define a raiz dos arquivos de mídia
`assets.imagem(nome)` / `assets.som(nome)`	Resolve um arquivo existente dentro da raiz; rejeita saída da raiz
`Animacao(quadros, fps=10.0, repetir=True)`	Exige ao menos um quadro e fps positivo
`animacao.atualizar(dt)`	Avança o tempo e devolve o quadro atual
`animacao.reiniciar()`	Retorna ao primeiro quadro e zera o tempo
`Camera(x=0, y=0, largura=800, altura=450)`	Define o deslocamento e a área da câmera
`camera.seguir(sprite, suavidade=1.0)`	Define o alvo; suavidade é limitada ao intervalo de 0 a 1
`camera.tela(x, y)`	Converte uma posição do mundo em posição da tela

Para usar uma câmera, atribua sua instância a `cena.camera`. A cena atualiza a câmera e transforma as posições ao desenhar. Para animar, atribua uma `Animacao` a `sprite.animacao`; seus quadros representam caminhos de imagens. A atribuição de

`sprite.velocidade_x` dispensa movimentá lo manualmente por essa mesma velocidade no callback: fazer ambos soma os dois deslocamentos.

Callbacks do jogo recebem `(jogo)`, exceto atualização, que recebe `(jogo, dt)`. Callbacks da cena recebem `(jogo, cena)`; atualização da cena recebe `(jogo, cena, dt)`. Registre funções com `quando_iniciar`, `quando_atualizar`, `quando_desenhar` e `quando_encerrar` no jogo, ou com os registradores correspondentes de entrada, saída, atualização e desenho na cena. Uma assinatura incompatível causa erro na execução.

Projeto curto: Movimento sem janela

Execute dois quadros com BackendNulo e verifique a posição de um sprite antes e depois de um movimento.

A orientação de solução está no Apêndice B, seção Projeto 35.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`Sprite(...)` **· módulo** `coral.jogos`

O que faz

Mantém posição, tamanho, apresentação e dados de um elemento visual.

Construtor

Assinatura para consulta

```
Sprite(x: 'float', y: 'float', largura: 'float', altura: 'float', imagem: 'str | Path |
  QuadroSprite | None' = None, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255,
  alfa=255), velocidade_x: 'float' = 0.0, velocidade_y: 'float' = 0.0, visivel: 'bool' =
  True, nome: 'str' = 'sprite', dados: 'dict[str, Any]' = <factory>, animacao: 'Any' =
  None, escala: 'float' = 1.0, rotacao: 'float' = 0.0, espelhado_horizontalmente: 'bool' =
   False, espelhado_verticalmente: 'bool' = False, origem: 'str' = 'topo_esquerdo', pivo:
  'tuple[float, float] | None' = None, opacidade: 'float' = 1.0, colisao_acompanha_escala:
   'bool' = False, colisao_acompanha_rotacao: 'bool' = False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	float	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
largura	float	sim	sem padrão	Largura em unidades da apresentação.
altura	float	sim	sem padrão	Altura em unidades da apresentação.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
imagem	str | Path | QuadroSprite | None	não	None	Imagem ou quadro do sprite; nulo usa sua representação de cor.
cor	Cor	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)	Cor de apresentação do sprite.
velocidade_x	float	não	0.0	Velocidade horizontal.
velocidade_y	float	não	0.0	Velocidade vertical.
visivel	bool	não	True	Se verdadeiro, o sprite participa da apresentação.
nome	str	não	'sprite'	Nome da série ou do objeto.
dados	dict[str, Any]	não	<factory>	Dicionário livre de dados associados ao sprite; a fábrica cria um dicionário independente.
animacao	Any	não	None	Animação associada ao sprite.
escala	float	não	1.0	Fator de escala visual.
rotacao	float	não	0.0	Ângulo de rotação da apresentação.
espelhado_horizontalmente	bool	não	False	Se verdadeiro, inverte horizontalmente a imagem.
espelhado_verticalmente	bool	não	False	Se verdadeiro, inverte verticalmente a imagem.
origem	str	não	'topo_esquerdo'	Ponto de referência da posição visual.
pivo	tuple[float, float] | None	não	None	Ponto usado como pivô de transformações.
opacidade	float	não	1.0	Intensidade de opacidade entre zero e um.
colisao_acompanha_escala	bool	não	False	Se verdadeiro, a forma de colisão acompanha a escala visual.
colisao_acompanha_rotacao	bool	não	False	Se verdadeiro, a forma de colisão acompanha a rotação visual.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal.
`y`	Posição vertical.
`largura`	Largura da forma base.
`altura`	Altura da forma base.

Atributo	Significado
`dados`	Dados associados ao objeto.

Métodos principais

Assinatura para consulta

```
mover(self, dx: 'float', dy: 'float') -> 'None'
```

Referência para leitura

```
atualizar(self, dt: 'float') -> 'None'
```

Referência para leitura

```
colide_com(self, outro: "'Sprite'") -> 'bool'
```

Retorno

Instância de Sprite.

Exemplo

Código Coral

```
de coral.jogos importe Sprite
defina jogador como Sprite(0, 0, 10, 10)
mostre jogador.largura
```

Resultado

Saída esperada

```
10
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`criar_jogo(...)` · módulo `coral.jogos`

O que faz

Cria um jogo com dimensões, backend e fontes explicitamente configuráveis.

Assinatura

Assinatura para consulta

```
criar_jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *,
   backend=None, mundo=None, relogio=None, eventos=None, tela_cheia: 'bool' = False) ->
  'Jogo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
titulo	str	sim	sem padrão	Valor titulo usado na operação.
largura	int	não	800	Largura em unidades da apresentação.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
altura	int	não	450	Altura em unidades da apresentação.
fps	int	não	60	Frequência alvo de quadros por segundo.
backend	valor dinâmico	não	None	Backend que realiza desenho e entrada; BackendNulo permite executar sem janela.
mundo	valor dinâmico	não	None	Mundo associado ao jogo, quando houver.
relogio	valor dinâmico	não	None	Fonte de tempo que o jogo usa.
eventos	valor dinâmico	não	None	Motor de eventos do jogo.
tela_cheia	bool	não	False	Se verdadeiro, pede modo de tela cheia ao backend.

Retorno

Jogo, ainda sem iniciar seu laço.

Exemplo

Código Coral

```
de coral.jogos importe criar_jogo, BackendNulo
defina jogo como criar_jogo("Teste", backend=BackendNulo())
execute jogo.executar(max_quadros=2)
mostre jogo.backend.quadros
```

Resultado

Saída esperada

```
2
```

`carregar_spritesheet(...)` · módulo `coral.jogos`

O que faz

Define uma folha de sprites dividida em quadros numa grade declarada.

Assinatura

Assinatura para consulta

```
carregar_spritesheet(caminho: 'str | Path', largura_imagem: 'int', altura_imagem: 'int',
   largura_quadro: 'int', altura_quadro: 'int', *, margem: 'int' = 0, espacamento: 'int' =
   0) -> 'Spritesheet'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura_imagem	int	sim	sem padrão	Largura total da imagem declarada.
altura_imagem	int	sim	sem padrão	Altura total da imagem declarada.
largura_quadro	int	sim	sem padrão	Largura de cada quadro da grade.
altura_quadro	int	sim	sem padrão	Altura de cada quadro da grade.
margem	int	não	0	Margem entre a borda da imagem e a grade.
espacamento	int	não	0	Separação entre quadros.

Retorno

Spritesheet.

Exemplo

Código Coral

```
de coral.jogos importe carregar_spritesheet
defina folha como carregar_spritesheet("heroi.png", 96, 64, 32, 32)
mostre len(folha)
```

Resultado

Saída esperada

```
6
```

Atenção

O exemplo descreve a grade. A renderização da imagem exige o recurso correspondente; não está sendo aberta uma janela.

O que você aprendeu

A atualização avança o estado; o desenho apresenta esse estado. `BackendNulo` permite testes sem janela; uma janela real exige o backend gráfico.

Prática

Explique a vantagem de testar um jogo com `` `BackendNulo` ``.

Conferir a resposta comentada

O backend permite exercitar o estado e o fluxo sem janela nem renderização física. Ele não comprova desempenho gráfico ou funcionamento do dispositivo real.

Resposta do capítulo 35 no Apêndice B.

36. Espaço e geometria

Objetivo de leitura

Representar posições e consultar distâncias e relações em 2D e 3D.

Antes de começar

Conheça valores numéricos e coleções.

A Coral trata posição, distância, direção, formas, movimento e relações espaciais como conceitos gerais da linguagem. Eles pertencem à base espacial da linguagem e estão disponíveis **sem importação**. Não existe um módulo público separado para habilitar esses recursos. Jogos é apenas um consumidor 2D dessa infraestrutura.

Posições e formas em 2D

Código Coral

```
defina origem como posição(0, 0)
defina destino como posição(3, 4)
defina distancia_total como distância entre origem e destino
defina area_util como retângulo(0, 0, 100, 60)
mostre distancia_total
mostre area_util
```

Saída esperada

```
5.0
Retangulo(origem=Posicao(coordenadas=(0.0, 0.0)), largura=100.0, altura=60.0)
```

Exemplo completo: `Exemplos/Espaco/01_espaco_2d_e_3d.coral`.

As mesmas operações em 3D

Adicionar uma terceira coordenada cria um valor tridimensional real. A dimensão faz parte do contrato do valor; uma posição 2D não ganha `z = 0` implicitamente.

Código Coral

```
defina origem como posição(0, 0, 0)
defina destino como posição(2, 3, 6)
defina volume como caixa(0, 0, 0, 20, 30, 40)
defina distancia_total como distância entre origem e destino
mostre distancia_total
```

Saída esperada

```
7.0
```

Exemplo completo: `Exemplos/Espaco/01_espaco_2d_e_3d.coral`.

Relações geométricas

As relações são matemáticas e independentes de jogos. `intersecta` aceita contato de borda; `sobrepõe` exige interior comum; `toca` descreve contato sem sobreposição interior. Formas e regiões podem ser usadas em simulações, dados científicos, interfaces, mapas abstratos ou qualquer outro domínio que precise raciocinar espacialmente.

Consultas espaciais sobre coleções comuns

Uma consulta não exige cena, mundo nem classe base especial. A coleção pode conter valores espaciais diretamente ou objetos dos quais a Coral consiga obter posição ou forma.

Código Coral

```
defina sensores como [posição(1, 1), posição(4, 4), posição(20, 0)]
defina proximos como procure em sensores os que estão a até 6 de posição(0, 0)
defina mais_perto como procure em sensores o mais próximo de posição(3, 3)
mostre proximos
mostre mais_perto
```

Saída esperada

```
[Posicao(coordenadas=(1.0, 1.0)), Posicao(coordenadas=(4.0, 4.0))]
Posicao(coordenadas=(4.0, 4.0))
```

Exemplo completo: `Exemplos/Espaco/02_consultas_e_movimento.coral`. O mesmo arquivo mostra a consulta em 2D e 3D; a dimensionalidade vem dos dados.

Movimento também é geral

Velocidade, aceleração, integração por `dt` e limites não pertencem a `coral.jogos`. Isso permite modelar sondas, partículas, robôs, corpos simulados ou trajetórias sem abrir uma janela.

Código Coral

```
defina posicao_inicial como posição(0, 0, 200)
defina velocidade_inicial como velocidade(12, 4, menos 2)
defina gravidade como aceleração(0, 0, menos 9.8)
defina estado como integrar_movimento(posicao_inicial, velocidade_inicial, gravidade,
  dt=0.5)
mostre estado.posicao
```

Saída esperada

```
Posicao(coordenadas=(6.0, 2.0, 197.775))
```

Exemplo completo: `Exemplos/Espaco/02_consultas_e_movimento.coral`.

Relação com coral.jogos

`coral.jogos` permanece 2D nesta versão. Sprites e colisores reutilizam as formas e relações gerais, mas uma forma geométrica não é sinônimo de colisor. Isso permite que a mesma matemática seja útil dentro e fora de jogos sem transformar o restante da linguagem em uma API de game engine.

Projeto curto: Distâncias no espaço

Compare a distância de dois pontos em 2D com um exemplo em 3D e identifique a dimensão de cada dado.

A orientação de solução está no Apêndice B, seção Projeto 36.

O que você aprendeu

Geometria é geral e está disponível sem importação. Dados 2D e 3D têm dimensões próprias; contato de borda e sobreposição interior são relações diferentes.

Prática

Diferencie um vetor de posição e uma transformação em 3D.

Conferir a resposta comentada

O vetor representa coordenadas; a transformação especifica como alterar coordenadas. Em matrizes homogêneas, pontos e direções têm papéis diferentes e não devem ser confundidos.

Resposta do capítulo 36 no Apêndice B.

37. Procedural

Objetivo de leitura

Produzir dados reproduzíveis com sementes, campos e receitas.

Antes de começar

Conheça funções, coleções e sementes.

Uma operação procedural produz dados a partir de uma regra. Quando preservamos a receita e a semente, podemos repetir o resultado sob o mesmo contrato. Isso é útil para mapas, simulações e dados sintéticos.

Semente, campo e receita

`coral.procedural` é uma biblioteca geral de geração reproduzível. Ela não depende de Jogos, RPG nem interface gráfica. `Semente`, `GeradorProcedural`, `SequenciaProcedural` e `CampoProcedural` permitem derivar domínios independentes, consultar valores por chave ou coordenada sem depender da ordem de consultas anteriores, compor campos e distribuir pontos sob limites finitos. Na 1.6.15, pesos inválidos recebem diagnóstico explícito, campos declaram dimensionalidade e intervalos conhecidos, e a ordem de operações não comutativas permanece observável.

`ReceitaProcedural`, `receita_procedural`, `reconstruir_receita` e `diagnosticar_custo_receita` formam a camada declarativa portátil. As receitas suportadas registram algoritmo, versão, semente e parâmetros e podem ser preservadas

por `coral.persistencia`. Callbacks não entram na forma portátil. Reconstrução e materializações grandes possuem limites de custo explícitos, sem esconder trabalho ou alocação potencialmente desproporcional.

O módulo serve a simulações, mapas, ecologia, dados sintéticos e outros domínios. Jogos e Laboratório podem consumi lo, mas não são sua finalidade obrigatória.

Na 1.6.17, `amostrar_campo` e `matriz_campo` tornam a observação do campo uma operação explícita e limitada. `anotar_pontos` combina distribuições espaciais com grandezas como altitude, umidade ou risco. `proveniencia_procedural` fornece metadados portáteis sem afirmar que um callback arbitrário pode ser reconstruído.

A ponte com Jogos vive no consumidor: receitas de `distribuir_pontos` podem preencher cenas com sprites reprodutíveis e campos podem escrever valores em `sprite.dados`, sem abrir janela. A ponte com Laboratório trata receita e semente procedural como parte do desenho experimental, separadas da semente usada pelo próprio protocolo de medição. `planejar_experimento_procedural` calcula execuções e custo máximo antes de reconstruir a receita; `executar_experimento_procedural` preserva a receita base e as sementes nos metadados do resultado.

Procedural em jogos e pesquisa

A fundação procedural permite que o mesmo processo determinístico possa ser usado na construção de cenários de jogo e em experimentos reproduzíveis.

A regra central continua simples: uma semente e uma receita descrevem como produzir algo. Jogos decide como apresentar esse resultado. Laboratório decide como medir e comparar execuções. Procedural não precisa conhecer nenhum dos dois.

Usando pontos em uma cena

Uma receita de distribuição pode ser passada a `sprites_de_receita` ou `popular_cena_procedural`. O adaptador cria sprites nas mesmas posições sempre que a receita, a semente e a versão do contrato forem as mesmas. Nenhuma janela é aberta por essa conversão.

Campos procedurais podem ser aplicados aos sprites depois. O valor fica em `sprite.dados`, permitindo que um programa use, por exemplo, relevo para escolher aparência, fertilidade para crescimento ou perigo para regras de encontro.

Medindo uma receita

`planejar_experimento_procedural` combina sementes, parâmetros e repetições sem executar a receita. Além do número de execuções, ele mostra o custo máximo declarado. Isso é especialmente útil quando uma distribuição possui muitas tentativas possíveis.

`executar_experimento_procedural` reconstrói a receita para cada semente e entrega o produto à função estudada. Cada medição preserva a semente procedural, e o resultado

guarda a receita base nos metadados. Assim é possível comparar diferentes sementes sem perder de vista exatamente qual processo gerou cada observação.

Enviando campos a Gráficos

`amostrar_campo` cria registros gerais. Eles podem ser enviados a `series_de_registros`. `matriz_campo` cria uma matriz 2D que pode ser enviada diretamente a `mapa_calor`. A integração não exige que Gráficos conheça `CampoProcedural`.

Quando usar cada caminho

Use `amostrar_campo` quando você precisa preservar coordenadas individuais. Use `matriz_campo` para grades 2D finitas. Use `anotar_pontos` quando uma distribuição espacial precisa receber grandezas ambientais. Use as funções de Jogos quando o resultado vai virar objeto visual. Use as funções de Laboratório quando sementes ou parâmetros precisam ser comparados sob um protocolo experimental explícito.

Projeto curto: Campo com receita

Gere um campo procedural com receita e semente registradas. Repita a consulta em ordem diferente.

A orientação de solução está no Apêndice B, seção Projeto 37.

O que você aprendeu

Uma receita registra como gerar o resultado. Preserve algoritmo, versão, semente e parâmetros; limites de custo evitam materializações desproporcionais.

Prática

Explique como semente e receita ajudam a reproduzir um campo procedural.

Conferir a resposta comentada

A semente controla o gerador e a receita registra algoritmo, versão e parâmetros. A reconstrução só é garantida para operações e versões suportadas; callbacks arbitrários não viram receita portátil.

Resposta do capítulo 37 no Apêndice B.

38. Computação numérica e científica

Objetivo de leitura

Escolher operações numéricas e reconhecer seus requisitos e limites de interpretação.

Antes de começar

Conheça matemática e coleções; estatística básica ajuda nas seções avançadas.

`coral.numerico` reúne operações sobre séries, vetores e matrizes. Comece escolhendo o cálculo necessário e conferindo o formato dos dados. As fichas deste capítulo apresentam as assinaturas e exemplos completos; as seções de estatística avançada podem ser consultadas depois.

Uma série é uma sequência de valores. Uma matriz organiza valores em linhas e colunas. Dados finitos excluem valores como infinito e `NaN`, que representa um resultado numérico indefinido. Quando uma operação exige NumPy, o pacote precisa estar instalado; importar o módulo não carrega essa dependência antecipadamente.

Validar os dados antes de calcular

A API normaliza erros públicos relevantes como `ErroNumerico`, valida matrizes, estatísticas, intervalos e reduções vazias e evita vazar detalhes acidentais do backend NumPy para a superfície Coral. As estatísticas numéricas rejeitam dados ausentes, `NaN` e infinito com diagnóstico público em português. `selecionar_coluna` permite escolher explicitamente se registros ausentes geram erro ou são ignorados. Percentis e quartis usam interpolação linear declarada; correlação rejeita variância zero; moda preserva empates em vez de inventar uma única categoria vencedora.

A mesma política cobre álgebra linear e geometria 3D. `norma` e `normalizar` trabalham com vetores finitos; `posto` e `minimos_quadrados` validam formas; `produto_vetorial` é restrito a três dimensões. `rotacionar3` usa a fórmula de Rodrigues e ângulo em radianos. `transformar_ponto3` recebe uma matriz afim homogênea 4 × 4 e usa vetor coluna, mantendo a coordenada homogênea apenas como detalhe interno.

Nas operações estatísticas, `erro_padrao` usa desvio padrão amostral e exige pelo menos duas observações. `assimetria` e `curtose` usam momentos centrais padronizados e rejeitam variância zero; curtose é devolvida em excesso. `matriz_covariancia` interpreta linhas como observações e colunas como variáveis e `matriz_correlacao` rejeita colunas constantes. `regressao_linear` devolve inclinação, intercepto, resíduos, R^2 e quantidade de observações.

`intervalo_bootstrap` implementa intervalo percentil por reamostragem e `teste_permutacao` implementa teste bilateral por diferença da estatística escolhida pelo programa. Ambos usam um gerador local por semente, registram repetição e custo estimado e recusam trabalho acima do limite interno. Intervalos e valores p são resultados numéricos; a biblioteca não os transforma automaticamente em causalidade, superioridade ou significância substantiva.

A camada tridimensional oferece `matriz_rotacao3`, `matriz_translacao3`, `matriz_escala3`, `compor_transformacoes3` e `transformar_vetor3`. As relações ponto plano são cobertas por `distancia_ponto_plano3`, `projetar_ponto_plano3` e `intersecao_reta_plano3`. `coordenadas_esfericas3` e `ponto_de_esfericas3` fazem a conversão explícita entre representações, enquanto `area_triangulo3` e `volume_tetraedro3` tratam medidas geométricas sem recorrer ao backend gráfico.

Estatística avançada em `coral.numerico`

As operações desta seção fazem parte do runtime 1.7.6.

A implementação permanece em `coral.numerico`, com NumPy opcional na importação e exigido somente quando uma operação numérica que dele depende é chamada. SciPy não é dependência desta versão.

Medidas de forma e incerteza

`erro_padrao(valores)` calcula o erro padrão da média usando o desvio padrão amostral, com divisor `n - 1`, e exige ao menos duas observações finitas.

`assimetria(valores)` calcula o terceiro momento central padronizado sem correção de viés amostral. `curtose(valores)` calcula o quarto momento padronizado e devolve curtose em excesso, subtraindo 3. As duas operações rejeitam variância zero porque o denominador seria indefinido.

Matrizes de associação

`matriz_covariancia(dados, populacional=False)` interpreta cada linha como uma observação e cada coluna como uma variável. O padrão é amostral; `populacional=True` troca o divisor para `n`.

`matriz_correlacao(dados)` devolve a matriz de correlação de Pearson para as colunas observadas conjuntamente. Colunas constantes são rejeitadas explicitamente. As rotinas usam centralização escalada para reduzir transbordamentos intermediários evitáveis.

Regressão linear

`regressao_linear(x, y)` ajusta `y = intercepto + inclinacao * x` por mínimos quadrados simples. O retorno é `ResultadoRegressaoLinear` e inclui:

- `inclinacao`;
- `intercepto`;
- `residuos`;
- `r_quadrado`;
- `quantidade`.

As séries precisam ter o mesmo comprimento, conter ao menos duas observações e usar somente valores finitos. Uma série `x` constante não identifica uma inclinação e produz

`ErroNumerico`. Uma série `y` constante é válida e resulta em inclinação zero e R² igual a 1 para o ajuste constante exato.

Bootstrap

`intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente)` usa reamostragem com reposição e devolve `ResultadoBootstrap`.

O método disponível nesta edição é o intervalo percentil. A função estatística é fornecida pelo programa, deve ser chamável e precisa produzir um número real finito. A semente é inteira e alimenta uma instância local de `random.Random`, portanto o gerador aleatório global do programa não é modificado.

O resultado registra estimativa observada, limites inferior e superior, confiança, repetições, semente, tamanho da amostra, custo estimado e o identificador `bootstrap_percentil`.

Teste de permutação

`teste_permutacao(a, b, estatistica, repeticoes, semente)` estima bilateralmente a probabilidade de obter uma diferença de magnitude pelo menos igual à observada quando os rótulos dos dois grupos são permutáveis sob a hipótese nula.

O valor p usa a correção `(extremos + 1) / (repeticoes + 1)`, evitando zero artificial em uma simulação finita. O resultado registra diferença observada, valor p, tamanhos dos grupos, repetições, semente, custo estimado e o identificador `permutacao_bilateral`.

Limites e interpretação

Bootstrap e permutação possuem limite interno de 5 000 000 de observações processadas por chamada para impedir custos acidentais muito grandes. O limite é validado antes de iniciar a repetição.

Intervalos e valores p são resultados de métodos declarados, não conclusões automáticas. A Coral não afirma causalidade, não classifica um efeito como importante e não transforma um limiar arbitrário em verdade científica. A interpretação continua sob responsabilidade do programa e do contexto da análise.

Generalidade

As rotinas aceitam séries e matrizes comuns e não dependem de `coral.laboratorio`. Resultados de experimentos podem ser convertidos em séries pelo programa, mas as fórmulas não assumem uma origem específica para os dados.

Projeto curto: Resumo de medições

Calcule um resumo dos valores 20, 22, 24 e 26 e identifique o que acontece com dados ausentes.

A orientação de solução está no Apêndice B, seção Projeto 38.

O que você aprendeu

Confira formato, finitude e quantidade dos dados. Uma medida estatística descreve um cálculo; sua interpretação depende do método e do contexto.

Prática

Explique por que correlação com coluna constante é inválida.

Conferir a resposta comentada

O cálculo divide pela dispersão de cada variável. Se uma coluna não varia, essa dispersão é zero e a correlação não pode ser definida pelo contrato; a biblioteca produz diagnóstico.

Resposta do capítulo 38 no Apêndice B.

39. Gráficos

Objetivo de leitura

Criar uma especificação de gráfico e escolher um formato de apresentação.

Antes de começar

Conheça coleções numéricas e arquivos.

`coral.graficos` recebe coleções comuns e produz especificações imutáveis. A criação da especificação é separada da renderização e não abre janela. Linhas e dispersão exigem séries alinhadas e finitas; barras mantêm zero na escala vertical por padrão. Registros comuns podem ser adaptados por campos escolhidos com `series_de_registros`.

Histogramas e caixas não mantêm uma segunda implementação estatística: `grafico_histograma` usa o contrato de `coral.numerico.histograma`, e `grafico_caixa` usa os quartis de Numérico. Isso mantém contagens, bordas e quartis consistentes entre cálculo e apresentação.

`grafico_funcao` limita a quantidade de amostras e representa avaliações inválidas como lacunas para não ligar uma falha por uma linha enganosa. `configurar` retorna cópia com título, rótulos, unidades, legenda, tema, proveniência e texto alternativo. `para_svg` e `salvar` produzem SVG 2D autossuficiente usando somente a biblioteca padrão.

Código Coral

```
de coral.graficos importe linha, configurar, salvar

defina figura como linha([0, 1, 2], [20, 21, 23])
defina figura como configurar(figura, titulo="Temperatura", eixos=["Tempo",
  "Temperatura"], unidades=["min", "°C"])
execute salvar(figura, "temperatura.svg")
```

Este exemplo não imprime texto no terminal.

Na 1.6.11, `mapa_calor` representa matrizes retangulares e `mapa_correlacao` reutiliza o cálculo de `coral.numerico.matriz_correlacao`. `barras_erro` e `faixa_incerteza`

representam incerteza sem chamar toda faixa de intervalo de confiança. `grafico_experimento` importa Laboratório apenas durante a chamada explícita e converte medições em séries comuns preservando proveniência. `painel` combina múltiplos gráficos sem alterar as especificações filhas. SVG 2D continua disponível somente com a biblioteca padrão; PNG e PDF usam Matplotlib opcional carregado sob demanda.

Na 1.6.12, `dispersao3d` e `trajetoria3d` criam séries tridimensionais sem alterar a ordem ou as coordenadas de entrada. `superficie3d` representa uma grade retangular explícita e não interpola dados ausentes. A especificação 3D continua independente do backend. `para_svg` recusa reduzir 3D a uma projeção implícita; `salvar` usa Matplotlib sob demanda para SVG, PNG e PDF tridimensionais. A superfície espacial dedicada da linha 1.6.x termina em 3D; não há uma API pública planejada para quatro dimensões espaciais.

Gráficos interativos locais

Na 1.6.13, `para_html` gera HTML interativo autossuficiente e `salvar_html` grava o resultado localmente. Plotly é carregado somente nessas operações ou em `explorar`; importar `coral.graficos` continua sem carregar o backend. O arquivo final não usa CDN e contém os dados e o JavaScript necessários à interação.

Código Coral

```
de coral.graficos importe dispersao, configurar, salvar_html

defina figura como dispersao([1, 2, 3], [2, 4, 6])
defina figura como configurar(figura, titulo="Relação entre medidas")
execute salvar_html(figura, "medidas_interativas.html")
```

Este exemplo não imprime texto no terminal.

`explorar` abre a visualização somente quando solicitado. A ausência de sessão gráfica produz diagnóstico que orienta `salvar_html`. `amostrar_grafico` é a única forma pública de redução automática planejada nesta edição e exige decisão explícita; a exportação nunca elimina pontos silenciosamente.

Painéis mistos podem conter gráficos 2D e 3D. Zoom, inspeção e rotação da câmera são recursos de apresentação e não alteram coordenadas, malhas ou resultados estatísticos. Texto alternativo, avisos e proveniência permanecem disponíveis na saída, e o HTML informa que os dados ficam incorporados no arquivo.

Gráficos 2D

`coral.graficos` é a primeira camada pública de visualização científica da Coral. O módulo separa três responsabilidades: Numérico calcula, Laboratório produz medições e Gráficos apresenta dados. Criar uma especificação não abre janela e não acessa rede.

Séries diretas

`linha(x, y)` e `dispersao(x, y)` recebem duas coleções numéricas de mesmo comprimento. Valores ausentes, `NaN` e infinito são recusados explicitamente.

`barras(categorias, valores)` apresenta categorias e mantém zero na escala vertical por padrão, evitando truncamento enganoso.

Estatística visual

`grafico_histograma(valores, faixas)` delega o cálculo a `coral.numerico.histograma`, portanto contagens e bordas são exatamente as mesmas do módulo Numérico.

`grafico_caixa(grupos)` usa os quartis de `coral.numerico` e bigodes de Tukey de 1,5 × IQR. Pontos extremos ficam preservados e grupos com menos de cinco observações recebem aviso.

Registros comuns

`series_de_registros(registros, campo_x, campo_y)` aceita mapeamentos e objetos com atributos. O adaptador retorna duas séries alinhadas sem importar `coral.laboratorio`. Campo inexistente ou valor ausente é erro, nunca descarte silencioso.

Funções e descontinuidades

`grafico_funcao(funcao, inicio, fim, amostras)` amostra uma função escalar em intervalo fechado. Avaliações que falham ou produzem valores não finitos viram lacunas e não são ligadas por uma linha. O limite é 100000 amostras por chamada.

Configuração

`configurar` devolve uma nova especificação e preserva os dados originais. É possível definir título, rótulos de eixos, unidades, legenda, tema, proveniência e texto alternativo. Os temas iniciais são `claro`, `escuro` e `alto_contraste`.

SVG

`para_svg` produz SVG autossuficiente em memória. `salvar` grava SVG 2D pela biblioteca padrão; PNG e PDF usam Matplotlib opcional. A renderização usa a biblioteca padrão e não carrega Matplotlib, Plotly, Pygame ou Tkinter.

Compor gráficos e representar incerteza

A superfície gráfica continua geral. Ela recebe listas, matrizes e séries comuns sem exigir `coral.laboratorio`. O adaptador de experimentos é acionado apenas quando `grafico_experimento` é chamado.

`mapa_calor(matriz)` representa valores já calculados em uma matriz retangular. `mapa_correlacao(dados)` delega o cálculo de Pearson a

`coral.numerico.matriz_correlacao` e apenas apresenta o resultado. Assim, a definição estatística continua pertencendo ao módulo numérico.

`barras_erro(x, y, erros)` aceita incertezas simétricas ou pares inferior e superior. `faixa_incerteza(x, centro, inferior, superior)` recebe limites absolutos. A faixa não é chamada automaticamente de intervalo de confiança porque seu significado vem do método que produziu os limites.

`grafico_experimento(resultado, eixo_x, metrica)` converte medições bem sucedidas de um `ResultadoExperimento` para a mesma `Serie2D` usada pelos gráficos gerais. Nome do experimento, sementes e protocolo de medição ficam registrados na proveniência da especificação.

`painel(graficos, colunas)` compõe várias especificações sem alterar seus dados ou escalas. `para_svg` continua funcionando apenas com a biblioteca padrão e também aceita painéis.

`salvar(grafico, caminho)` reconhece `.svg`, `.png` e `.pdf`. SVG continua sendo o backend básico e autossuficiente. PNG e PDF carregam Matplotlib somente quando solicitados. Se o backend opcional não estiver instalado, a Coral devolve erro orientando a instalação em vez de falhar durante `import coral.graficos`.

A importação de `coral.graficos` não carrega Matplotlib, Plotly, NumPy nem Laboratório antecipadamente. HTML interativo usa Plotly opcional, descrito adiante.

Gráficos 3D

`coral.graficos` passa a aceitar especificações tridimensionais sem transformar visualização em cálculo geométrico. `coral.numerico` continua responsável por vetores, transformações, planos, coordenadas esféricas, áreas e volumes. Gráficos recebe coordenadas já definidas e as apresenta.

Pontos e trajetórias

`dispersao3d(x, y, z, nome=None)` cria uma coleção de pontos tridimensionais. `trajetoria3d(x, y, z, nome=None)` preserva a mesma sequência de coordenadas e apenas indica que os pontos devem ser ligados na ordem fornecida.

Os três eixos precisam ter o mesmo comprimento. Valores ausentes, `NaN` e infinito são rejeitados. As coleções de entrada não são modificadas nem reordenadas.

Superfícies

`superficie3d(x, y, z, nome=None)` representa uma grade retangular explícita. Há duas formas aceitas.

Quando `x` e `y` são eixos unidimensionais, `z` deve possuir `len(y)` linhas e `len(x)` colunas. Quando `x` e `y` são grades bidimensionais, as três grades precisam ter exatamente a mesma forma.

A função não interpola pontos ausentes, não corrige grades irregulares e não inventa valores entre amostras. Uma superfície descreve somente os pontos presentes na grade fornecida.

Configuração dos eixos

`configurar` aceita dois ou três rótulos e unidades. Em gráficos 3D, a forma `eixos=("x", "y", "z")` e `unidades=("m", "m", "m")` também configura o terceiro eixo. Mapeamentos podem usar as chaves `x`, `y` e `z`.

Renderização estática

Criar `dispersao3d`, `trajetoria3d` ou `superficie3d` não importa Matplotlib, não abre janela e não escolhe uma projeção escondida. `para_svg` continua sendo o renderizador próprio da Coral para especificações 2D e, por isso, rejeita gráficos 3D em vez de achatá los silenciosamente.

`salvar` usa Matplotlib somente quando uma renderização 3D é solicitada. Gráficos 3D podem ser salvos em SVG, PNG ou PDF quando o backend opcional está instalado. A importação de `coral.graficos` permanece independente de Matplotlib.

Fronteira de responsabilidade

A câmera e a projeção visual usadas pelo backend servem apenas à apresentação. As coordenadas originais guardadas em `Serie3D` e `Superficie3D` permanecem inalteradas. Operações geométricas como rotação, transformação afim, projeção sobre plano e interseção reta plano pertencem a `coral.numerico`.

A linha 1.6.x não possui uma superfície pública dedicada a espaço 4D. A decisão da 1.6.12 concentra a evolução espacial da Coral em 3D.

Consultar os gráficos interativos

`coral.graficos` pode transformar as especificações científicas já existentes em HTML interativo autossuficiente. Plotly continua opcional e só é carregado quando a capacidade interativa é solicitada.

HTML local e offline

`para_html(grafico)` devolve um documento HTML completo em memória. `salvar_html(grafico, caminho)` grava o mesmo documento de forma atômica. O JavaScript necessário é incorporado no próprio arquivo, portanto a visualização não depende de CDN nem de conexão com a internet.

O HTML preserva os valores já presentes na especificação. O backend não recalcula estatística, não interpola superfícies, não suaviza trajetórias e não reduz coleções silenciosamente.

Exploração explícita

`explorar(grafico)` cria uma prévia local e pede ao navegador do sistema que a abra. A operação só acontece quando chamada explicitamente. Criar um gráfico ou importar `coral.graficos` não abre janela, navegador, servidor ou conexão.

Em sessão sem interface gráfica, `explorar` produz um diagnóstico orientando o uso de `salvar_html`. Nenhuma porta é aberta como alternativa implícita.

Painéis 2D e 3D

Painéis podem combinar gráficos 2D e 3D na mesma saída interativa. Cada subgráfico mantém título, eixos, unidades e tipo de cena. Séries de `grafico_experimento` continuam séries comuns depois da adaptação e preservam avisos e proveniência no texto acessível do documento.

Para 3D, dispersões, trajetórias e superfícies preservam as coordenadas originais. Rotação, zoom e posição da câmera pertencem apenas à apresentação.

Limites e amostragem

A exportação HTML possui um limite de observações ou células para evitar materialização acidental de documentos desproporcionais. Ao exceder o limite, a operação falha antes da renderização e explica as alternativas disponíveis.

`amostrar_grafico(grafico, max_pontos)` oferece uma redução uniforme explícita apenas para linhas, dispersões e séries 3D simples. A especificação devolvida registra a quantidade original, a quantidade mantida e um aviso de amostragem. A função nunca é chamada automaticamente por `para_html`.

Mapas, superfícies, caixas, barras e faixas não são reduzidos automaticamente porque uma seleção genérica poderia alterar sua semântica. Nesses casos, o programa deve preparar uma representação menor antes de construir o gráfico.

Privacidade e acessibilidade

O HTML autossuficiente contém os dados do gráfico incorporados. A página inclui um aviso visível lembrando que dados sensíveis devem ser revisados antes de compartilhar o arquivo.

A visualização recebe papel de imagem e descrição acessível. `texto_alternativo`, avisos e proveniência participam dessa descrição. Em painéis, as descrições dos filhos também são preservadas.

Integração com servidor local

`coral.web.servidor.resposta_grafico(grafico)` converte explicitamente um gráfico em `RespostaServidorHTTP`. A função não cria nem inicia servidor. Ela apenas prepara a resposta HTML e usa `Cache-Control: no-store` por padrão porque o documento contém os próprios dados.

Para publicar localmente, o programa cria um servidor, registra uma rota que devolve `resposta_grafico` e então chama `iniciar()` de forma explícita. Assim Gráficos não passa a depender de Web e Web não inicia rede durante a adaptação.

Dependência opcional

Plotly 6.0 ou superior pertence ao componente opcional `graficos_interativos`. A ausência do pacote não impede `import coral.graficos`, SVG, PNG, PDF ou as especificações 2D e 3D. O erro aparece apenas ao solicitar HTML interativo.

Projeto curto: Gráfico de temperatura

Crie uma linha com quatro medições e salve SVG com título e unidades. Preserve a lista de origem.

A orientação de solução está no Apêndice B, seção Projeto 39.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`linha(...)` · **módulo** `coral.graficos`

O que faz

Cria uma especificação imutável de gráfico de linhas.

Assinatura

Assinatura para consulta

```
linha(x: 'Iterable[Any]', y: 'Iterable[Any]', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	Iterable[Any]	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
nome	str | None	não	None	Nome da série ou do objeto.

Retorno

Grafico 2D.

Exemplo

Código Coral

```
de coral.graficos importe linha
defina figura como linha([0, 1], [2, 3])
mostre figura.elementos[0].estilo
```

Resultado

Saída esperada

```
linha
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`dispersao(...)` · módulo `coral.graficos`

O que faz

Cria uma especificação imutável de gráfico de pontos.

Assinatura

Assinatura para consulta

```
dispersao(x: 'Iterable[Any]', y: 'Iterable[Any]', nome: 'str | None' = None) ->
  'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	Iterable[Any]	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
nome	str | None	não	None	Nome da série ou do objeto.

Retorno

Grafico 2D.

Exemplo

Código Coral

```
de coral.graficos importe dispersao
defina figura como dispersao([0, 1], [2, 3])
mostre figura.elementos[0].estilo
```

Resultado

Saída esperada

```
dispersao
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`configurar(...)` · módulo `coral.graficos`

O que faz

Cria uma cópia do gráfico com as opções solicitadas.

Assinatura

Assinatura para consulta

```
configurar(grafico: 'Grafico', titulo: 'str | None' = None, eixos: 'Any' = None,
  unidades: 'Any' = None, legenda: 'bool | None' = None, tema: 'str | None' = None, *,
  proveniencia: 'Iterable[Any] | None' = None, texto_alternativo: 'str | None' = None) ->
  'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
titulo	str | None	não	None	Valor titulo usado na operação.
eixos	Any	não	None	Rótulos dos eixos; a quantidade precisa corresponder à dimensionalidade.
unidades	Any	não	None	Unidades dos eixos; acompanham a dimensionalidade do gráfico.
legenda	bool | None	não	None	Se verdadeiro, inclui a legenda.
tema	str | None	não	None	Identificador do tema visual suportado.
proveniencia	Iterable[Any] | None	não	None	Metadados que explicam de onde vêm os dados.
texto_alternativo	str | None	não	None	Descrição textual para acessibilidade.

Retorno

Nova especificação Grafico.

Exemplo

Código Coral

```
de coral.graficos importe configurar
de coral.graficos importe linha
defina figura como configurar(linha([0, 1], [2, 3]), titulo="Medidas")
mostre figura.titulo
```

Resultado

Saída esperada

```
Medidas
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`salvar(...)` · módulo `coral.graficos`

O que faz

Renderiza um gráfico num arquivo escolhido pelo formato de saída.

Assinatura

Assinatura para consulta

```
salvar(grafico: 'Grafico | PainelGraficos', caminho: 'str | Path') -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico | PainelGraficos	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.

Retorno

Path do arquivo salvo.

Exemplo

Código Coral

```
de coral.graficos importe salvar
de coral.graficos importe linha
execute salvar(linha([0, 1], [2, 3]), "figura.svg")
de coral.arquivos importe existe
mostre existe("figura.svg")
```

Resultado

Saída esperada

```
True
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`salvar_html(...)` · **módulo** `coral.graficos`

O que faz

Exporta um gráfico interativo autossuficiente, sem CDN.

Assinatura

Assinatura para consulta

```
salvar_html(grafico: 'Any', caminho: 'str | Path', *, limite_pontos: 'int' = 200000) ->
  'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.
limite_pontos	int	não	200000	Teto explícito de observações aceitas na exportação HTML.

Retorno

Path do HTML salvo.

Exemplo

Código Coral

```
de coral.graficos importe salvar_html
de coral.graficos importe linha
execute salvar_html(linha([0, 1], [2, 3]), "figura.html")
de coral.arquivos importe existe
mostre existe("figura.html")
```

Resultado

Saída esperada

```
True
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

O que você aprendeu

Criar um gráfico não abre uma janela. SVG 2D usa a biblioteca padrão; outras saídas podem exigir backends opcionais. Unidades e proveniência ajudam a interpretar os dados.

Prática

Separe criar uma especificação de gráfico e salvar sua imagem.

Conferir a resposta comentada

Criar `linha` ou `dispersao` descreve os dados sem abrir janela. `salvar` escolhe o formato e renderiza. A disponibilidade do backend depende do formato solicitado.

Resposta do capítulo 39 no Apêndice B.

40. Web

Objetivo de leitura

Distinguir HTML local, cliente HTTP e servidor HTTP.

Antes de começar

Conheça textos, funções e tratamento de erros.

A base Web separa geração local de HTML e URLs, cliente HTTP e servidor HTTP. Escolha apenas a capacidade que sua aplicação precisa.

Montar HTML e manipular URLs

`coral.web.html` e `coral.web.url` formam a base Web local da Coral. Nesta versão as duas superfícies trabalham somente em memória: importar os módulos não abre conexão, porta, navegador ou janela. As requisições HTTP usam o módulo próprio, apresentado mais adiante.

HTML seguro por padrão

`escapar_texto(valor)` codifica texto para o conteúdo de elementos. `escapar_atributo(valor)` codifica texto para atributos entre aspas. Escape não transforma uma URL perigosa em URL segura, por isso atributos de URL são validados separadamente.

`elemento(nome, atributos, filhos)` cria um `ElementoHTML` imutável. Nomes de elemento e atributo são validados, atributos de evento inline como `onclick` são recusados e esquemas executáveis como `javascript:` não são aceitos em `href`, `src`, `action` e atributos equivalentes. Texto comum em `filhos` nunca é interpretado como marcação confiável.

`fragmento(filhos)` agrupa nós sem inserir um contêiner extra. `documento(titulo, corpo, idioma)` cria um `DocumentoHTML` completo com UTF 8 e idioma declarado. `renderizar(no)` produz a serialização em memória e não escreve arquivo.

Tabelas de registros gerais

`tabela(registros, colunas)` aceita mapeamentos ou objetos com atributos. As colunas são explícitas e mantêm a ordem fornecida. Um campo ausente produz erro em vez de desaparecer silenciosamente, e o valor de cada célula continua sujeito ao escape de texto. Isso permite apresentar dados comuns e, por conversão explícita, dados produzidos por outros módulos sem importar `coral.laboratorio`.

Consultas e URLs

`codificar_consulta(parametros)` aceita mapeamento ou sequência ordenada de pares. Chaves repetidas são preservadas e o texto usa UTF 8. `ler_consulta(texto)` devolve os pares na mesma ordem, preservando valores vazios e duplicatas.

`decompor(endereco)` exige esquema explícito e devolve `EnderecoURL`, com esquema, autoridade, host, porta, caminho, consulta e fragmento. A propriedade `parametros` decodifica a consulta sem perder chaves repetidas. `juntar(base, referencia)` resolve uma referência relativa contra uma base usando apenas regras textuais. Nenhuma dessas operações resolve DNS ou acessa a rede.

Exemplo

O exemplo `Exemplos/Citados/Web/01_html_e_url.coral` monta um documento com tabela, escapa conteúdo potencialmente ambíguo e prepara uma URL com consulta repetida. Ele pode ser compilado e executado sem conexão de rede.

Escolher o módulo adequado

HTML e URL trabalham em memória. Para acessar um endereço, use `coral.web.http`; para atender requisições locais, use `coral.web.servidor`. Montar uma página não a hospeda automaticamente.

Fazer uma requisição HTTP

`coral.web.http` é o primeiro cliente HTTP geral da Coral. Importar o módulo não abre conexão, não inicia servidor e não consulta recursos externos. A rede só é usada quando `requisitar`, `obter` ou `enviar` é chamada. O módulo não depende de `coral.laboratorio`.

Resposta preservada em bytes

`RespostaHTTP` preserva `status`, `cabecalhos`, `corpo_bytes`, `url`, `metodo` e a quantidade de `redirecionamentos`. A propriedade `sucesso` apenas indica status entre 200 e 299. Respostas 3xx, 4xx e 5xx continuam inspecionáveis e não são transformadas automaticamente em erro de transporte.

`texto(codificacao)` decodifica o corpo sob demanda. O corpo recebido nunca é convertido silenciosamente em texto. `json()` interpreta JSON sob demanda e usa `ErroFormato` para conteúdo inválido, separando erro de formato de falha de transporte.

Requisições

`requisitar(metodo, url, cabecalhos, corpo, tempo_limite)` é a operação geral. `obter(url, ...)` é o atalho para GET e `enviar(url, corpo, ...)` é o atalho para POST. Texto enviado exige `codificacao_corpo` explícita; bytes são enviados sem conversão.

O cliente aceita somente URLs `http` e `https`, rejeita credenciais embutidas e valida nomes e valores de cabeçalhos antes de montar a requisição. A execução usa a biblioteca padrão e não requer pacote externo.

Timeout e limite de resposta

Toda requisição tem `tempo_limite` finito. `limite_resposta` limita o corpo materializado em memória; se `Content-Length` já declara tamanho acima do teto ou se a leitura excede o limite, a operação falha com `ErroLimiteHTTP` em vez de truncar silenciosamente.

Esses limites controlam o cliente, não substituem políticas da aplicação. O programa ainda decide quais hosts pode acessar, que dados envia e que respostas considera confiáveis.

Redirecionamentos

GET, HEAD e OPTIONS seguem redirecionamentos por padrão, com `max_redirecionamentos` finito. Métodos com potencial de efeito, como POST, não repetem a requisição após redirecionamento sem `seguir_redirecionamentos=true` explícito. Apenas destinos `http` e `https` podem ser seguidos.

Formulários

`codificar_formulario(campos)` produz `application/x-www-form-urlencoded` em bytes ASCII com valores codificados em UTF 8. Sequências de pares preservam ordem e chaves repetidas. Multipart, upload de arquivos, cookies persistentes e autenticação automática não fazem parte desta base.

TLS e proxy

HTTPS usa `ssl.create_default_context()` quando nenhum `contexto_tls` é fornecido. Isso mantém verificação de certificado e de hostname habilitadas por padrão. Um `SSLContext` explícito pode ser passado por aplicações que realmente precisam controlar a política TLS.

`proxy` é explícito. Quando não é fornecido, o cliente não depende de uma camada Coral de proxy. Quando configurado, aceita uma URL textual para HTTP e HTTPS ou um mapeamento por esquema.

Exemplo local

`Exemplos/Citados/Web/02_cliente_http_loopback.coral` recebe a URL de um servidor local efêmero como argumento do programa, faz GET de JSON e POST de formulário e inspeciona status e corpo. O gate da release executa esse fluxo somente em loopback, sem serviço externo.

Limites do cliente

O cliente não oferece upload multipart, cookies persistentes, cache ou autenticação automática. O servidor é um módulo separado. HTML e URL continuam utilizáveis sem conexão.

Servidor HTTP local da Coral

`coral.web.servidor` oferece um servidor pequeno para desenvolvimento, demonstrações e testes locais. Importar o módulo não abre porta. `criar_servidor()` apenas cria a configuração; a escuta começa com `iniciar()` ou ao usar o objeto em um bloco de contexto.

Por padrão, o servidor usa `127.0.0.1` e porta `0`. A porta zero pede ao sistema uma porta efêmera livre. Exposição fora do loopback exige `permitir_externo=True`, portanto não acontece por acidente.

Rotas literais

`adicionar_rota` é o método `ServidorHTTP.adicionar_rota(metodo, caminho, tratador)` e registra uma combinação literal como `GET /estado`. A 1.6.9 não interpreta parâmetros de rota, curingas ou expressões. O tratador recebe `RequisicaoHTTP` e deve retornar `RespostaServidorHTTP`.

A requisição preserva `metodo`, `caminho`, `consulta`, `cabecalhos`, `corpo_bytes` e `cliente`. `texto()`, `json()` e `formulario()` interpretam o corpo somente quando o programa pede. Consultas e formulários preservam ordem e chaves repetidas.

Respostas

Use `resposta_bytes`, `resposta_texto`, `resposta_html` ou `resposta_json`. HTML reutiliza a serialização segura de `coral.web.html`; texto comum não vira marcação por acidente. JSON recusa valores que exigiriam conversão silenciosa e não aceita NaN como JSON válido.

Limite de corpo

`limite_corpo` é aplicado a `Content-Length` antes de o tratador ser chamado. Corpos acima do limite recebem status 413. `tempo_limite_corpo` impede que uma leitura parcial mantenha a conexão indefinidamente. O servidor básico não implementa corpo de requisição com `Transfer-Encoding`.

Arquivos sob raiz declarada

Ao criar o servidor, `raiz_arquivos` pode apontar para uma pasta explícita. `adicionar_arquivo` associa um arquivo relativo a uma rota literal. Caminhos absolutos, `..` que saiam da raiz e links simbólicos que saiam da raiz declarada são recusados. `limite_arquivo` impede leitura acidental de arquivos acima do teto escolhido.

Encerramento

`encerrar()` faz `shutdown`, fecha o socket e aguarda a thread de atendimento. A chamada é idempotente. Usar `com servidor:` também encerra o serviço ao sair do bloco.

O servidor é uma ferramenta de desenvolvimento local. Ele não é um servidor de produção, não oferece TLS de servidor, autenticação, balanceamento, HTTP/2 ou gestão de processos. O cliente HTTP da Coral permanece em `coral.web.http` e continua independente do servidor.

Resposta de gráfico interativo

Na 1.6.13, `resposta_grafico(grafico)` adapta uma especificação de `coral.graficos` para uma `RespostaServidorHTTP` com HTML interativo autossuficiente. A adaptação é explícita e não inicia servidor, não abre porta e não acessa recursos remotos.

A resposta usa `Cache-Control: no-store` por padrão porque o HTML contém os dados da figura. O programa continua responsável por criar o servidor, registrar a rota e chamar `iniciar()` quando realmente quiser publicar a página no loopback.

Exemplo guiado sem serviço externo

Este programa inicia um servidor apenas no loopback, consulta sua própria rota e encerra o servidor em finalmente. A resposta esperada é 200 e Coral local em linhas separadas.

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Saída esperada

```
200
Coral local
```

Projeto curto: Página de estado local

Adapte o exemplo de loopback para devolver um texto de estado diferente e confirme o encerramento.

A orientação de solução está no Apêndice B, seção Projeto 40.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`criar_servidor(...)` · **módulo** `coral.web.servidor`

O que faz

Cria a configuração de um servidor HTTP pequeno para desenvolvimento local. Não começa a escuta até iniciar.

Assinatura

Assinatura para consulta

```
criar_servidor(host: 'str' = '127.0.0.1', porta: 'int' = 0, **opcoes: 'Any') ->
  'ServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
host	str	não	'127.0.0.1'	Endereço de escuta; o padrão limita o servidor ao loopback.
porta	int	não	0	Porta de escuta; zero pede uma porta efêmera livre.
opcoes	Any	não	sem padrão	Opções nomeadas aceitas pelo contrato chamado, conforme a referência de ServidorHTTP ou requisitar nesta parte.

Retorno

ServidorHTTP.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

`resposta_texto(...)` · **módulo** `coral.web.servidor`

O que faz

Cria uma resposta HTTP cujo corpo é texto, com tipo de conteúdo e codificação definidos.

Assinatura

Assinatura para consulta

```
resposta_texto(texto: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  codificacao: 'str' = 'utf-8') -> 'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	Any	sim	sem padrão	Texto a enviar no corpo da resposta.
status	int	não	200	Código HTTP da resposta.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.
codificacao	str	não	'utf-8'	Codificação de texto usada na resposta.

Retorno

RespostaServidorHTTP.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

`obter(...)` · módulo `coral.web.http`

O que faz

Faz um pedido HTTP GET com limites explícitos.

Assinatura

Assinatura para consulta

```
obter(url: 'str', cabecalhos: 'Any' = None, tempo_limite: 'float' = 10.0, **opcoes:
  'Any') -> 'RespostaHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
url	str	sim	sem padrão	URL do recurso HTTP.
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.
tempo_limite	float	não	10.0	Tempo limite finito da operação, em segundos.
opcoes	Any	não	sem padrão	Opções nomeadas aceitas pelo contrato chamado, conforme a referência de ServidorHTTP ou requisitar nesta parte.

Retorno

RespostaHTTP, com status, cabeçalhos e corpo em bytes; a interpretação do conteúdo é solicitada por métodos da resposta.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

O que você aprendeu

Montar HTML não acessa a rede. O cliente faz requisições; o servidor atende rotas. Conexões e início da escuta são ações explícitas.

Prática

Diferencie HTML local, cliente HTTP e servidor HTTP.

Conferir a resposta comentada

HTML local é conteúdo de apresentação; o cliente faz pedidos; o servidor responde a pedidos. Gerar HTML não abre conexão nem inicia um servidor.

Resposta do capítulo 40 no Apêndice B.

41. Sistema e hardware

Objetivo de leitura

Consultar capacidades do sistema e reconhecer limites de testes simulados.

Antes de começar

Conheça funções e tratamento de erros.

Um script pode consultar o ambiente em que está sendo executado e, quando necessário, iniciar outro programa. São ações diferentes: descobrir se um executável existe não significa executá lo.

Comece pelas operações de consulta nas fichas. Depois estude captura de saída, prazo de execução e hardware. Um teste simulado verifica a lógica, mas o comportamento de uma placa física precisa ser observado no equipamento.

Inspeção e execução

`coral.sistema` separa inspeção de capacidade de execução. `localizar_programa` e `programa_disponivel` consultam executáveis sem iniciá los; `quantidade_processadores`, `pasta_temporaria` e `terminal_interativo` descrevem capacidades do hospedeiro sem prometer recursos efetivos sob quotas. `ambiente_selecionado` copia apenas nomes explícitos e `variavel_ambiente_obrigatoria` falha sem incluir valores em diagnósticos. `argumentos_programa` expõe somente os argumentos do programa Coral, enquanto `caminho_executavel` identifica o executável que hospeda o runtime.

`executar_comando` continua sem shell implícito e agora aceita `codificacao`, política explícita de `erros_decodificacao` e `limite_saida_bytes`. O limite é aplicado enquanto stdout e stderr são drenados, e `saida_truncada` e `erro_truncado` registram descarte sem transformar truncamento em falha. Timeout tem prioridade sobre código de saída. `ResultadoComando` preserva seus campos históricos e acrescenta leituras por linhas, `codigo_saida`, `tempo_decorrido` e `exigir_sucesso`, que produz `ComandoFalhou` preservando o resultado completo.

Hardware e serial

Inventário de hardware degrada de forma previsível quando dependências opcionais não estão instaladas. A serial possui um backend simulado determinístico usado por contratos e testes. Comunicação com Arduino ou ESP32 físicos continua uma evidência de campo separada e não é declarada aprovada por simulação.

Coral Sistema e processos

`coral.sistema` reúne informações portáteis do processo Coral e operações explícitas sobre programas do sistema operacional. A biblioteca não substitui `coral.hardware`: ela descreve capacidades úteis para scripts, ferramentas, automação e aplicações, sem tentar inventariar profundamente CPU, memória ou dispositivos.

Consultar capacidades

`localizar_programa(nome)` resolve um executável usando o ambiente corrente e devolve seu caminho ou `nulo`. `programa_disponivel(nome)` usa a mesma resolução e apenas informa se o programa pode ser encontrado. Nenhuma das duas funções inicia o executável.

`quantidade_processadores()` devolve a quantidade lógica observada pelo runtime ou `nulo` quando o hospedeiro não informa esse valor. O número não deve ser interpretado como garantia de capacidade efetiva quando o processo estiver sob quotas, contêineres ou outras restrições.

`pasta_temporaria()` informa a pasta temporária configurada sem criar arquivos. `terminal_interativo(canal)` aceita `entrada`, `saida` ou `erro` e informa se o fluxo está associado a um terminal.

Ambiente

`ambiente_selecionado(nomes)` copia somente as variáveis explicitamente pedidas e omite as ausentes. O resultado é uma nova estrutura, portanto alterá lo não modifica o ambiente do processo.

`variavel_ambiente_obrigatoria(nome)` devolve a variável solicitada ou produz `ErroSistema` quando ela não existir. A mensagem de erro informa o nome, mas não inclui valor secreto.

Programa Coral e executável

`argumentos_programa()` devolve uma cópia dos argumentos fornecidos ao programa Coral. Ao usar o runtime pela linha de comando, separe esses argumentos com `--`:

Referência para leitura

```
python coral.pyz meu_programa.coral -- entrada.txt modo-seguro
```

Nesse caso o programa recebe `entrada.txt` e `modo-seguro`; opções internas do interpretador Coral não aparecem na lista.

`caminho_executavel()` informa o executável que hospeda o runtime, normalmente o Python usado para iniciar o PYZ. `processo_atual()` informa separadamente o executável, o programa ou PYZ corrente, os argumentos e a pasta atual.

ResultadoComando

Além dos campos históricos `codigo`, `saida`, `erro` e `duracao_segundos`, um resultado oferece:

- `linhas_saida()` e `linhas_erro()` para leitura por linhas sem alterar o texto original;
- `codigo_saida()` como leitura natural de `codigo`;
- `tempo_decorrido()` como leitura natural de `duracao_segundos`;
- `exigir_sucesso()` que devolve o próprio resultado quando o código é zero e produz `ComandoFalhou` caso contrário, preservando o resultado e o stderr.

Codificação e captura limitada

`executar_comando` continua executando sem shell. `codificacao` escolhe a codificação textual e `erros_decodificacao` aceita `estrito`, `substituir` ou `ignorar`. O padrão é estrito, preservando o comportamento histórico de falhar quando bytes inválidos não podem ser decodificados.

`limite_saida_bytes` limita separadamente a quantidade mantida em memória para stdout e stderr. O runtime aplica o teto enquanto lê os fluxos e continua drenando o excedente, evitando que um processo de saída contínua bloqueie apenas porque o conteúdo deixou de ser armazenado. `saida_truncada` e `erro_truncado` registram se houve descarte.

Timeout tem prioridade: quando o prazo termina, a Coral encerra o grupo de processos quando a plataforma oferece esse mecanismo e produz `TempoLimiteExcedido`. Truncamento não transforma código de saída zero em falha. Código não zero continua disponível em `ResultadoComando` e só vira exceção quando `exigir_sucesso()` é chamado.

Limites

A biblioteca não executa comandos por meio de shell implicitamente, não interpreta metacaracteres, não estima memória livre e não promete que a quantidade de processadores represente recursos realmente disponíveis sob quotas. Arquivos continuam sob responsabilidade de `coral.arquivos`.

Projeto curto: Relatório de capacidades

Consulte a quantidade de processadores e a disponibilidade de um programa sem iniciá lo.

A orientação de solução está no Apêndice B, seção Projeto 41.

O que você aprendeu

Consultar um executável não o inicia. O inventário observado não garante recursos efetivos, e uma simulação serial não comprova comunicação física.

Prática

Explique por que inventário simulado não comprova comunicação com uma placa física.

Conferir a resposta comentada

A simulação verifica o contrato do programa sem exercer cabo, porta, driver ou dispositivo. A comunicação física exige uma sessão de campo com aquele hardware.

Resposta do capítulo 41 no Apêndice B.

42. Laboratório

Objetivo de leitura

Planejar, registrar e comparar execuções de um experimento.

Antes de começar

Conheça funções, sementes e medições.

Um experimento executa uma operação sob configurações conhecidas e registra medições. Antes de executar, decida quais parâmetros variar, quantas repetições fazer e o que medir. Duas configurações com três repetições produzem seis execuções.

O planejamento ajuda a perceber o custo antes de iniciar. Depois, preserve resultados e contexto para repetir o procedimento e comparar as medições.

Da configuração ao resultado

Experimentos aceitam repetições, parâmetros, sementes e métricas. A implementação preserva os estados aleatórios globais do chamador e não encerra um `tracemalloc` que já estivesse ativo antes do experimento. Na 1.6.10, `planejar_experimento` calcula dimensões, combinações e execuções previstas antes da execução; `selecionar_medicoes` filtra resultados sem alterar o original; e `resumir_por_parametro` apresenta contagens, falhas, tempos e memória por cenário.

`validar_resultado` produz diagnósticos estruturados antes de persistir ou comparar. O JSON completo usa `coral.laboratorio.resultado/1`, esquema 1, e `carregar_resultado_json` reconstrói tipos portáteis e reconhece explicitamente o formato

histórico conhecido até a 1.6.9. CSV e Markdown continuam formatos de relatório, enquanto JSON é a representação recuperável. `comparar_experimentos` descreve diferenças de configuração, sucessos, tempo e memória usando `b - a`, mas não calcula significância estatística, causalidade nem superioridade geral.

Na 1.6.17, receitas procedurais podem ser planejadas e executadas como fator experimental. O plano registra quantidade de sementes, grade, repetições e custo preventivo. Cada medição preserva `semente_procedural`, enquanto o resultado guarda a receita base em metadados. Esse mecanismo mede configurações; ele não escolhe automaticamente a melhor semente nem transforma diferenças descritivas em conclusão científica.

Organizar um experimento

`coral.laboratorio` permite planejar, executar, filtrar, resumir, persistir e comparar experimentos reproduzíveis. O módulo registra e descreve resultados; a inferência estatística exige métodos próprios.

Planejar antes de executar

`planejar_experimento(parametros, repeticoes=1, limite=None)` calcula dimensões, combinações e execuções previstas sem construir toda a grade cartesiana. Quando `limite` é informado, o plano registra se o total o excede e em quanto. A função nunca executa o código estudado.

Selecionar e resumir medições

`selecionar_medicoes(resultado, criterio)` aceita uma função de decisão ou um mapeamento de igualdade. Use `parametros.nome` e `metricas.nome` para tornar a origem de uma chave explícita. A seleção preserva a ordem original e não altera o resultado.

`resumir_por_parametro(resultado, nome)` agrupa medições pelo valor do parâmetro e informa execuções, sucessos, falhas, tempo médio, tempo mediano e médias de memória. A ordem dos grupos segue a primeira aparição no resultado.

Validar, salvar e reabrir

`validar_resultado(resultado)` devolve diagnósticos estruturados para estrutura e portabilidade. JSONs novos usam o contrato `coral.laboratorio.resultado/1` e esquema 1. `carregar_resultado_json(caminho)` restaura resultados desse contrato e reconhece explicitamente o formato histórico conhecido até a 1.6.9; contratos ou esquemas desconhecidos são recusados. Valores portáteis, incluindo bytes, são reconstruídos em vez de convertidos silenciosamente para texto.

CSV e Markdown continuam exportações de leitura e análise externa. O JSON versionado é a representação completa destinada à recuperação de um `ResultadoExperimento`.

Comparar experimentos

`comparar_experimentos(a, b)` apresenta diferenças de configuração, quantidade de execuções, sucessos, falhas, tempos e memória. Diferenças numéricas são expressas como `b - a`. A função é deliberadamente descritiva: não calcula valor p, intervalo de confiança, causalidade nem superioridade geral. Para percentis, correlação ou outras operações numéricas, selecione as medições desejadas e use `coral.numerico`.

Superfície principal

`ResultadoExperimento.resumo()` continua sendo a visão geral. As operações novas complementam essa visão: `planejar_experimento`, `selecionar_medicoes`, `resumir_por_parametro`, `validar_resultado`, `carregar_resultado_json` e `comparar_experimentos`.

Projeto curto: Experimento com seis execuções

Planeje duas configurações e três repetições antes de executar. Defina como registrar falhas.

A orientação de solução está no Apêndice B, seção Projeto 42.

O que você aprendeu

Planeje o custo antes de executar. Preserve parâmetros e sementes; comparar resultados descritivamente não comprova causalidade ou superioridade geral.

Prática

Planeje um experimento com duas configurações e três repetições.

Conferir a resposta comentada

O plano contém seis execuções. Registre parâmetros, sementes, métricas e falhas; apresente diferenças como observações. Esse desenho não estabelece causalidade ou significância por si só.

Resposta do capítulo 42 no Apêndice B.

APÊNDICE A: Glossário

Termo	Significado	Primeira explicação
Algoritmo	Sequência de instruções para resolver uma tarefa.	Cap. 1
Argumento	Valor fornecido numa chamada de função.	Cap. 8
Assinatura	Forma que apresenta o nome da função, parâmetros e retorno declarado.	Cap. 8
Assíncrono	Modelo de execução em que uma tarefa pode suspender uma espera e permitir outras tarefas no ciclo de eventos.	Cap. 20
Atributo	Valor ou propriedade acessível num objeto.	Cap. 16
Backend	Implementação responsável por realizar uma capacidade, como renderização ou tradução.	Cap. 35
Booleano	Valor lógico: verdadeiro ou falso.	Cap. 10
Classe	Definição da estrutura e das operações de objetos.	Cap. 16
Coerção	Transformação automática de um valor para outro tipo.	Cap. 10
Coleção	Conjunto organizado de valores, como uma lista ou dicionário.	Cap. 14
Contrato	Regras de uso e comportamento que uma operação se compromete a manter.	Cap. 8
Dicionário	Coleção que relaciona chaves com valores.	Cap. 14
Efeito colateral	Mudança observável fora do retorno de uma função, como escrever num arquivo.	Cap. 26
Escopo	Região em que um nome pode ser consultado.	Cap. 15
Evento	Registro de um acontecimento ao qual uma regra pode reagir.	Cap. 32
Forma natural	Escrita oficial em português corrente equivalente a uma operação da linguagem ou biblioteca.	Cap. 8
Gerador	Função que entrega valores progressivamente com produza.	Cap. 19

Termo	Significado	Primeira explicação
Herança	Relação que permite a uma classe reutilizar a estrutura e o comportamento de uma base.	Cap. 16
Instância	Objeto concreto criado a partir de uma classe.	Cap. 16
JSON	Formato textual para valores estruturados portáteis.	Cap. 27
MRO C3	Regra que estabelece a ordem de procura de métodos em uma hierarquia de classes com herança múltipla.	Cap. 16
Mutação	Alteração de um objeto existente.	Cap. 25
Método	Operação associada a uma classe ou instância.	Cap. 16
Módulo	Unidade importável que reúne funções, classes e outros nomes.	Cap. 30
Nulidade	Possibilidade de um valor representar ausência com nulo.	Cap. 10
Parâmetro	Nome previsto na assinatura para receber um argumento.	Cap. 8
Persistência	Gravação de estado para uso em outra execução.	Cap. 29
Retorno	Valor devolvido por uma função ao chamador.	Cap. 15
Semente	Valor usado para inicializar uma fonte pseudoaleatória reproduzível.	Cap. 28
Sentinela	Valor interno especial usado para distinguir estados, como argumento omitido.	Cap. 8
Tipo parametrizado	Tipo que descreve também os elementos de uma coleção.	Cap. 10
Tupla estruturada	Tipo de tupla que informa o tipo de cada posição.	Cap. 10
Unicode	Padrão de representação de caracteres de diferentes sistemas de escrita.	Cap. 24
Valor padrão	Valor adotado quando um argumento opcional é omitido.	Cap. 8
API	Conjunto público de operações e contratos que um módulo oferece.	Cap. 8
CSV	Formato tabular textual que separa campos de cada registro com um delimitador.	Cap. 27

Termo	Significado	Primeira explicação
Callback	Função fornecida para ser chamada por outra operação quando um acontecimento ocorre.	Cap. 32
Codificação	Regra usada para representar texto como bytes.	Cap. 26
DAP	Protocolo que conecta o editor ao depurador.	Cap. 5
Depurador	Ferramenta para acompanhar execução, variáveis e pontos de parada.	Cap. 5
Esquema	Descrição da estrutura e versão dos dados persistidos.	Cap. 29
Frequência	Quantidade de ocorrências de um valor.	Cap. 25
Getter	Operação que fornece a leitura de uma propriedade.	Cap. 16
HMAC	Código de autenticação de mensagem baseado em hash e uma chave.	Cap. 28
HTTP	Protocolo de pedidos e respostas usado na comunicação Web.	Cap. 40
Hash	Resumo calculado a partir de dados; não equivale a cifragem.	Cap. 28
Imutável	Valor cuja estrutura não é alterada depois de sua criação.	Cap. 39
Iterável	Objeto que fornece elementos para uma passagem sequencial.	Cap. 14
LSP	Protocolo que conecta o editor ao servidor de linguagem.	Cap. 5
Loopback	Endereço de rede que se refere à própria máquina, como 127.0.0.1.	Cap. 40
Matriz	Organização retangular de valores em linhas e colunas.	Cap. 38
NaN	Representação numérica de um resultado que não é um número válido.	Cap. 38
NumPy	Biblioteca opcional usada em capacidades numéricas de Coral.	Cap. 38
Parâmetro nomeado	Parâmetro associado ao argumento pelo nome na chamada.	Cap. 15
Parâmetro posicional	Parâmetro associado ao argumento pela ordem na chamada.	Cap. 15
Plotly	Backend opcional para gráficos HTML interativos.	Cap. 39

Termo	Significado	Primeira explicação
Ponto de parada	Local em que o depurador interrompe temporariamente a execução para inspeção.	Cap. 5
Proveniência	Informação que registra origem e contexto dos dados.	Cap. 39
Pseudoaleatório	Sequência produzida por algoritmo a partir de um estado; pode ser repetida sob as mesmas condições.	Cap. 28
Recursão	Uso de uma função para resolver parte do problema por outra chamada à própria função.	Cap. 15
SVG	Formato vetorial de imagem que descreve formas e texto.	Cap. 39
Serial	Canal de comunicação em que dados são transmitidos sequencialmente.	Cap. 41
Setter	Operação que define o novo valor de uma propriedade.	Cap. 16
Subprocesso	Processo iniciado por outro programa para executar uma tarefa.	Cap. 41
TLS	Camada que verifica e protege uma conexão de rede quando configurada corretamente.	Cap. 40
Tupla	Sequência de posições cuja estrutura não é alterada como uma lista mutável.	Cap. 14
UTF 8	Codificação de texto Unicode usada por padrão em vários fluxos de arquivo.	Cap. 26
VSIX	Arquivo instalável de uma extensão do VS Code.	Cap. 5
Vetor	Sequência de componentes que representa posição, direção ou outra grandeza.	Cap. 36

APÊNDICE B: Respostas dos exercícios

Resposta do capítulo 1

Escreva um programa que mostre seu nome e o nome Coral em duas linhas.

Use duas instruções `mostre`, uma com o seu nome entre aspas e outra com `"Coral"`. Cada instrução termina sua saída com uma quebra de linha.

Resposta do capítulo 2

Escolha a instalação para uma máquina sem internet e identifique o que precisa transferir antes.

Transfira o pacote completo da release e, quando houver dependências opcionais, os pacotes dessas dependências para a plataforma alvo. O runtime portátil do pacote não precisa baixar dependências do núcleo. A instalação com pip só é offline quando todos os wheels necessários estão disponíveis.

Resposta do capítulo 3

Confirme no terminal a versão de Coral e execute um arquivo local.

Execute `coral --versao` e depois `coral programa.coral`. Se usar o runtime portátil, execute `python3 Instalacao/Runtime/coral-1.7.6.pyz --versao` e substitua `coral` por essa chamada.

Resposta do capítulo 4

Explique por que possuir um script Windows não comprova a qualificação daquela plataforma.

O script registra um fluxo de instalação, mas a qualificação exige sua execução na plataforma. A 1.7.5 considera Linux autoritativo e não promete evidência nativa Windows.

Resposta do capítulo 5

Localize a configuração de runtime da extensão e compare executar com depurar.

Executar inicia o programa no terminal. Depurar por F5 também conecta o adaptador e permite observar pontos de parada, pilha e variáveis. O runtime escolhido precisa corresponder à versão da aplicação.

Resposta do capítulo 6

Separe modelos e aplicação em arquivos diferentes e identifique a entrada no projeto.

Coloque classes e funções reutilizáveis em `modelos.coral` e importe os símbolos necessários em `principal.coral`. Configure o arquivo de entrada no `coral.toml`; esse caminho é independente do bloco opcional `programa principal`.

Resposta do capítulo 7

Altere a mensagem do primeiro programa e preveja a saída.

Troque apenas o texto entre aspas. O comando `mostre` imprime o novo texto; as aspas não aparecem na saída.

Resposta do capítulo 8

Diferencie parâmetro, argumento, padrão e retorno em uma ficha.

Parâmetro é o nome previsto pela função; argumento é o valor fornecido numa chamada; padrão é o valor usado quando um argumento opcional é omitido; retorno é o valor devolvido ao chamador.

Resposta do capítulo 9

Troque o valor inicial de `10` por `20` no exemplo de pontos. Antes de executar, preveja o resultado.

A saída será `23`: o programa começa com vinte pontos, adiciona cinco e retira dois. A ordem das instruções continua sendo a mesma.

Resposta do capítulo 10

Diferencie uma lista que pode ser nula de uma lista cujos itens podem ser nulos.

`lista de inteiro ou nulo` permite nulo no valor externo; `lista de (inteiro ou nulo)` permite nulo dentro da lista. Essas anotações não convertem nem validam profundamente os valores em execução.

Resposta do capítulo 11

Calcule o dobro da soma de 3 e 4, preservando a ordem desejada.

Use `(3 mais 4) vezes 2`. O resultado é 14. Parênteses tornam a ordem explícita.

Resposta do capítulo 12

Altere a média do exemplo para 6 e para 4; acrescente rejeição de valor negativo.

Para 6, o ramo de recuperação é escolhido; para 4, o de reprovação. Antes da comparação de aprovação, teste se a média é menor que zero e informe valor inválido.

Resposta do capítulo 13

Mostre os inteiros de 0 até antes de 3 e explique a terminação.

Use `para n de 0 até antes de 3 faça`, `mostre n` e `fim`. A saída é 0, 1 e 2 em linhas separadas; o limite superior não é visitado.

Resposta do capítulo 14

Diferencie alterar uma lista e produzir outra lista.

Uma alteração muda o objeto compartilhado por referências. Uma operação que produz nova lista preserva a original; confira o campo Atenção da ficha antes de assumir qualquer uma das duas situações.

Resposta do capítulo 15

Crie uma função que receba um inteiro e devolva seu dobro.

Declare `crie a função dobro com numero do tipo inteiro retornando inteiro`, use `retorne numero vezes 2` e feche com `fim`. Uma chamada com 21 devolve 42.

Resposta do capítulo 16

Explique a diferença entre instância, classe e atributo.

A classe descreve a estrutura e as operações; a instância é um objeto criado a partir dela; um atributo guarda parte do estado desse objeto. Duas instâncias podem manter estados diferentes.

Resposta do capítulo 17

Verifique se verdadeiro é do tipo inteiro.

O teste oficial `e_tipo(verdadeiro, "inteiro")` devolve falso. A superfície Coral não trata booleano como inteiro implícito.

Resposta do capítulo 18

Capture uma divisão por zero e imprima uma mensagem ao encerrar.

Use `tente`, a divisão, `se der erro do tipo ZeroDivisionError` com a mensagem, e `finalmente` para a mensagem final. O bloco final roda tanto no caso normal quanto na falha tratada.

Resposta do capítulo 19

Explique por que produzir números pode consumir menos memória que montar uma lista inteira.

Uma função geradora entrega um valor a cada solicitação e mantém apenas seu estado de execução. A economia existe quando o consumidor também processa os valores progressivamente.

Resposta do capítulo 20

Explique o papel de aguarde dentro de uma função assíncrona.

`aguarde` suspende a função até que a operação entregue um resultado, permitindo ao ciclo de eventos avançar outras tarefas. Não transforma um cálculo bloqueante em cálculo paralelo.

Resposta do capítulo 21

Coloque a função chamada abaixo do bloco programa principal. O que acontece ao importar o arquivo?

A execução direta inicializa o módulo inteiro e só depois inicia a entrada, portanto a função já está disponível. A importação disponibiliza os símbolos, mas não inicia o bloco principal.

Resposta do capítulo 22

Leia um nome e mostre uma saudação.

Leia uma linha com `ler_linha("Nome: ")` ou a forma natural equivalente. Guarde o texto e use `mostre "Olá, " mais nome`. A conversão é desnecessária para um nome.

Resposta do capítulo 23

Converta 12 e depois uma entrada inválida com padrão nulo.

`inteiro("12")` devolve 12. `inteiro("inválido", nulo)` devolve nulo. Sem o padrão, a entrada inválida produz `ErroConversao`.

Resposta do capítulo 24

Remova espaços externos de um texto e converta para maiúsculas.

`aparar(" Coral ")` devolve `"Coral"`; passar esse resultado para `maiusculas` devolve `"CORAL"`. O texto original não é modificado.

Resposta do capítulo 25

Remova valores repetidos de [3, 1, 3, 2] e ordene o resultado.

`unicos` produz [3, 1, 2], conservando a primeira ocorrência. `ordenar` produz [1, 2, 3]. A lista original permanece igual.

Resposta do capítulo 26

Escreva Coral num arquivo e leia o conteúdo.

Use `escrever_texto("dados.txt", "Coral")` e `ler_texto("dados.txt")`. A escrita tem efeito no sistema de arquivos e a leitura devolve texto. Considere permissão, codificação e pasta de trabalho.

Resposta do capítulo 27

Converta um dicionário para JSON e reconstrua o valor.

Use `para_json` para obter texto JSON e `de_json` para recuperar o dicionário. A ida e volta preserva os valores compatíveis com JSON, não objetos arbitrários ou identidade de instância.

Resposta do capítulo 28

Calcule a média de 2, 4 e 6.

A soma é 12 e a quantidade é 3, portanto a média é 4. `coral.matematica.media([2, 4, 6])` devolve 4.0. Não use uma coleção vazia.

Resposta do capítulo 29

Explique por que persistência não deve guardar callbacks e janelas ativas.

Esses objetos dependem de execução e recursos externos. O formato portátil guarda estado declarativo, com contrato e esquema, e reconstrói apenas os tipos que possuem adaptadores explícitos.

Resposta do capítulo 30

Diferencie arquivo de entrada, importação e bloco programa principal.

O projeto escolhe qual arquivo executar; a importação carrega um módulo; o bloco principal organiza a ação da execução direta. Bibliotecas reutilizáveis não devem iniciar a aplicação durante a importação.

Resposta do capítulo 31

Escreva uma verificação para uma função dobro e provoque uma falha intencional.

Use `garanta que dobro(3) for igual a 6`. Trocar 6 por 7 provoca a falha da verificação. Reponha o valor correto antes de considerar o teste aprovado.

Resposta do capítulo 32

Diferencie evento, regra e relógio de teste.

O evento registra um acontecimento; a regra escolhe como reagir; o relógio de teste permite controlar o tempo explicitamente. Uma passagem de tempo simulada não deve depender de espera real.

Resposta do capítulo 33

Modele dois objetos e uma relação fora do domínio de jogos.

Um laboratório pode modelar equipamentos e medições, com uma relação indicando o equipamento que gerou cada medição. Mundo oferece entidades e relações sem exigir janela ou combate.

Resposta do capítulo 34

Identifique os dados que devem ser reproduzíveis numa simulação de combate.

Estado inicial, participantes, regras, ordem de turno e semente ou resultados das jogadas precisam ser controlados. Uma vitória observada numa única execução não comprova equilíbrio.

Resposta do capítulo 35

Explique a vantagem de testar um jogo com BackendNulo.

O backend permite exercitar o estado e o fluxo sem janela nem renderização física. Ele não comprova desempenho gráfico ou funcionamento do dispositivo real.

Resposta do capítulo 36

Diferencie um vetor de posição e uma transformação em 3D.

O vetor representa coordenadas; a transformação especifica como alterar coordenadas. Em matrizes homogêneas, pontos e direções têm papéis diferentes e não devem ser confundidos.

Resposta do capítulo 37

Explique como semente e receita ajudam a reproduzir um campo procedural.

A semente controla o gerador e a receita registra algoritmo, versão e parâmetros. A reconstrução só é garantida para operações e versões suportadas; callbacks arbitrários não viram receita portátil.

Resposta do capítulo 38

Explique por que correlação com coluna constante é inválida.

O cálculo divide pela dispersão de cada variável. Se uma coluna não varia, essa dispersão é zero e a correlação não pode ser definida pelo contrato; a biblioteca produz diagnóstico.

Resposta do capítulo 39

Separe criar uma especificação de gráfico e salvar sua imagem.

Criar `linha` ou `dispersao` descreve os dados sem abrir janela. `salvar` escolhe o formato e renderiza. A disponibilidade do backend depende do formato solicitado.

Resposta do capítulo 40

Diferencie HTML local, cliente HTTP e servidor HTTP.

HTML local é conteúdo de apresentação; o cliente faz pedidos; o servidor responde a pedidos. Gerar HTML não abre conexão nem inicia um servidor.

Resposta do capítulo 41

Explique por que inventário simulado não comprova comunicação com uma placa física.

A simulação verifica o contrato do programa sem exercer cabo, porta, driver ou dispositivo. A comunicação física exige uma sessão de campo com aquele hardware.

Resposta do capítulo 42

Planeje um experimento com duas configurações e três repetições.

O plano contém seis execuções. Registre parâmetros, sementes, métricas e falhas; apresente diferenças como observações. Esse desenho não estabelece causalidade ou significância por si só.

Projeto 30

Cadastro em três arquivos. Use modelos.coral para as classes, principal.coral para a entrada e um arquivo de teste para as verificações. Importe nomes explícitos; a importação de modelos não deve iniciar a aplicação.

Projeto 31

Contrato de uma função. Os resultados são 0, 6 e menos 6 para entradas 0, 3 e menos 3. A conversão do texto inválido deve ser tratada na borda de entrada, e não mascarada no cálculo de dobro.

Projeto 32

Alerta de estoque. O estado guarda a quantidade; a condição da regra compara a quantidade com três. Teste a transição de 3 para 2 e a permanência abaixo do limite, respeitando a política de repetição da regra.

Projeto 33

Mapa de ocupação. Coloque um bloco numa célula da camada e consulte a mesma posição; uma célula sem bloco representa ausência conforme o contrato do mapa. A visualização deve ficar separada da ocupação.

Projeto 34

Duelo reproduzível. Crie um mundo, adicione os personagens, inicie o combate com a semente e registre cada alteração. Para comparar, reconstrua o estado inicial e use a mesma semente; não reutilize o estado final da execução anterior.

Projeto 35

Movimento sem janela. Crie o jogo sem janela, forneça uma entrada simulada e execute com max_quadros=2. Verifique deslocamento e contagem de quadros. A prova de renderização física permanece separada.

Projeto 36

Distâncias no espaço. Use coordenadas alinhadas e a mesma unidade. Pontos (0, 0) e (3, 4) distam 5; em 3D, (0, 0, 0) e (0, 0, 5) também distam 5. Esses resultados não tornam as duas representações intercambiáveis.

Projeto 37

Campo com receita. Registre algoritmo, versão, semente e parâmetros suportados. Consultas determinísticas do mesmo campo devem preservar os valores independentemente da ordem; não introduza callbacks não portáteis na receita.

Projeto 38

Resumo de medições. A média é 23 e a amplitude é 6. O contrato numérico rejeita ausência ou valores não finitos quando não há uma política explícita de seleção; não transforme dados ausentes em zero.

Projeto 39

Gráfico de temperatura. Configure a especificação com rótulos e unidades antes de salvar. SVG 2D usa o backend local da biblioteca; a criação da figura não abre janela. Confirme que o arquivo existe e que os dados de origem permanecem iguais.

Projeto 40

Página de estado local. Altere o corpo de resposta_texto e mantenha o encerramento em finalmente. A resposta deve ter status 200 e o texto escolhido. Não remova o limite de resposta ou exponha o servidor externamente por acidente.

Projeto 41

Relatório de capacidades. Use quantidade_processadores e programa_disponivel. Disponibilidade não significa que o programa executou, e quantidade de processadores não é uma promessa de recursos efetivos sob quotas.

Projeto 42

Experimento com seis execuções. O plano deve prever seis execuções. Preserve parâmetros, sementes, sucessos e falhas. Compare os resultados descritivamente; não declare significância ou superioridade geral a partir de uma diferença de tempo isolada.

APÊNDICE C: Índice remissivo

Recurso	Localização
O que é Coral	Cap. 1
Instalando Coral	Cap. 2
Linux	Cap. 3
Outras plataformas suportadas	Cap. 4
VS Code	Cap. 5
Projetos Coral	Cap. 6
Primeiro programa (passo a passo)	Cap. 7
Como ler exemplos e fichas de API	Cap. 8
Valores e variáveis	Cap. 9
Tipos e tipagem opcional	Cap. 10
Operadores e expressões	Cap. 11
Condições	Cap. 12
Repetições	Cap. 13
Coleções, índices e fatias	Cap. 14
Funções e escopo	Cap. 15
Classes e objetos	Cap. 16
Verificando tipos na prática	Cap. 17
Erros, padrões e contextos	Cap. 18
Geradores	Cap. 19
Assíncrono	Cap. 20
Programa principal	Cap. 21
Entrada e saída	Cap. 22
Conversões	Cap. 23
Texto	Cap. 24
Coleções	Cap. 25
Arquivos e caminhos	Cap. 26
JSON e serialização	Cap. 27
Matemática, datas e aleatoriedade	Cap. 28
Persistência	Cap. 29

Recurso	Localização
Projetos, módulos e coral.toml	Cap. 30
Testes	Cap. 31
Regras, eventos e tempo	Cap. 32
Mundo	Cap. 33
RPG	Cap. 34
Jogos	Cap. 35
Espaço e geometria	Cap. 36
Procedural	Cap. 37
Computação numérica e científica	Cap. 38
Gráficos	Cap. 39
Web	Cap. 40
Sistema e hardware	Cap. 41
Laboratório	Cap. 42
Algoritmo	Cap. 1; Apêndice A
Argumento	Cap. 8; Apêndice A
Assinatura	Cap. 8; Apêndice A
Assíncrono	Cap. 20; Apêndice A
Atributo	Cap. 16; Apêndice A
Backend	Cap. 35; Apêndice A
Booleano	Cap. 10; Apêndice A
Classe	Cap. 16; Apêndice A
Coerção	Cap. 10; Apêndice A
Coleção	Cap. 14; Apêndice A
Contrato	Cap. 8; Apêndice A
Dicionário	Cap. 14; Apêndice A
Efeito colateral	Cap. 26; Apêndice A
Escopo	Cap. 15; Apêndice A
Evento	Cap. 32; Apêndice A
Forma natural	Cap. 8; Apêndice A
Gerador	Cap. 19; Apêndice A
Herança	Cap. 16; Apêndice A

Recurso	Localização
Instância	Cap. 16; Apêndice A
JSON	Cap. 27; Apêndice A
MRO C3	Cap. 16; Apêndice A
Mutação	Cap. 25; Apêndice A
Método	Cap. 16; Apêndice A
Módulo	Cap. 30; Apêndice A
Nulidade	Cap. 10; Apêndice A
Parâmetro	Cap. 8; Apêndice A
Persistência	Cap. 29; Apêndice A
Retorno	Cap. 15; Apêndice A
Semente	Cap. 28; Apêndice A
Sentinela	Cap. 8; Apêndice A
Tipo parametrizado	Cap. 10; Apêndice A
Tupla estruturada	Cap. 10; Apêndice A
Unicode	Cap. 24; Apêndice A
Valor padrão	Cap. 8; Apêndice A
Chamadas multilinha	**Cap. 15, seção Chamadas explícitas em várias linhas**
Pertencimento a classe	Cap. 17, seção Consultar classes do programa
Celula	Cap. 16, seção Exemplo: uma célula que guarda conteúdo
Snapshots do editor	Volume II, Implementação e arquitetura

APÊNDICE D: Migração desde a Coral 1.7.0

A 1.7.2 acrescenta expressões naturais para entrada, conversões, texto, coleções, JSON e consulta de tipos. As APIs explícitas permanecem válidas. A mudança é opcional e não exige reescrever programas existentes. Consulte os Caps. 17 e 22 a 27. Os diagnósticos de execução passam a manter uma classificação comum entre terminal e depurador; consulte o Cap. 18.

A 1.7.3 acrescenta coleções parametrizadas, tuplas estruturadas e nulidade opcional nas anotações, sem obrigar o uso de tipos. Atualize anotações onde elas tornarem a intenção mais clara e mantenha testes de dados recebidos. Consulte o Cap. 10.

Herança múltipla e chamadas cooperativas seguem MRO C3. Uma chamada ao pai continua a cadeia; ela não seleciona indiscriminadamente todas as bases. Construtores precisam cooperar explicitamente. Consulte o Cap. 16, seção Para ir além.

O bloco `programa principal` pode organizar a entrada de uma aplicação e permite declarar suas funções abaixo. Ele começa apenas na execução direta do módulo; scripts lineares continuam válidos. Consulte o Cap. 21.

A coleção deixa de ser um único volume. As referências internas passam a indicar capítulo e seção. Esta revisão preserva os livros anteriores e usa fontes compartilhadas para os dois volumes e a projeção local do site.

Migração para a Coral 1.7.4

Instale Coral Language 0.75.2 e confirme o runtime 1.7.4. As formas de escrever classes, imports e `programa principal` permanecem disponíveis. Revise novas advertências entre módulos usando as assinaturas reais. O realce semântico acompanha o tema e pode ser desativado. Ao reutilizar um projeto antigo, permita que o build regenere o cache de contratos.

Migração para a Coral 1.7.6

Instale Coral Language 0.77.0 e confirme o runtime 1.7.6. A forma `objeto for do tipo Classe` substitui a chamada comum `e_tipo(objeto, Classe)` quando isso melhorar a leitura; a API continua compatível. Consulte o Cap. 17, seção Consultar classes do programa.

Chamadas longas podem usar `chame alvo com`, argumentos indentados e `fim`; a forma compacta permanece válida. Consulte o Cap. 15, seção Chamadas explícitas em várias linhas. Essa mudança não exige migrar o código existente.

Os dois volumes são a edição oficial corrente. O livro único da 1.7.0 é histórico e permanece disponível para compatibilidade. Os relatórios da coleção registram a validação da edição e as lacunas normativas anteriores que ainda precisam de revisão.

APÊNDICE E: Mapa de recursos novos

Release	Recurso	Localização
1.7.2	Formas naturais	Cap. 8; Caps. 22 a 27
1.7.2	Entrada	Cap. 22
1.7.2	Conversões	Cap. 23
1.7.2	Texto	Cap. 24
1.7.2	Coleções	Cap. 25
1.7.2	JSON	Cap. 27
1.7.2	Diagnósticos	Cap. 18
1.7.3	Tipos parametrizados	Cap. 10
1.7.3	Tuplas e nulidade	Cap. 10, seção Para ir além
1.7.3	Herança múltipla e MRO C3	Cap. 16, seção Para ir além
1.7.3	Chamadas e inicialização do pai	Cap. 16, seção Para ir além
1.7.3	Programa principal	Cap. 21

Release	Recurso	Localização
1.7.4	Realce semântico e opções do editor	Cap. 5
1.7.4	Tipagem entre módulos	Cap. 10 e Cap. 30
1.7.4	Contratos transitivos e reexportações	Cap. 30; Volume II, Implementação e arquitetura
1.7.4	Manifesto inicial e ícone coral.toml	Cap. 5 e Cap. 30

Release	Recurso	Localização
1.7.5	Consulta natural de classes e ancestrais	Cap. 17; Cap. 16, exemplo Celula
1.7.5	Chamadas explícitas multilinha	Cap. 15; Volume II, Referência da linguagem
1.7.5	Ajuda de assinatura e autocomplete de tipos	Cap. 5
1.7.5	Snapshots e índice seletivo	Volume II, Implementação e arquitetura
1.7.6	Parâmetros, imports e bases em listas explícitas	Cap. 15, 16 e 30
1.7.6	Ambiente isolado, wheels e lock por alvo	Cap. 30; Volume II, CLI